

INTERVIEW_GUIDE

Helios - Interview Guide

Autonomous Growth Diagnosis Engine - the definitive prep

This guide is the single source of truth for defending Helios in placement interviews for Product, Growth, Data, Business, Analytics-Engineering, and AI/Data-Engineering roles. It pairs the concepts an interviewer probes with the exact Helios decision that proves you have done the work - so every answer moves from a general principle to concrete, quantified evidence.

How to use this guide

Study order. Read the Fact Sheet below until you can reproduce it on a whiteboard from memory - the decomposition identity, the eval numbers, and the three pillars are non-negotiable. Then work the sections in order: the early sections build the problem framing and architecture mental model; the middle sections drill the technical centerpiece (mix-shift vs rate-change), the statistics, and the trust story; the later sections cover honesty/limitations, role positioning, and rapid-fire rebuttals. Skim everything once end-to-end before deep-drilling, so you can cross-reference.

How to drill the Q&A. Each Q&A is built as: (a) the concept stated generally, (b) the tradeoff and the failure mode it avoids, (c) the "Why Helios demonstrates it" tie-back to a specific named component. Do not memorize answers verbatim - internalize the *structure* so you can recombine pieces when a question comes in a shape you did not rehearse. For every question, force yourself to name a number, name an alternative you rejected, and name the component.

The rehearse-out-loud rule. Reading is not rehearsing. Say every answer aloud, end to end, under 90 seconds, before you consider it learned. If you cannot say it without filler or without inventing a number, you do not know it yet. Record yourself once per section; an answer you cannot deliver fluently out loud will collapse under follow-up pressure.

Table of Contents

- 1. The Problem and the Insight-to-Action Gap
- 2. What Helios Is (and Is Not)
- 3. System Architecture and the MCP Server Design
- 4. The Agent State Machine and Orchestration
- 5. The Technical Centerpiece - Mix-Shift vs Rate-Change Decomposition
- 6. The Semantic Layer and Governed SQL
- 7. The Data Model (dbt, GA4, Sessionization)
- 8. Statistics, Forecasting, and Experiment Design
- 9. The Critic, Adversarial Verification, and the Offline Benchmark
- 10. Memory and the Did-the-Fix-Work Loop
- 11. Honesty, Limitations, and Senior Judgment
- 12. Role-Specific Positioning and Behavioral Framing
- 13. Rapid-Fire Rebuttals and Hard Follow-Ups

The Helios Fact Sheet (memorize this)

One-line thesis. Helios is an always-on AI Growth Analyst that diagnoses *why* an e-commerce funnel moved, quantifies revenue-at-risk in dollars, and ships a prioritized, statistically-defensible experiment backlog - all grounded in governed SQL it never hand-authors.

The problem in one line. Dashboards report *what* happened, not *why*; manual root-cause takes ~1-3 analyst-days per anomaly, routinely confuses mix-shift with rate-change, and rarely carries dollar impact or a prescribed experiment.

The architecture in 5 lines:

Layer	What it is
5 MCP servers	warehouse-mcp (sole BigQuery client; dry_run -> run_query byte gate; reconcile <=0.5%), semantic-mcp (ONLY SQL path; build_query from governed registry), stats-mcp (ONLY math path), experiment-mcp (power/runtime/design), report-mcp (render_brief + memory)
7 agents	Orchestrator (Opus), Monitor (Sonnet), Decompose (Sonnet), Diagnose (Opus), Critic (Opus), Prescribe (Sonnet), Narrator (Sonnet) - on a deterministic FSM; LLM composes tool calls, never controls flow
Semantic layer	semantic_layer.yaml: 47 metrics + 19 dimensions, referential-integrity compiled, maps 1:1 to dbt MetricFlow - single source of truth for all SQL

Layer	What it is
Memory	BigQuery helios_memory + vector store: diagnosis history (embeddings), suppression list, seasonality/launch calendars, action-tracking loop
Evaluation	Offline labeled benchmark: synthetic anomalies injected into a frozen GA4 copy; the pipeline must rediscover them

The decomposition identity (write it cold):

$\Delta R = \text{mix} + \text{rate} + \text{interaction}$ where $R = \sum_i (w_i * r_i)$, $\text{mix} = \sum_i (dw_i * r_i(t_0))$, $\text{rate} = \sum_i (w_i(t_0) * dr_i)$, $\text{interaction} = \sum_i (dw_i * dr_i)$ (w = segment volume share, r = segment rate). Computed deterministically by stats-mcp.decompose_change. Drill rate-effects (real behavior) before mix-effects (composition artifacts) - this dissolves Simpson's paradox.

Numbers to never forget:

Metric	Value
Root-cause segment accuracy	>=85% vs <=45% naive baseline
Hallucinated columns	0
Decomposition MAPE	<=10%
Runtime	<5 min/run
BigQuery budget	5 GiB fixed
Labeled scenarios	50 across 7 buckets
Semantic registry	28 metrics / 16 dimensions
Reconciliation tolerance	<=0.5%

The three defensibility pillars: (1) **governed-SQL grounding** - semantic-mcp is the only SQL path (0 hallucinated columns); (2) **deterministic statistics** - stats-mcp is the only math path; (3) **adversarial verification + offline eval** - the Critic plus the 85%-vs-45% benchmark. Thesis: the hard part is not generating insights, it is making them *correct and trusted*.

The four honesty points: (1) dataset is **observational & historical** (Nov 2020-Jan 2021), so Helios *designs and sizes* experiments and runs quasi-experimental readbacks (pre/post, diff-in-diff) - it does not run live A/Bs; (2) **~3-month window** -> true LTV deferred, report short-horizon

proxies with stated assumptions; (3) **user_id almost always null** -> identity is cookie-grain (user_pseudo_id), user-level metrics are approximations; (4) **traffic_source gotcha** -> event-level source is user FIRST-TOUCH not session source, so Helios derives session-scoped source/medium and falls back to first-touch.

The canonical funnel: session_start -> view_item -> add_to_cart -> begin_checkout -> add_shipping_info -> add_payment_info -> purchase. Revenue = purchase_revenue_in_usd. Session = (user_pseudo_id, ga_session_id). Dataset = bigquery-public-data.ga4_obfuscated_sample_ecommerce.

The 10 GA4 channel groups: Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other.

One-sentence positioning per role

- **Product Analyst:** "I built a system that goes past *what* a funnel did to *why* it moved - decomposing each conversion-step drop into mix vs rate, attaching dollar revenue-at-risk, and prescribing the experiment to fix it."
 - **Growth Analyst:** "Helios turns an anomaly into a prioritized, power-sized experiment backlog with quantified upside, so growth bets are ranked by defensible impact rather than by whoever argues loudest."
 - **Data Analyst:** "I automated the 1-3-day root-cause grind into a <5-minute run that never hallucinates a column, because every query is composed from a governed 28-metric semantic registry and reconciled to $\leq 0.5\%$."
 - **Analytics Engineer:** "I built a GA4 -> dbt warehouse - staging, sessionized intermediate, monotonic funnel flags, marts - with a semantic layer mapping 1:1 to MetricFlow as the single source of truth for every downstream metric."
 - **AI/Data Engineer:** "I orchestrated 7 Claude agents on a deterministic finite-state machine over 5 MCP servers, where the LLM only composes governed tool calls and never controls flow or authors SQL - LLM judgment, deterministic guarantees."
 - **Founder/Consulting:** "I shipped an autonomous analyst that closes the insight-to-action gap end-to-end - diagnosis, dollar impact, prescription, and an executive Decision Brief - and proved it on a 50-scenario labeled benchmark, not a demo."
-

Answer voice reminder

Three rules govern every answer in this guide - obey them out loud, not just on paper:

1. **Concept first, then Helios.** Explain the general principle and why it matters *before* citing Helios. Never talk only about the project - show you understand the idea independently, then use Helios as the concrete proof.
 2. **Quantify + tradeoff + honesty.** State a number, name the alternative you rejected and the tradeoff, and surface the relevant limitation. Honesty about constraints (observational data, cookie-grain identity, 3-month window) reads as senior judgment, not weakness.
 3. **Tie to a specific component.** Close with a "Why Helios demonstrates it" line that names the exact server, agent, or artifact - semantic-mcp, the Critic, decompose_change, semantic_layer.yaml - using canonical names accurately. Tight, high-signal prose; no buzzword soup.
-

Helios â€™ Story & Elevator Pitches

Section 1: Project Story

The same project, told five ways. Each version is tuned to one audience's attention budget and the one thing that audience is actually deciding. Use the version that matches the room; do not blend them.

Resume (1-2 lines)

Helios â€™ Autonomous AI Growth Analyst (BigQuery Â· dbt Â· Claude Agent SDK Â· MCP).

Built a self-running, seven-agent diagnosis engine that distinguishes mix-shift from rate-change, prices funnel movement as dollars of revenue-at-risk, and prescribes a power-analyzed experiment backlog from governed SQL â€™ **85%+ root-cause accuracy vs a 45% naive baseline**, graded automatically in CI, with **0 hallucinated columns**.

Recruiter screen (~30 seconds, plain language)

"Helios is an AI system I built that acts like an always-on growth analyst for an online store. When a sales funnel moves â€™ say checkout conversion drops â€™ a human analyst normally spends one to three days figuring out *why*. Helios does it in under five minutes, on a schedule, without anyone asking. It figures out the real reason behind a number changing, puts a dollar value on it, and recommends the one experiment most worth running. I built it on Google's public e-commerce analytics dataset using BigQuery, dbt, and Claude's agent framework. The headline result: it gets the root cause right 85% of the time versus 45% for the obvious naive approach, and I prove that with an automated test suite that runs on every code change."

Technical interview (~2 minutes, architecture + trust)

"Helios is an autonomous growth-diagnosis engine. The product loop is: detect an anomaly in a funnel metric, decompose *why* it moved, diagnose the root cause, verify it adversarially, size a fix, and ship an executive Decision Brief" all on a schedule.

The technical centerpiece is a **mix-shift versus rate-change decomposition**. Any aggregate rate $R = \sum w \cdot r$ over segments, where w is each segment's volume share and r its rate. The change ΔR splits exactly into $\Delta R = \sum w \cdot \Delta r(t_0)$, $\Delta R = \sum \Delta w \cdot r(t_0)$, and an interaction term. That tells you whether conversion fell because in-segment behavior got worse, or merely because traffic composition shifted toward lower-converting segments. It's how I dissolve Simpson's paradox "the aggregate can fall while every segment rises."

The trust story is three structural pillars. First, **governed-SQL grounding**: the LLM never authors raw SQL. It composes governed metrics through a semantic layer "semantic-mcp is the *only* path to SQL" so hallucinated columns are structurally impossible, not just discouraged. Second, **deterministic statistics**: all decomposition, significance testing, power, and forecasting run in real Python via `stats-mcp`; nothing statistical is generated in token-space. Third, **adversarial verification plus offline eval**: a Critic agent tries to refute every finding "mix confound, insufficient sample, seasonality, data-quality artifact" and a 50-scenario labeled benchmark grades root-cause accuracy in CI on every commit.

Architecturally that's five MCP servers as a capability/trust boundary and seven agents on a deterministic finite-state machine. The LLM composes tool calls; it never controls flow."

Hiring manager round (~2 minutes, impact + collaboration + decisions)

"The problem Helios attacks is the insight-to-action gap. Every growth org has dashboards that tell you *what* happened, but the expensive question is *why*, and that still routes to a human analyst for one to three days per anomaly. That analyst is a serial, manually-triggered bottleneck "and they routinely confuse a traffic-mix change with a real behavior change, which funds the wrong fix."

Helios inverts that. Diagnosis becomes continuous and proactive: every scheduled run re-derives the funnel, flags what moved, separates mix from rate, prices it in revenue-at-risk, and ships a brief the head of growth can forward to the CEO "before anyone asks. I designed it around four real personas: the Head of Growth who owns the revenue number, the PM who owns a funnel surface, the analyst who's tired of being the human RCA oracle, and the founder who just wants the three things that moved revenue and the plan."

The decision I'm proudest of is making trust *measurable* instead of asserted. 'The AI says so' isn't shippable, so I built a labeled benchmark and wired it into GitHub Actions: every commit grades diagnosis accuracy, and a regression that drops top-1 accuracy by more than two points or introduces any hallucination fails CI. The hardest tradeoff was scope discipline â€” it would have been easy to drift into a dashboard or a chatbot. I held a hard anti-product line: conversation is a secondary drill-down surface; the heartbeat is the autonomous run."

Founder round (~2 minutes, business + ROI + what next)

"Helios creates a category rather than competing in one. Descriptive BI tells you *what*; conversational text-to-SQL tools answer one human-prompted question at a time and frequently hallucinate; product analytics gives you funnels with some *who*. None of them autonomously answer *why*, *how much in dollars*, and *what to do next*. That white space â€” autonomous, governed, statistically-defensible, dollar-quantified diagnosis â€” is the wedge.

The ROI math is direct. Manual root-cause analysis costs one to three analyst-days per anomaly, and a senior analyst's funnel is dozens of metrics across many segments every week. Helios takes that to under five minutes per run on a fixed BigQuery byte budget â€” and, more importantly, it stops misdiagnosis. Every week a depressed conversion rate goes undiagnosed is compounding lost revenue; every wrong mix-vs-rate call funds the wrong engineering work. Helios converts that invisible liability into a visible, prioritized, dollar-denominated backlog.

The wedge is deliberately narrow so it can be excellent and verifiable, but the expansion path is clear: multi-tenant productionization, warehouse-agnostic adapters behind the semantic layer, and eventually true causal inference â€” difference-in-differences and uplift modeling â€” plus a closed loop that pushes experiment cards to a testing platform, reads back the result, and updates the backlog. I'm honest that the public dataset is observational and three months long, so today Helios *designs and sizes* experiments and runs quasi-experimental readbacks rather than live A/Bs. The architecture is built to absorb live experimentation the moment the data supports it."

Why Helios exists â€” the core arguments

Why Helios exists. Modern e-commerce orgs are drowning in descriptive signal and starving for diagnostic answers. A dashboard can show `session_conversion_rate` fell from 2.1% to 1.7%, but cannot say why, cannot price it, and cannot prescribe a fix. Helios exists to close that insight-to-action gap â€” to make the question "why did the funnel move?" answerable by an artifact that already exists, is grounded in governed SQL, is defended statistically, and has survived an adversarial critique, rather than by a Slack ping that kicks off two days of ad-hoc querying.

What problem it solves. It automates root-cause analysis of the conversion funnel and makes every finding actionable. Concretely: detect the anomaly, decompose mix vs rate vs interaction, verify the root cause against governed data, quantify revenue-at-risk in dollars, and prescribe a powered experiment “collapsing a one-to-three-day manual process to under five minutes, while structurally preventing the two failure modes that wreck manual RCA: hallucinated metrics and confusing composition change with behavior change.

Why dashboards are insufficient. Dashboards report *what*, never *why* “no causal-style attribution, no mix/rate split. They are pull, not push: someone must notice and ask. They are blind to interaction confounds across dimensions like `channel_group` ~ `device_category`. They carry no dollar quantification, so a percentage-point drop never becomes a budget decision. And they prescribe nothing “no experiment, no power analysis, no prioritized backlog. A dashboard is the *input* to diagnosis, not diagnosis.

Why autonomous diagnosis matters. A human analyst is a serial, reactive oracle queried only after someone notices a number looks off “which can be days later. Autonomy inverts that latency: diagnosis becomes a continuous background process that runs whether or not anyone is watching, the way CI runs on every commit. Proactive beats reactive because the cost of an undiagnosed week compounds, and because a scheduled, governed, repeatable run is something a team can *trust* in a way they can never trust an ad-hoc one-off.

Why MCP was chosen. Model Context Protocol lets me split capability across servers and enforce hard boundaries the LLM cannot cross. `semantic-mcp` is the *only* path to SQL and `stats-mcp` the *only* path to math, with per-agent tool allow-lists “the Narrator literally cannot call `run_query`. That capability isolation is the trust argument made structural: hallucinated columns and hand-computed statistics aren't policed after the fact, they're impossible by construction. MCP turns “please don't write bad SQL” into “you cannot write SQL at all.”

Why an agent architecture was chosen. Diagnosis is genuinely multi-step and benefits from specialization and adversarial review “a single monolithic prompt can't both confidently propose a root cause *and* rigorously refute it. So I use seven agents on a deterministic finite-state machine: the LLM composes tool calls at each node, but the *flow* is code, not model-controlled. That gives me the reasoning flexibility of agents (a best-first hypothesis tree, an adversarial Critic that can downgrade or drop a finding) with the reproducibility and cost control of a fixed pipeline “Opus for orchestration and critique where judgment matters, Sonnet for high-volume sub-tasks.

Section 2: Elevator Pitches

Five ready-to-speak scripts. Each tier strictly adds depth on top of the previous one – pick by the time you actually have.

30 seconds – thesis + wedge

"Helios is an autonomous AI Growth Analyst. Instead of another 'ask your data' chatbot, it runs on a schedule and proactively diagnoses *why* an e-commerce funnel moved – and it nails the one thing analysts get wrong: it separates a real behavior change from a mere shift in traffic mix. Every finding ships with a significance test, a dollar revenue-at-risk number, and a recommended experiment. On a labeled benchmark it hits 85%+ root-cause accuracy versus 45% for the naive approach."

1 minute – the problem + how it works at a glance

"Helios is an autonomous AI Growth Analyst. The problem it solves: when a funnel metric moves, a human analyst spends one to three days finding the root cause – and routinely confuses *mix-shift* (traffic composition changed) with *rate-change* (in-segment behavior changed), which is Simpson's paradox and funds the wrong fix.

Helios runs on a schedule and, in under five minutes, does the whole loop: detect the anomaly, decompose the move into mix versus rate versus interaction, diagnose the root-cause segment, price it in dollars of revenue-at-risk, and prescribe a powered experiment – then ships an executive Decision Brief before anyone asks. The decomposition is the centerpiece: it tells you whether conversion dropped because behavior got worse or because your traffic mix shifted toward lower-converting segments. On a labeled benchmark, 85%+ root-cause accuracy versus a 45% naive baseline."

3 minutes – architecture + the trust pillars

"[Open with the 1-minute pitch, then continue:]

Under the hood it's five MCP servers and seven agents. The five MCP servers are the trust boundary. `warehouse-mcp` is the sole BigQuery client, with a mandatory dry-run byte-budget gate before any query and a reconcile check against canonical totals. `semantic-mcp` is the *only* path to SQL – it composes validated queries from a governed registry of 47 metrics and 19 dimensions, so the model references metric *names* like `view_to_cart_rate`, never raw columns. `stats-mcp` is the *only* path to math – decomposition, significance, forecasting, cohort, RFM. `experiment-mcp` does power analysis and experiment design. `report-mcp` renders the brief and handles memory.

The seven agents run on a deterministic state machine: an Orchestrator on Opus; a Monitor that detects anomalies; a Decompose agent that runs the mix/rate math; a Diagnose agent on Opus that builds a best-first hypothesis tree and SQL-verifies each branch; a Critic on Opus that adversarially tries to refute every finding; a Prescribe agent that builds experiment cards; and a Narrator that writes the brief. The LLM composes the tool calls â€” it never controls the flow.

That gives me three defensibility pillars. One, **governed-SQL grounding** â€” the semantic layer is the only SQL author, so hallucinated columns are structurally impossible. Two, **deterministic statistics** â€” all math is real code, never generated tokens. Three, **adversarial verification plus offline eval** â€” the Critic refutes findings, and a labeled benchmark grades accuracy in CI. The thesis is that the hard part of an AI analyst isn't generating insights; it's making them correct and *trusted*."

5 minutes â€” + the eval, results, and honest tradeoffs

"[Open with the 3-minute pitch, then continue:]

The reason I can claim 85% isn't a vibe â€” it's an offline labeled benchmark. The public GA4 dataset has real anomalies but they're *unlabeled*, so I can't grade against them. Instead I clone a baseline window into an eval dataset and surgically inject anomalies with a known ground truth, then make the pipeline rediscover what I hid. A *rate* perturbation multiplies one segment's conversion rate while holding its volume fixed â€” ground truth is rate-change. A *mix* perturbation multiplies a segment's sessions while holding rates fixed â€” ground truth is mix-shift. Because totals are conserved, the analytic mix/rate/interaction split is exact and becomes the gold target.

The benchmark is 50 scenarios across seven buckets: single- and multi-segment rate changes, single- and multi-segment mix shifts, seasonality decoys the system must *not* flag, no-anomaly controls that test false-positive rate, and data-quality artifacts the Critic must catch as data issues rather than behavior. The contract: top-1 root-cause accuracy â‰¥85% versus â‰¤45% for a naive 'largest absolute segment delta' baseline; decomposition error â‰¤10% MAPE; 0 hallucinated columns; 100% of findings carrying a significance test and a dollar figure; under five minutes per run; and cost under a fixed 5 GiB BigQuery byte budget. CI gates regressions on every PR.

Now the honest tradeoffs, because they read as maturity. The dataset is observational and historical â€” about three months, late 2020 â€” so Helios *designs and sizes* experiments and runs quasi-experimental readbacks like difference-in-differences; it does not run live A/Bs. That short window means true LTV and long-horizon retention are deferred to short-horizon proxies with stated assumptions. `user_id` is almost always null, so identity is cookie-grain and user-level metrics are explicit cookie approximations. And there's a classic GA4 gotcha: event-level `traffic_source` is *first-touch*, not session source â€” I derive session-scoped source/medium

and fall back to first-touch only when null. I'd rather state these limits plainly than have an interviewer find them."

10 minutes " deep dive: one autonomous run end-to-end, decisions, limits, roadmap

"[Open with the 5-minute pitch, then walk one run end-to-end:]

One run, end to end. The scheduler fires. The **Monitor** agent pulls a canonical series " say `session_conversion_rate` " through `semantic-mcp` (which composes the SQL) and `warehouse-mcp` (which dry-runs the cost, executes, and reconciles to within 0.5%), then calls `stats-mcp.detect_anomaly`. It flags that conversion fell week-over-week. The **Decompose** agent then calls `stats-mcp.decompose_change` across canonical dimensions like `channel_group` and `device_category`. Suppose the result is: per-segment rates are essentially flat, but there's a large *mix* effect " a surge of low-converting mobile, Paid Social traffic shifted the weights. That's the Simpson's-paradox moment: the naive read was 'checkout is broken'; the decomposition says behavior didn't change at all, the mix did.

The **Diagnose** agent (Opus) takes that and builds a best-first hypothesis tree, verifying each branch with governed SQL " confirming the mobile/Paid Social volume spike is real and localizing it. It dollarizes: affected sessions $\tilde{\Delta}$ — rate delta $\tilde{\Delta}$ — revenue-per-session, attributed by segment, yields revenue-at-risk. Then the **Critic** (Opus) attacks the finding: is this a mix confound? Is the sample big enough for significance? Could it be a known seasonal swing " post-holiday in this dataset? Is it a data-quality artifact like a NULL spike or a late shard? It issues a verdict " PASS, DOWNGRADE, or DROP. Only survivors proceed. The **Prescribe** agent (Sonnet) sizes the fix " here, an acquisition-mix intervention, not a checkout fix " via `experiment-mcp.power_analysis` and `runtime_estimate`. Finally the **Narrator** renders the Decision Brief through `report-mcp`, and the diagnosis is saved to memory so the next run can recall it and track whether the recommended action actually worked. Findings hand off between agents as a typed JSON 'Finding' envelope.

Key design decisions and the alternatives I rejected. First, a deterministic state machine over a free-roaming agent " I traded some autonomy for reproducibility, cost control, and the ability to put a hard Critic gate in the flow. Second, MCP capability isolation over prompt instructions " 'you may not call run_query' enforced by tool allow-lists beats 'please write good SQL,' so the Narrator structurally cannot touch the warehouse. Third, an Opus/Sonnet split " Opus where judgment matters (orchestrate, diagnose, critique), Sonnet for high-volume sub-tasks, to keep cost sane. Fourth, deterministic stats in real Python over LLM-computed math " the decomposition is unit-tested against hand-worked golden values, because a keystone transform that's silently wrong corrupts every downstream number. Fifth, the offline injection benchmark

over eyeballing live anomalies â€” it's the only way to get labeled ground truth and a defensible accuracy number.

Limitations, stated honestly. Observational, three-month, late-2020 data: experiments are designed and quasi-experimentally read back, not run live; LTV and long retention are proxied; identity is cookie-grain because `user_id` is null; channel attribution navigates the first-touch `traffic_source` gotcha. The decomposition is correlational, not causal â€” which is exactly why the Critic attaches explicit confound caveats rather than overclaiming.

Roadmap. Phase 3 is productionization: multi-tenant isolation, near-real-time streaming ingestion, and warehouse-agnostic adapters behind `warehouse-mcp` so the semantic layer stays the only SQL author across Snowflake, Databricks, or DuckDB. Phase 4 is the frontier: replace correlational decomposition with true causal inference â€” difference-in-differences, synthetic control, double-ML â€” where data supports it; join GA4 with CRM and cost data for true CAC and ROAS; and close the loop by pushing experiment cards to a real experimentation platform, reading results back, and auto-updating the backlog. The architecture was built to absorb each of those without touching the trust spine."

Section 3: Product Analytics Interview

This section covers the three workhorse frameworks of product analytics - funnels, retention, and cohorts - at the depth an interviewer expects from someone who has shipped funnel diagnosis to production. Throughout, I tie the concepts to Helios, where funnel logic lives in `fct_funnel` with monotonic `reached_*` flags, and retention/cohort math is computed deterministically by `stats-mcp.cohort_retention` under an explicit short-horizon caveat.

Funnels

Q1. What is the difference between step (step-to-step) conversion and overall conversion, and why does an analyst need both?

Ideal answer. Overall conversion is `purchasing_sessions / sessions` - the end-to-end yield of the whole funnel - while step conversion is the rate between two adjacent stages, e.g. `add_to_cart_sessions / view_item_sessions`. Overall conversion tells you *how much* business you're losing; step conversion tells you *where* you're losing it, because the overall rate is the product of all step rates (`overall = step1 x step2 x ... x stepN`). I report both because a flat overall number can hide a single catastrophic step (a checkout that 90%-abandons), and conversely a single bad step rate is meaningless without knowing how many sessions actually reach it. The discipline is: watch overall as the headline KPI, then attribute movement to the

specific step whose rate moved most. **Why Helios demonstrates it.** Helios materializes both `session_conversion_rate` (overall) and the four step rates (`view_to_cart_rate` , `cart_to_checkout_rate` , etc.) in the semantic layer, so the Decompose agent can decompose a movement in the headline rate down to the exact step that caused it before any hypothesis is formed. **Follow-ups.** If overall conversion is flat but two step rates moved in opposite directions, how do you surface that? / Why is multiplying step rates not exactly equal to overall conversion in real data? / How would you weight which step to investigate first?

Q2. Explain ordered vs unordered funnels. When is each appropriate?

Ideal answer. An ordered funnel requires users to complete steps in a specific sequence and counts a step only if the prior steps happened first (view -> cart -> checkout -> purchase); an unordered funnel counts whether each event occurred at all within the window, regardless of sequence. Ordered funnels model genuine sequential processes like e-commerce checkout where order is causally meaningful, and they're stricter, so they report lower, more honest conversion. Unordered funnels suit feature-adoption questions ("did the user do A, B, and C in any order this week?") where sequence is irrelevant. The trap is using an unordered count for an inherently ordered process: you'll over-count "conversions" because a user who purchased via a deep link without viewing an item still satisfies an unordered "viewed AND purchased" filter. For a purchase funnel I default to ordered. **Why Helios demonstrates it.** Helios's macro funnel (`session_start` → `view_item` → `add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase`) is strictly ordered and enforced by the `reached_*` max-downstream flags, so a session marked `reached_begin_checkout` provably passed through the earlier stages. **Follow-ups.** How do you handle a user who skips a step (e.g. one-click buy)? / What breaks if you compute an ordered funnel without a per-user timestamp ordering? / When would you deliberately relax ordering?

Q3. How do you choose the conversion window for a funnel, and what bias does the choice introduce?

Ideal answer. The window is the time allowed for a user to complete downstream steps after entering, and it should match the natural decision latency of the action: minutes-to-hours for an impulse e-commerce purchase, days-to-weeks for a high-consideration B2B signup. Too short a window under-counts conversions that legitimately complete later (truncation bias); too long inflates conversion by crediting unrelated later sessions and makes the funnel slow to react to recent changes. The right approach is to inspect the empirical distribution of time-to-convert and set the window at a high percentile (e.g. p90) of genuine conversions, then hold it fixed so trends are comparable. I also distinguish *session-scoped* funnels (window = one session) from *user-scoped* funnels (window spans sessions), since they answer different questions. **Why Helios demonstrates it.** Helios scopes its funnel to a single session - session key `(user_pseudo_id,`

`ga_session_id`) - which makes the conversion window unambiguous (intra-session) and sidesteps cross-session attribution noise, a deliberate simplification stated as a known limitation rather than hidden. **Follow-ups.** How would you measure time-to-convert to justify the window? / What changes if you move from session-scoped to 7-day user-scoped? / How does window choice interact with seasonality?

Q4. A funnel step's drop-off jumped last week. Walk me through how you attribute the cause.

Ideal answer. First I confirm the drop is real and not noise - is the change statistically significant given the sample reaching that step, or is it a small-N artifact? Then I decompose the rate change: did the *segments' behavior* worsen (rate-change), or did the *traffic mix* shift toward inherently lower-converting segments (mix-shift)? These have completely different fixes - rate-change points at a product/UX/pricing defect, mix-shift points at acquisition. I then segment the step by device, channel, geo, and landing page to localize the regression, and cross-check against deploy logs, tracking changes, and known launches before concluding. Finally I quantify the impact in dollars (sessions lost x downstream value) so the finding is prioritized by money, not by percentage-point drama. **Why Helios demonstrates it.** This is Helios's core loop: `stats-mcp.significance_test` gates the anomaly, `stats-mcp.decompose_change` splits the movement into `mix_effect` / `rate_effect` / `interaction`, and the Diagnose agent drills the rate effect first because that's where real behavior change lives. **Follow-ups.** What's your significance test for a proportion change? / How do you rule out a tracking/data-quality regression? / How do you convert a step drop into revenue-at-risk?

Q5. What is mix-shift vs rate-change in a funnel, and why does conflating them cause wrong decisions?

Ideal answer. For an aggregate rate $R = \sum_i (w_i * r_i)$ where w_i is a segment's volume share and r_i its conversion rate, a change decomposes as `mix = sum(dw_i * r_i(t0))`, `rate = sum(w_i(t0) * dr_i)`, and `interaction = sum(dw_i * dr_i)`. Mix-shift means the aggregate moved purely because traffic composition changed (e.g. a paid-social campaign flooded the top of funnel with low-intent mobile users) even though *every segment's* conversion held steady; rate-change means in-segment behavior genuinely changed. Conflating them is the classic error: you "fix" a checkout you think is broken when actually the checkout is fine and acquisition got cheaper-and-worse. Decomposing first tells you whether to call product or marketing. **Why Helios demonstrates it.** This decomposition identity is Helios's technical centerpiece - `stats-mcp.decompose_change` computes it deterministically in Python (never in token-space), which is exactly how Helios dissolves Simpson's paradox before forming any hypothesis. **Follow-ups.** Give an example where overall conversion rises but every segment's rate falls. / Why compute this in real code rather than letting an LLM estimate it? / How big does interaction have to be before you report it separately?

Q6. Why can an aggregate funnel mislead you (Simpson's paradox), and how do you guard against it?

Ideal answer. Simpson's paradox is when a trend present in every subgroup reverses or vanishes in the pooled data because the subgroup *weights* changed. In funnels this means overall conversion can rise while mobile, desktop, and tablet conversion each fall - simply because desktop (the higher-converting segment) gained share. The aggregate is a weighted average, and weighted averages move for two reasons: the weights or the rates. The guard is to never trust a pooled rate movement on its own: always decompose into mix vs rate, and always compute rates as `SUM(numerator)/SUM(denominator)` after grouping rather than averaging per-segment ratios, which silently re-weights every segment to equal. **Why Helios demonstrates it.** Helios bakes this in two places: the dbt convention that rates are `SUM(num)/SUM(den)` post-group (never a mean of ratios), and `decompose_change` as a mandatory step so no finding ships on an un-decomposed aggregate. **Follow-ups.** Why does averaging per-segment ratios re-weight the data? / Which dimension would you decompose by first and why? / How do you present a Simpson reversal to a non-technical exec?

Q7. How do you analyze a funnel by segment without drowning in dimension combinations?

Ideal answer. I treat segmentation as a guided search, not a full cross-join. Start with the single dimension whose decomposition contributes the largest *rate effect* to the movement, drill into its worst-performing level, then conditionally expand to a second dimension only within that branch - a best-first traversal rather than enumerating device x channel x geo x landing-page exhaustively. I cap segment cardinality and enforce a minimum sample per cell so I'm not chasing 12-session segments with noisy rates. The output is a short, ranked path ("Paid Social -> mobile -> /sale landing page") rather than a 400-row pivot, because a prioritized narrative is what drives action. **Why Helios demonstrates it.** The Diagnose agent runs a best-first hypothesis-tree RCA over Helios's conforming dimensions (`device_category`, `channel_group`, `country`, etc.), expanding only the most promising branch and SQL-verifying each node, which keeps it inside the 5 GiB byte budget instead of blowing it on a full cube. **Follow-ups.** What minimum sample do you require per segment cell? / How do you stop the tree from over-fitting to one noisy segment? / How do you handle high-cardinality dimensions like landing_page?

Q8. How would you design and QA a funnel data model so step rates are never greater than 1 and never silently wrong?

Ideal answer. The invariant is monotonicity: `sessions ≥ reached_step1 ≥ reached_step2 ≥ ... ≥ reached_purchase`. I enforce it by defining each step flag as "reached this step *or any later step*" (a max-downstream flag) computed from the maximum stage achieved per session, rather than independent per-event booleans that can produce a later step with no earlier one. Then every step rate is guaranteed `≤ 1` by construction. I QA it with tests asserting the

monotonic chain and with a reconciliation that the funnel's terminal count matches the orders fact to the cent. These transforms fail *silently* - a subtly wrong sessionization poisons every downstream number - so they get golden-value tests, not just spot checks. **Why Helios demonstrates it.** Helios's `int_ga4__funnel_steps` uses `reached_*` monotonic flags (the `did_*` naming is retired precisely because it allowed non-monotonic states), and revenue at the terminal step reconciles against `fct_orders` via the `revenue_reconciles` test - the keystones flagged as silent-failure risks. **Follow-ups.** Show the SQL shape of a max-downstream flag. / What test catches a sessionization bug before it reaches the funnel? / Why is reconciliation to $\leq 0.5\%$ drift part of the funnel's definition of done?

Retention

Q9. Define classic (N-day), rolling, and bracket (range) retention. When do you use each?

Ideal answer. Classic N-day retention measures the share of a cohort active *exactly* on day N after acquisition - precise but harsh and spiky for low-frequency products. Rolling (a.k.a. "N-day or later" / unbounded) retention counts users active on day N *or any day after*, which is smoother and better for products used irregularly. Bracket/range retention buckets activity into windows (day 0, days 1-7, days 8-30) and counts a user retained if active anywhere in the bracket - the practical default for weekly or monthly products because it tolerates natural usage cadence. The choice must match usage frequency: applying classic day-7 retention to a product people use monthly will report near-zero retention that's an artifact of the metric, not the product. **Why Helios demonstrates it.** Helios exposes retention through `stats-mcp.cohort_retention` as a deterministic, parameterized computation, so the retention *definition* (classic vs rolling vs bracket) is an explicit, auditable argument rather than an ad-hoc SQL decision buried in a query. **Follow-ups.** Which would you pick for a monthly-use product and why? / How does rolling retention relate to a survival/Kaplan-Meier curve? / What's the failure mode of classic retention for low-frequency apps?

Q10. What does a retention curve look like, and what do its shape, slope, and floor tell you?

Ideal answer. A retention curve plots the fraction of a cohort still active against days/weeks since acquisition. Healthy products show a steep initial drop (the "shedding" of low-intent signups) that then *flattens into a non-zero asymptote* - the "smile" or plateau - which is the retained core that drives long-term value. A curve that decays to zero with no floor means no durable habit formed and the product is leaky no matter how much you spend on acquisition. The two things I read are the *floor* (does it asymptote above zero?) and the *slope after the elbow* (a slowly declining plateau still erodes the base). A rising or "smiling" curve, where later periods recover, signals genuine product-market fit. **Why Helios demonstrates it.** Helios computes the per-period retained fractions via `cohort_retention` so the curve's elbow and floor are quantified outputs, but it

explicitly caps interpretation at the ~3-month data window - it will not extrapolate a plateau it cannot observe. **Follow-ups.** How do you distinguish a true plateau from not-enough-data? / What slope on the plateau still worries you? / How would you compare two cohorts' curves statistically?

Q11. N-day vs unbounded retention: what's the trade-off and which is more honest?

Ideal answer. N-day (exact-day) retention is unbiased for the precise question "active on day N" but is high-variance and punishes any product not used daily, often understating real engagement. Unbounded ("day N or later") retention is smoother and arguably more honest about whether a user is *still a user*, but it's monotonically non-increasing only in N and can mask declining frequency because a user who logs in once on day 60 counts as retained through day 30. Neither is universally "right" - the honest move is to state the definition explicitly and pick the one whose question matches the decision. For churn/health monitoring I lean unbounded; for measuring a specific day-1 onboarding intervention I use exact N-day so the signal isn't diluted by later activity. **Why Helios demonstrates it.** Because retention is a tool argument in `stats-mcp.cohort_retention`, Helios forces the definition to be named and stored with the diagnosis in memory, so a later run reads back retention computed the same way - no silent redefinition between runs. **Follow-ups.** Why is unbounded retention monotonic in N? / When does unbounded retention hide a real decline? / Which do you put on the exec dashboard?

Q12. Why is retention often called the truest signal of product health - truer than acquisition or even revenue?

Ideal answer. Acquisition and top-line revenue can be bought - a spend spike inflates both without any improvement in the product - whereas retention measures whether users came *back* unprompted, which is the closest observable proxy for delivered value and the engine of compounding growth. Retention sets the ceiling on LTV (LTV is essentially the integral under the retention curve times per-period value), so a leaky bucket caps every other metric no matter how good the funnel is. It's also lagging and hard to game, which is exactly what makes it trustworthy. The mature framing: funnels tell you if you can *get* users, retention tells you if you *deserve* them.

Why Helios demonstrates it. Helios prices movements in dollars of revenue-at-risk and prescribes experiments, but it's explicit that on a ~3-month observational window it reports *short-horizon retention proxies with stated assumptions* rather than claiming true LTV - treating retention as the health signal while being honest about how far it can see. **Follow-ups.** How is LTV related to the area under the retention curve? / Why is retention harder to game than revenue? / What short-horizon proxy would you trust on 3 months of data?

Q13. What is survivorship bias in retention analysis and how does it distort conclusions?

Ideal answer. Survivorship bias is drawing conclusions from only the users who "survived" to the point of measurement while ignoring those who churned earlier - so the surviving population looks healthier and more engaged than the original cohort ever was. It shows up when you compute "average engagement of active users" over time: the average rises not because anyone improved but because the disengaged dropped out, leaving a denominator of self-selected loyalists. Another form is analyzing only cohorts old enough to have "matured," which excludes recently-acquired (and possibly worse-retaining) users. The fix is to always anchor metrics to the *original cohort denominator at acquisition*, and to be explicit when a metric conditions on still being active. **Why Helios demonstrates it.** `cohort_retention` always divides by the cohort's *acquisition-time* size, not the surviving count, so retained fractions can't drift upward purely from attrition - and the Critic agent flags "insufficient sample" / survivorship-style confounds as a refutation category before a finding ships. **Follow-ups.** Where does "average sessions per active user" hide survivorship? / How do you correct an engagement metric for it? / How does the Critic catch this class of error?

Q14. How would you statistically compare retention between two cohorts (e.g. before vs after a change)?

Ideal answer. I compare retention at matched periods-since-acquisition (day-7 vs day-7), not calendar dates, and test the difference in retained proportions with a two-proportion z-test or chi-square, reporting a confidence interval on the gap, not just a point delta. For the whole curve I'd use a log-rank test on the survival curves so I'm comparing shapes rather than one cherry-picked day. Crucially I check that the cohorts are comparable on mix (channel, device, geo) first - a "retention improvement" that's really a mix-shift toward better-retaining channels is the same Simpson trap as in funnels. Because this is observational data, I frame it as a quasi-experimental pre/post or difference-in-differences readback, not a causal A/B claim. **Why Helios demonstrates it.** Helios routes all such tests through `stats-mcp.significance_test` (deterministic, seeded) and is explicit that on its historical dataset it runs quasi-experimental pre/post and diff-in-diff readbacks rather than live A/Bs - sizing experiments it cannot actually run live. **Follow-ups.** Why match on period-since-acquisition rather than calendar week? / When is a log-rank test better than a per-day z-test? / How does diff-in-differences strengthen a pre/post retention claim?

Q15. A retention curve flattened into a plateau after a release. Is that good, and how do you verify it's real?

Ideal answer. A plateau above zero is the goal - it signals a retained core - but after a release I'm suspicious of three alternatives before I celebrate: (1) not enough elapsed time, so the "plateau" is just the youngest cohorts not having had time to churn yet; (2) survivorship, where the denominator quietly shifted; and (3) seasonality, where a holiday lull or spike coincided with the release. I verify by comparing same-age cohorts across the release boundary, checking significance on the plateau-period retention, and decomposing whether the improvement is in-

segment (real) or a mix-shift toward stickier channels. Only after those refutations survive do I attribute the plateau to the release. **Why Helios demonstrates it.** This maps directly onto the Critic's adversarial refutation categories - seasonality decoy, insufficient sample, mix-confound, data-quality - which must all return PASS before Helios reports the plateau as a genuine win rather than a DOWNGRADE/DROP. **Follow-ups.** How do you tell a real plateau from cohort immaturity? / What seasonality control would you use on GA4 holiday data? / Why decompose a retention gain by segment?

Q16. How do you handle low-frequency products where day-N retention looks terrible by construction?

Ideal answer. For a product genuinely used weekly or monthly, daily retention will look near-zero even with perfect engagement, so I switch the *unit of time* to match cadence - weekly or monthly retention buckets (bracket retention) - and define "active" by a meaningful action, not just a session open. I also look at frequency/recency directly (how often does the core return, and how recently) rather than forcing a daily lens. The mistake is keeping the daily metric and concluding the product is failing; the metric is failing the product. Picking the retention granularity is itself an analytical decision that should be justified from the observed usage-interval distribution. **Why Helios demonstrates it.** Because retention granularity is a parameter to `stats-mcp.cohort_retention`, Helios can switch between daily/weekly/monthly bucketing without rewriting SQL - and it pairs retention with RFM-style recency/frequency via `stats-mcp.rfm_segment` for low-frequency populations. **Follow-ups.** How do you pick weekly vs monthly from the data? / What defines "active" for a low-frequency product? / How does RFM complement a retention curve here?

Q17. What's the relationship between retention, churn, and engagement, and how do you avoid double-counting them as separate wins?

Ideal answer. Retention and churn are complements at a fixed horizon (period churn = 1 - period retention), so reporting both a "retention up" and a "churn down" headline from the same data is double-counting the same movement. Engagement (depth/frequency of use among the active) is a *different* axis: you can have stable retention with rising engagement, or shrinking-but-deepening usage. The clean framing separates the *width* of the base (retention/churn) from the *intensity* of use (engagement), and recognizes that engagement often *predicts* future retention - so I use engagement as a leading indicator and retention as the confirming lagging metric, never as two independent KPIs to celebrate twice. **Why Helios demonstrates it.** Helios keeps these distinct in its semantic layer - `engagement_rate` / `engaged_sessions` measure intensity while `cohort_retention` measures the durable base - so a Decision Brief never books a single underlying movement as two separate findings. **Follow-ups.** Give a case where retention is flat

but engagement rises - what do you do? / Why is engagement a leading indicator of retention? / How would you test whether engagement predicts next-period retention?

Cohorts

Q18. What is a cohort, and what's the difference between an acquisition cohort and a behavioral cohort?

Ideal answer. A cohort is a group of users sharing a defining event in a common time window, analyzed together over their lifetime so you compare like-aged users rather than mixing tenures. An *acquisition* cohort is grouped by *when* they arrived (the January-signups, the week-of-launch users) and answers "is each successive vintage of users better or worse?" A *behavioral* cohort is grouped by *what they did* (users who completed onboarding, users who added a payment method) and answers "does doing X change downstream retention/revenue?" Acquisition cohorts isolate temporal/quality trends in the funnel top; behavioral cohorts isolate the value of specific actions and are how you find activation "aha" moments. The two answer different questions and shouldn't be conflated. **Why Helios demonstrates it.** Helios's data model carries `is_new_user` and `session_number_bucket` dimensions plus an acquisition-date grain, so `cohort_retention` can be sliced as an acquisition cohort (by arrival week) or a behavioral one (e.g. sessions that `reached_add_to_cart`) without leaving governed SQL. **Follow-ups.** Which cohort type finds an activation "aha" moment? / How would you define a behavioral cohort in this GA4 funnel? / What confound threatens a behavioral-cohort conclusion?

Q19. How do you read a cohort retention triangle/grid, and what patterns matter?

Ideal answer. A cohort grid has one row per acquisition cohort and one column per period-since-acquisition, with the retained fraction in each cell - so you read *down a column* to compare the same age across cohorts (is the day-7 number improving for newer vintages?) and *across a row* to trace one cohort's decay curve. Verticals isolate whether product/onboarding changes lifted later cohorts (cohort-over-cohort improvement at fixed age); diagonals reveal calendar-time shocks (a holiday or outage hitting all live cohorts at once). The single most important read is the column trend at a stable horizon, because that's the apples-to-apples signal that the product is getting stickier independent of how much time has elapsed. **Why Helios demonstrates it.** Helios's `cohort_retention` output is exactly this period-since-acquisition grid, and the Decompose/Diagnose agents read columns (same-age cohort-over-cohort) so an improvement isn't faked by simply having older, more-decayed cohorts in the mix. **Follow-ups.** What does a diagonal pattern tell you vs a vertical one? / Why compare at fixed age rather than fixed calendar date? / How do you spot a tracking change in the grid?

Q20. What does "reading cohort-over-cohort" mean and what's the most common mistake?

Ideal answer. Reading cohort-over-cohort means comparing successive acquisition cohorts at the *same maturity* - January's day-30 retention vs February's day-30 retention - to judge whether the product is improving for each new vintage. The most common mistake is comparing cohorts at the same *calendar date* instead of the same age: a newer cohort has had less time to churn, so it will always look artificially better, an "immaturity" illusion that evaporates as they age. The second mistake is comparing cohorts of wildly different *composition* (a paid-campaign cohort vs an organic cohort) and attributing a retention gap to the product when it's really an acquisition-mix difference - the same mix-vs-rate problem that haunts funnels. **Why Helios demonstrates it.** Helios anchors cohort comparisons to period-since-acquisition (not calendar) and runs `decompose_change` to separate whether a cohort-over-cohort shift is mix (cohorts sourced differently) or rate (genuinely better retention) before crediting a release. **Follow-ups.** How does cohort immaturity inflate a young cohort? / How do you control for acquisition-mix differences between cohorts? / What's the minimum elapsed time before a cohort comparison is fair?

Q21. Why is the ~3-month data window a serious caveat for cohort and retention conclusions, and how do you handle it honestly?

Ideal answer. With only ~3 months of data, the earliest cohort has at most ~90 days of observable history and the latest has almost none, so any long-horizon retention or LTV claim is extrapolation beyond what the data can support - the curve's true asymptote and tail are simply unobserved. The honest handling is to (1) report only short-horizon proxies (e.g. day-7, day-30 retention) and label them as such, (2) state the assumptions any projection rests on rather than presenting a smooth LTV number, and (3) restrict cohort-over-cohort comparisons to ages every compared cohort has actually reached. Pretending to a 12-month LTV from 3 months of data is exactly the over-claiming that erodes trust with a senior stakeholder. **Why Helios demonstrates it.** This is one of Helios's explicit maturity points: the dataset is observational and ~3 months (Nov 2020-Jan 2021), so Helios *defers true LTV*, reports short-horizon retention proxies with stated assumptions, and won't extrapolate a tail it can't see. **Follow-ups.** What horizon would you trust on this dataset and why? / How would you caveat an LTV projection to an exec? / How does limited history interact with cohort immaturity?

Q22. How would you build an acquisition-cohort retention analysis on GA4 data, given user_id is mostly null?

Ideal answer. I'd grain the cohort on the first-seen date per identity, but I have to be explicit that GA4's `user_id` is almost always null, so identity is cookie-grain (`user_pseudo_id`) - meaning these are *cookie-based* cohorts that over-count users who clear cookies or switch devices and can't stitch cross-device journeys. I assign each `user_pseudo_id` to its acquisition week, then compute retained fractions at each period-since-acquisition, comparing cohorts down the column. I'd state the cookie-grain assumption up front because it caps the precision of any user-level retention claim, and I'd avoid implying these are true deduplicated humans. Naming the

limitation is what makes the analysis credible. **Why Helios demonstrates it.** Helios is explicit that `user_id` is almost always null so identity is cookie-grain (`user_pseudo_id`) and user-level metrics are cookie-based approximations - the cohort math in `cohort_retention` inherits and surfaces that caveat rather than hiding it. **Follow-ups.** How does cookie-grain identity bias retention up or down? / What would change with stitched cross-device identity? / How do you communicate "cookie-based, not user-based" to a PM?

Q23. How do you decide cohort granularity (daily vs weekly vs monthly) and cohort size?

Ideal answer. Granularity trades resolution against statistical stability: daily cohorts give fine temporal detail but small, noisy cells; monthly cohorts are stable but blur fast-moving changes. I pick the grain from two things - the product's natural usage cadence and the per-cohort sample size needed for the retained-fraction confidence intervals to be tight enough to act on. On a short window with moderate traffic, weekly cohorts are usually the sweet spot: enough users per cohort to be significant, enough cohorts to see a trend. I also align cohort width to the retention bucket width so I'm not comparing daily cohorts on monthly retention or vice versa, which mismatches the two time axes. **Why Helios demonstrates it.** Helios carries both `day` and `week` dimensions, and because cohort grain and retention bucketing are parameters to `cohort_retention`, it sizes cohorts to keep per-cell samples above the threshold the `significance_test` needs - all under the fixed 5 GiB byte budget. **Follow-ups.** How does cohort size affect the confidence interval on retention? / Why align cohort width with retention-bucket width? / What sample floor do you set per cohort cell?

Q24. How would you find an "aha moment" / activation metric using behavioral cohorts, and what's the causal caveat?

Ideal answer. The standard play is to build behavioral cohorts on candidate early actions (added an item, completed checkout setup, returned within 24h) and compare downstream retention between users who did vs didn't do each action within their first session/week - the action with the largest, most stable retention lift is the activation candidate. The non-negotiable caveat is that this is *correlational*: motivated users both perform the action and retain, so the action may be a marker of intent rather than a cause. To move toward causation I'd design an experiment that nudges the action for a random subset, or at minimum run a quasi-experimental matched comparison and stress-test for the obvious confounds before recommending the team optimize for it. **Why Helios demonstrates it.** Helios's Prescribe agent turns such a candidate into a *powered* experiment card (via `experiment-mcp.power_analysis / design_experiment`) precisely because Helios refuses to present a behavioral-cohort correlation as a causal lever without sizing the experiment to confirm it. **Follow-ups.** Why is a behavioral-cohort retention lift not proof of causation? / How would you design an experiment to confirm the aha moment? / What confounds would the Critic raise here?

Q25. Tie it together: how do funnels, retention, and cohorts combine into a single growth diagnosis?

Ideal answer. They're three lenses on one system and most powerful when chained. The funnel localizes *where* in the session conversion breaks and prices the leak in dollars; the mix-vs-rate decomposition says whether to fix product or acquisition; cohorts then ask whether the leak is a one-time shock (a diagonal calendar event) or a structural decline in successive vintages (a worsening column); and retention answers whether the users you *do* convert actually stick - because fixing a funnel that feeds a leaky bucket just spends faster. The senior synthesis is a single prioritized narrative: the largest dollar-impact, statistically-significant, mix-vs-rate-disambiguated finding, with the cohort/retention context that says whether it's acute or chronic, plus a prescribed, powered experiment - all bounded by an honest statement of what the 3-month window can and can't support. **Why Helios demonstrates it.** This is precisely Helios's end-to-end pipeline: Monitor -> Decompose (mix/rate) -> Diagnose (funnel/segment RCA) -> Critic (refute) -> Prescribe (powered card) -> Narrator (Decision Brief), with funnel logic in `fct_funnel / reached_*`, cohort/retention math in `stats-mcp`, and the whole chain graded against the offline 85%-vs-45% benchmark. **Follow-ups.** Which lens do you reach for first on a revenue drop and why? / How do you keep three analyses from producing three disjoint findings? / How does the dollar-impact ranking arbitrate between a funnel fix and a retention fix?

Experimentation

Q26. How do you turn a vague product hunch into a testable experiment hypothesis?

Ideal answer. A good hypothesis is falsifiable and pre-commits to a direction, a mechanism, and a magnitude: "Because [behavioral insight], if we [change], then [primary metric] will move by [at least the MDE], measured over [window], without harming [guardrail]." I force the mechanism clause because it prevents "we changed the button color and conversion went up" stories that don't generalize - the mechanism is what you actually learn. I also pre-register the primary metric, the analysis unit, the segment cuts, and the decision rule before I see data, so I can't retrofit a winner. The single most common failure I avoid is the unfalsifiable hypothesis ("this will improve engagement") that no result could ever contradict. Concretely, a Helios-style hypothesis reads: "Because Paid Search add_to_cart^begin_checkout rate dropped 6pp driven by a mobile rate-change (not mix), if we add an express-checkout entry point, begin_checkout rate recovers 3pp over 14 days, with bounce rate as a guardrail." **Why Helios demonstrates it.** The Prescribe agent emits experiment cards whose hypothesis is *derived* from the decompose_change rate-vs-mix verdict, so the mechanism clause is grounded in a quantified, SQL-verified diagnosis rather than a guess. **Follow-ups.** How do you decide the experiment's primary metric vs

secondary metrics? / What stops a stakeholder from changing the success metric after seeing results? / When is a directional hypothesis ("X goes up") insufficient and you need a magnitude?

Q27. Walk me through sizing an A/B test: MDE, power, significance, and sample size.

Ideal answer. Sample size is driven by four levers: baseline rate p , the minimum detectable effect (MDE), significance $\hat{\alpha}$ (false-positive tolerance), and power $1 - \hat{\beta}^2$ (false-negative tolerance, usually 80%). For a two-proportion test the per-arm $n \approx \frac{(z_{\{1-\hat{\alpha}/2\}} + z_{\{1-\hat{\beta}^2\}})^2 \hat{p}(1-\hat{p})}{(\hat{p}(1-\hat{p}) - \Delta^2)}$. The non-negotiable discipline is to set the MDE from the *smallest business-meaningful lift*, not from "what's detectable in two weeks" — sizing to a flattering MDE is how teams ship underpowered tests that can't distinguish a real null from noise. I always convert n into a *runtime* using actual daily eligible traffic and then sanity-check against business calendar (holidays, full weekly cycles). If the runtime is implausibly long, that's signal to widen the MDE, pick a more sensitive proxy metric, or not run the test at all. **Why Helios demonstrates it.** experiment-mcp exposes `power_analysis` and `runtime_estimate` exactly so every Prescribe card carries an MDE, required n per arm, and a feasibility-checked runtime in days against the funnel's measured traffic — no card ships "go run an A/B test" without the math attached. **Follow-ups.** How does variance reduction (CUPED) change your sample-size math? / Why is MDE the lever you should set first, not last? / How do you size a test on a continuous metric like revenue-per-session vs a proportion?

Q28. What are guardrail metrics and why do they matter more than the primary metric sometimes?

Ideal answer. Guardrail metrics are the metrics you're *not* trying to move but refuse to break — latency, error rate, refund rate, unsubscribe rate, overall revenue. They exist because almost any local optimization can shift harm elsewhere: a checkout redesign that lifts `begin_checkout` but tanks payment success is a net loss disguised as a win. I treat guardrails asymmetrically — the primary metric needs to clear a positive MDE, but a guardrail only needs to *not regress* beyond a tolerance, so I run a non-inferiority test on it rather than demanding significance. The maturity point is recognizing that a "winning" experiment with a silently degraded guardrail is the most dangerous outcome, because it ships and the damage compounds. I pre-register guardrails alongside the primary so no one quietly drops the inconvenient one after the fact. **Why Helios demonstrates it.** Every Prescribe experiment card names an explicit guardrail (e.g., bounce rate or downstream purchase rate), and the Critic agent will DOWNGRADE a prescription whose proposed change plausibly trades a near-term step-rate gain for a downstream-revenue loss. **Follow-ups.** How do you set the tolerance band on a guardrail? / What's the difference between a guardrail and a secondary metric? / How do you handle a test that wins on primary but trips a guardrail by a hair?

Q29. Explain the peeking problem and how sequential testing addresses it.

Ideal answer. Peeking is checking a fixed-horizon test repeatedly and stopping the moment $p < 0.05$ — this inflates the false-positive rate far above the nominal $\hat{\alpha}$ because you get many independent chances to cross the threshold (with daily peeks over two weeks, true type-I error can exceed 30%). The naive fix is "don't look," which is operationally unrealistic. The correct fixes are *sequential* methods that are valid under continuous monitoring: group-sequential designs with alpha-spending functions (O'Brien-Fleming, Pocock), or always-valid inference via mixture sequential probability ratio tests / always-valid confidence sequences. These spend your error budget across looks so the *anytime* false-positive rate stays bounded. The tradeoff is a modest sample-size premium versus a fixed-horizon test in exchange for the freedom to stop early and to look whenever you like. **Why Helios demonstrates it.** Helios runs on observational historical data so it doesn't *run* live sequential tests, but experiment-mcp's design templates pre-commit to a fixed horizon and decision rule precisely to model the discipline that prevents peeking — and the offline benchmark's significance gating uses pre-registered thresholds rather than stopping when a number looks good. **Follow-ups.** Why does daily peeking inflate type-I error so much? / O'Brien-Fleming vs always-valid confidence sequences — when would you pick each? / How does peeking interact with the multiple-comparisons problem across many metrics?

Q30. What are novelty and primacy effects, and how do you keep them from fooling you?

Ideal answer. Novelty effect is when users over-engage with anything new simply because it's new, inflating early treatment numbers that decay; primacy (change-aversion) is the mirror image — existing users resist a change and depress early numbers that recover as they adapt. Both make the *first few days* unrepresentative of steady state. I defend against them by running tests for at least one to two full behavioral cycles (commonly 2+ weeks), plotting the treatment effect *over time* rather than trusting a single pooled estimate, and segmenting new vs returning users since novelty/primacy mostly bite the returning cohort. If the effect curve hasn't stabilized, the test isn't done regardless of significance. The honest read is that a large day-1 lift that decays toward zero is usually novelty, not a durable win. **Why Helios demonstrates it.** This is the same logic as Helios's Critic seasonality/transient check: a finding driven by a short-lived spike gets challenged the same way a novelty-driven experiment result would, and the eval suite includes seasonality-decoy scenarios that punish mistaking a transient bump for a durable shift. **Follow-ups.** How long is "long enough" and how do you decide? / How would you formally test whether the treatment effect is stable over time? / Why segment new vs returning users when diagnosing novelty?

Q31. When should you NOT run an A/B test?

Ideal answer. Don't experiment when you can't get adequate power in a reasonable window (low traffic or a rare event makes the required runtime months long); when the change is a one-way door or ethically/legally mandated (you ship it regardless of result); when network/spillover

effects violate the stable-unit assumption (marketplace, social, pricing changes leak across arms); when the cost of the experiment exceeds the value of the information; or when the decision is reversible and cheap enough that just shipping and monitoring is faster learning. In those cases I reach for alternatives: quasi-experiments, interrupted time series, holdout geographies, or bandits for low-stakes optimization. The senior move is treating "should we test at all?" as the first question, not "how do we test?" â€” running an underpowered test you'll over-interpret is worse than running none. **Why Helios demonstrates it.** Helios is built on *observational* historical GA4 data where live A/B testing is impossible, so it designs and *sizes* experiments and then performs quasi-experimental readbacks (pre/post, difference-in-differences) â€” it explicitly does not claim to run live A/Bs, which is itself a "when not to experiment" judgment baked into the architecture. **Follow-ups.** How do network effects break the A/B assumption, and what do you do instead? / How do you value the information from an experiment to decide if it's worth running? / What's your fallback when traffic is too low to power any test?

Q32. How do you measure impact when you can't randomize â€” pre/post and difference-in-differences?

Ideal answer. With observational data I lean on quasi-experimental designs that approximate a counterfactual. Naive pre/post is the weakest because it confounds the intervention with everything else that changed over time (seasonality, macro trends), so I only trust it with a stable baseline and explicit seasonality controls. Difference-in-differences is stronger: I compare the treated unit's beforeâ€”after change against an *untreated control* unit's same-period change, which differences out any shock common to both â€” its load-bearing assumption is *parallel trends* (the two would have moved together absent treatment), which I validate on pre-period data. For a single treated series I'll use interrupted time series or a synthetic control (a weighted blend of donor units that reconstructs the pre-period). The honest caveat: these identify effects only under assumptions randomization would have guaranteed for free, so I state those assumptions out loud.

```
-- Difference-in-differences on funnel conversion
SELECT
  AVG(IF(treated AND post, rate, NULL)) - AVG(IF(treated AND NOT post, rate, NULL))
  - (AVG(IF(NOT treated AND post, rate, NULL)) - AVG(IF(NOT treated AND NOT post, rate, NULL)))
  AS did_effect
FROM daily_segment_rates;
```

Why Helios demonstrates it. Because the GA4 dataset is observational (Nov 2020â€”Jan 2021), Helios's experiment-mcp prescribes the *design* and then reads back impact via pre/post and diff-in-diff rather than asserting causal A/B results â€” and the Critic enforces the parallel-trends / seasonality caveats that keep a pre/post number from being oversold as causal. **Follow-ups.** How do you validate the parallel-trends assumption before trusting DiD? / When would you choose

synthetic control over DiD? / How do you communicate causal uncertainty from a quasi-experiment to an executive?

North Star Metrics

Q33. What makes a good North Star Metric, and how would you choose one for an e-commerce store?

Ideal answer. A North Star Metric (NSM) should sit at the intersection of customer value and durable business value, be a leading-ish indicator of revenue (not revenue itself), be measurable today, and be something teams can actually influence. For an e-commerce store I'd resist "revenue" as the NSM because it's a lagging outcome that every team can claim and none can directly move; I'd prefer something like *weekly purchasing customers* or *number of completed purchases per active user* — value is realized at purchase, and it captures both acquisition and retention. The test I apply: if this number goes up sustainably for a year, is the business healthier? If yes, and teams can map their work to it, it's a candidate. I pair it with 2–4 input metrics so a single number never becomes the only thing the org looks at. **Why Helios demonstrates it.** Helios treats *purchase* as the terminal funnel event and quantifies revenue-at-risk in dollars, so its diagnoses are always anchored to the value-realizing step — the NSM-style outcome — while decomposing the funnel steps that *lead* to it. **Follow-ups.** Why not just use revenue as the North Star? / How many input metrics should sit under a North Star? / How would you NSM a pre-monetization product where purchases don't exist yet?

Q34. Distinguish leading from lagging indicators with concrete examples.

Ideal answer. Lagging indicators measure outcomes after the fact (revenue, churn, LTV, retention) — accurate but you can't steer by them in time. Leading indicators move *first* and predict the lagging outcome (add_to_cart rate, week-1 activation, checkout-start rate, repeat-visit frequency), so they're what you actually manage day to day. The art is establishing that the "leading" metric is genuinely predictive of the lagging one rather than just correlated noise — I'd validate the lead-lag relationship statistically (e.g., does this week's activation predict next month's retention?) before trusting it. A practical rule: dashboards full of lagging metrics tell you the patient's temperature is high but not why; leading metrics tell you which intervention to make now. I report both, but I *act* on leading and *judge* on lagging. **Why Helios demonstrates it.** Helios's funnel is an explicit leading+lagging chain (session_start → view_item → add_to_cart → purchase); when it diagnoses a purchase shortfall it walks *upstream* to the leading step rate that moved, which is precisely "act on the leading indicator that predicts the lagging outcome." **Follow-ups.** How do you statistically confirm a metric is leading, not coincident? / Give an example where a leading indicator decoupled from its lagging outcome. / How does the lead time of a leading metric affect how you use it?

Q35. What is an input metrics tree and how does it connect daily work to the North Star?

Ideal answer. An input metrics tree (or metric tree) decomposes the North Star into the multiplicative or additive drivers a team can directly move, level by level, until you reach metrics that map to specific initiatives. For purchases you might decompose: Purchases = Sessions $\times$ (Purchase rate), and Purchase rate = product of step conversion rates across the funnel; each step then fans out into the levers behind it (traffic mix, page performance, friction). The discipline is that the tree must *reconcile* "children should roll up to the parent arithmetically" so you can attribute a North Star movement to a specific branch. This is what lets a PM owning "checkout friction" see exactly how their step rate flows up to the company NSM. Without the tree, every team optimizes locally and no one can explain why the top number moved. **Why Helios demonstrates it.** Helios's semantic layer plus `decompose_change` is an operational metric tree: an aggregate rate $R = \sum \text{wáμφράμφ}$ is decomposed into mix, rate, and interaction contributions per segment, so a North-Star-level movement is mechanically attributed down the tree to the exact segment and step that caused it. **Follow-ups.** How do you keep a metric tree reconciling when metrics are ratios? / What happens when two branches of the tree move in opposite directions? / How granular should the leaves of the tree be?

Q36. Explain Goodhart's Law and metric gaming "how do you design metrics that resist it?

Ideal answer. Goodhart's Law: "when a measure becomes a target, it ceases to be a good measure" "optimizing a proxy hard enough decouples it from the underlying goal it was standing in for. Classic gaming: a "sessions" target met by splitting one visit into many, or an "add_to_cart" target hit while purchases fall. I design against it with three tactics: pair every target metric with a guardrail or counter-metric that catches the cheap exploit (engagement paired with retention; clicks paired with downstream conversion); prefer metrics closer to true value (purchases over add-to-carts) so the proxy gap is small; and define metrics governed and centrally so a team can't quietly re-spec the numerator to flatter itself. The senior framing is that the more pressure a metric carries, the closer it must sit to real value and the more its definition must be locked down. **Why Helios demonstrates it.** Helios's governed semantic layer (semantic-mcp as the *only* SQL path) means no agent or analyst can redefine "purchase rate" to game a result, and the Critic's adversarial refutation explicitly hunts for the "looks good but is an artifact" failure mode that Goodhart-style gaming produces. **Follow-ups.** How do you choose the counter-metric that catches gaming of a target? / Why does governing a metric's definition reduce gaming? / Give an example where moving a metric closer to true value introduced a new gaming surface.

Q37. How do you avoid a single-metric tunnel vision while still having one North Star?

Ideal answer. The North Star is a *focusing* device, not the only number "the failure mode is treating it as the sole KPI and optimizing it to the detriment of everything around it. I surround it

with a small balanced set: the NSM, its input tree, and a ring of guardrails covering the dimensions the NSM ignores (quality, cost, fairness across segments, long-term retention). I also re-examine the NSM periodically, because the right North Star at one growth stage (activation early, retention/monetization later) is wrong at another. The judgment is holding the tension: enough focus that the org rows together, enough breadth that you notice when "winning" on the NSM is quietly losing the business. A North Star with no guardrails is just a Goodhart trap waiting to spring. **Why Helios demonstrates it.** Helios never reports a single number in isolation — its Decision Brief pairs the headline movement with the mix/rate decomposition, revenue-at-risk, and the Critic's caveats, so the executive sees the focal metric inside its guardrail context, not alone. **Follow-ups.** When should an NSM change as the company matures? / How big should the guardrail ring be before it's just a dashboard again? / How do you detect that you're optimizing the NSM at the expense of the business?

Q38. How would you validate that your North Star actually predicts long-term business value?

Ideal answer. I'd test the predictive link rather than assume it: build a cohort dataset, measure each cohort's early NSM trajectory, and check whether it explains later lagging outcomes (revenue, retention, LTV) — correlation over time, lead-lag regression, or out-of-sample prediction. If the NSM doesn't predict the outcome it's supposed to lead, it's the wrong star. I'd also stress it across segments to make sure the relationship isn't a Simpson's-paradox artifact of mix. The honest constraint: predicting *long-term* value needs a long observation window, and with a short dataset I can only validate short-horizon proxies and must state the assumption that the relationship persists. That caveat is part of the answer, not a weakness in it. **Why Helios demonstrates it.** Helios is candid that its ~3-month GA4 window defers true LTV / long-horizon retention, so it reports short-horizon proxies *with stated assumptions* — exactly the disciplined honesty required when you can't yet prove the long-term predictive link of a North Star. **Follow-ups.** What statistical test confirms the NSM predicts the lagging outcome? / How does a short data window limit which North Star you can validate? / How would Simpson's paradox fool you into trusting a bad North Star?

Segmentation

Q39. Explain RFM segmentation and when you'd reach for it.

Ideal answer. RFM scores customers on three behavioral axes: Recency (how recently they purchased), Frequency (how often), and Monetary (how much) — typically by quintile-ranking each axis and combining the scores into segments like "champions," "at-risk," and "hibernating." It's powerful because it's purely behavioral, computable from transaction logs alone, and maps directly to action (champions get loyalty perks, at-risk get win-back campaigns). I reach for it for retention and CRM/lifecycle targeting where the question is "who do I treat differently and how?"

rather than prediction. Its limits: it's descriptive not predictive, it ignores *why* behavior changed, and the quintile cutoffs need to be revisited as the base shifts. I'd use it as a fast, actionable first pass before reaching for a model.

```
SELECT user_pseudo_id,  
       NTILE(5) OVER (ORDER BY MAX(event_date)) AS r,  
       NTILE(5) OVER (ORDER BY COUNT(*))         AS f,  
       NTILE(5) OVER (ORDER BY SUM(revenue))      AS m  
FROM purchases GROUP BY user_pseudo_id;
```

Why Helios demonstrates it. stats-mcp exposes `rfm_segment` as a deterministic, governed primitive, so RFM scoring runs through the same audited stats path as decomposition and significance testing rather than being hand-rolled SQL per analysis. **Follow-ups.** Why quintiles and not k-means clustering for RFM? / How do you keep RFM cutoffs stable as the customer base grows? / How would you make RFM predictive rather than just descriptive?

Q40. Behavioral vs demographic segmentation – which do you trust and why?

Ideal answer. Demographic segmentation (age, geo, device) is easy to get and easy to explain but often weakly predictive of behavior – two 25-year-olds in the same city can have opposite purchase intent. Behavioral segmentation (what users actually *did*: pages viewed, cart actions, purchase cadence, channel) is usually far more predictive and more actionable because it's closer to the moment of value. I default to behavioral for product and lifecycle decisions, and use demographic mainly for acquisition targeting where behavior isn't observable yet. The nuance: demographics are a useful *prior* before behavior accumulates (cold start), and the strongest segmentations combine both. The trap to avoid is treating an easily-measured demographic cut as causal when it's just a proxy for an underlying behavior. **Why Helios demonstrates it.** Helios's 16 governed dimensions blend behavioral cuts (funnel step reached, channel group, session source/medium) with contextual ones (device, geo), and its diagnoses prioritize *behavioral* rate-change segments because that's where real, actionable signal lives – composition/demographic shifts are flagged as mix effects, not behavior. **Follow-ups.** When is a demographic segment actually a proxy for a behavior? / How do you segment a brand-new user before any behavior exists? / How would you test which segmentation scheme is more predictive?

Q41. What makes a segment actionable, and how do you avoid useless segmentation?

Ideal answer. A segment is actionable only if it's (1) measurable from data you have, (2) substantial enough to matter, (3) differentiable – members respond differently to a treatment than non-members, and (4) reachable – you can actually do something different for them. The classic waste is a statistically tidy segment you can neither target nor treat differently (interesting cluster, no lever), or a segment so small the effort exceeds the payoff. My test before shipping a

segmentation: name the *specific different action* each segment gets. If "at-risk users" and "champions" would receive the same treatment, the split earned nothing. I'd rather have three crude, actionable segments than twelve elegant, inert ones. **Why Helios demonstrates it.** Helios always ties a segment-level finding to a *prescribed action* — when decompose_change attributes a drop to, say, mobile Paid Search, the Prescribe agent emits an experiment card targeting that exact reachable segment, so segmentation never terminates in an un-actionable cluster. **Follow-ups.** How small is too small for a segment to be worth treating? / How do you test that a segment is differentiable (responds differently)? / What do you do with an interesting-but-unreachable segment?

Q42. What is the "segment of one" / personalization, and what are its tradeoffs?

Ideal answer. "Segment of one" is the limit of segmentation — personalizing to the individual via models that predict each user's next-best-action, propensity, or recommendation, rather than bucketing into coarse groups. The upside is relevance and lift; the downsides are real: sparse per-user data (most users have thin histories), the cold-start problem, infrastructure and latency cost, harder explainability and governance, and filter-bubble/fairness risks. I treat it as a spectrum, not a binary — most value is captured by good behavioral *cohorts*, and true 1:1 is worth it only where data density and stakes justify the complexity. The mature stance is starting with actionable segments and graduating to personalization only where it measurably beats the cohort approach. **Why Helios demonstrates it.** Helios deliberately operates at the cookie-grain segment level (user_pseudo_id, since user_id is almost always null), which is honest about *why* fine-grained per-person personalization isn't credible on this dataset — identity is a cookie-based approximation, so segment-level diagnosis is the defensible altitude. **Follow-ups.** When does a segment-of-one beat well-chosen cohorts enough to justify the cost? / How does sparse per-user data limit personalization? / How would cookie-grain identity (no stable user_id) constrain a personalization system?

Q43. How do you decide segmentation granularity without slicing into noise?

Ideal answer. Granularity is a bias-variance tradeoff: coarse segments hide real heterogeneity (Simpson's paradox lurks), fine segments fragment into samples too small for stable estimates. I let *sample size and effect stability* set the floor — a segment needs enough volume that its rate has a tight enough confidence interval to act on — and I let *actionability* set the useful ceiling (don't split finer than you can treat). Practically I'll start coarse, decompose, and only drill into a sub-segment when its contribution is both large and statistically distinguishable from noise. The discipline that prevents over-slicing is requiring significance before trusting a small-segment delta; otherwise you're chasing sampling variance and calling it insight. **Why Helios demonstrates it.** Helios's Diagnose agent does best-first hypothesis-tree RCA — it drills *rate effects before mix effects* and only deeper into segments that materially move — while the Critic

explicitly refutes findings built on *insufficient sample*, which is the over-granular-segmentation failure mode. **Follow-ups.** What's your rule for the minimum sample a segment needs before you trust its rate? / How does Simpson's paradox push you toward finer segmentation? / How do you stop a drill-down from becoming a multiple-comparisons fishing trip?

Q44. A segment looks like it's driving a metric change – how do you confirm it's real and not a mix artifact?

Ideal answer. First I separate two questions: did this segment's *behavior* change (rate), or did its *share of traffic* change (mix)? A naive "largest absolute segment delta" reading conflates them and routinely fingers the wrong cause. I run the mix/rate/interaction decomposition: $\Delta R = \Delta(\text{dw} \cdot \text{r}) [\text{mix}] + \Delta(\text{w} \cdot \text{dr}) [\text{rate}] + \Delta(\text{dw} \cdot \text{dr}) [\text{interaction}]$; if the segment's contribution is mostly mix, the "driver" is just composition shifting, not behavior. Then I test the rate change for significance so I'm not chasing a small-sample wobble, and I check for confounds (seasonality, a launch, a tracking change). Only a contribution that is rate-driven, significant, and confound-free earns a causal-sounding story. This is exactly how you dissolve Simpson's paradox in attribution. **Why Helios demonstrates it.** This is Helios's technical centerpiece:

`decompose_change` deterministically splits a movement into mix/rate/interaction, the Diagnose agent prioritizes the rate-driven (real-behavior) segments, and the Critic adversarially challenges any finding as a possible mix-confound – beating a naive baseline 85% vs ~45% on the labeled benchmark. **Follow-ups.** Walk me through the mix vs rate decomposition arithmetic. / How does a pure mix shift create a Simpson's-paradox reversal? / Why is "largest absolute segment delta" such a dangerous default heuristic?

Metric Design

Q45. Walk me through numerator/denominator discipline in defining a rate metric.

Ideal answer. Every rate is a numerator over a denominator, and most metric bugs are denominator bugs – the numerator is usually obvious, the denominator is where definitions silently diverge. For "checkout conversion rate," is the denominator sessions that reached checkout, all sessions, or users? Each gives a different number and answers a different question. My discipline: state the unit of analysis explicitly (session vs user vs event), ensure the numerator is a strict subset of the denominator population (you can't have purchases from sessions that aren't in the denominator), align the time windows of both, and define inclusion/exclusion rules (bots, internal traffic, refunds) once. A rate where numerator and denominator are computed over mismatched populations or windows is the single most common way two teams "measure the same metric" and get different answers. **Why Helios demonstrates it.** Helios's `reached_*` monotonic max-downstream flags in `int_ga4_funnel_steps` guarantee numerator/denominator coherence – a session that reached purchase is provably counted in every upstream step's

denominator “ so step rates are arithmetically consistent and reconcile to ±0.5%. **Follow-ups.** Give a case where changing only the denominator flips the conclusion. / How do you align numerator and denominator time windows for a lagging conversion? / Why is the unit of analysis (session vs user) a definitional choice, not a detail?

Q46. Why are ratio metrics dangerous, and how does Simpson's paradox show up in them?

Ideal answer. Ratio metrics (rates, averages-of-ratios, per-user values) are dangerous because they don't aggregate the way people assume “ you cannot average two groups' rates to get the pooled rate unless you weight by denominator, and a weighted aggregate can move opposite to every one of its parts. That's Simpson's paradox: each segment's conversion rate can *rise* while the overall rate *falls*, purely because traffic mix shifted toward a lower-converting segment. The defense is to always decompose an aggregate-rate change into mix vs rate effects before interpreting it, and to be explicit about the weighting. Operationally I also avoid "average of per-user ratios" when I mean "ratio of totals" “ they're different metrics and the per-user-ratio version is heavily distorted by low-denominator users. **Why Helios demonstrates it.** Simpson's paradox is the exact pathology Helios was built to dissolve: `decompose_change` mechanically separates the mix component (Î£ dwáµ¢Â-ráµ¢) from the rate component (Î£ wáµ¢Â-dráµ¢), so an aggregate that fell while segments rose is correctly attributed to composition rather than mistaken for behavioral decline. **Follow-ups.** Show me a numeric example where pooled rate moves opposite to every segment. / When is "ratio of sums" right and "average of ratios" wrong? / How does denominator weighting change which aggregate you should report?

Q47. What does it mean for a metric to be "governed," and why does it matter?

Ideal answer. A governed metric has a single, version-controlled, centrally-owned definition “ its SQL logic, unit, filters, and dimensions live in one registry, and every consumer (dashboard, model, agent, report) derives from that definition rather than re-implementing it. It matters because the alternative is *metric drift*: ten teams write ten slightly different "active user" queries, three numbers reach the exec, and the meeting becomes a debate about whose SQL is right instead of what to do. Governance buys consistency, auditability (you can trace any number to its definition), and trust. The tradeoff is upfront cost and some rigidity “ you can't redefine a metric on a whim “ but for any metric that drives decisions, that rigidity is the point. The hard part of analytics isn't computing a number; it's everyone computing the *same* number. **Why Helios demonstrates it.** Helios's semantic layer (models/semantic/semantic_layer.yaml: 47 metrics + 19 dimensions, referential-integrity compiled, mapping 1:1 to dbt MetricFlow) is the single source of truth, and semantic-mcp's `build_query` is the *only* path to SQL “ so there are zero hallucinated columns and no agent can author an off-definition metric. **Follow-ups.** How do you stop teams from re-implementing a metric outside the registry? / What's the cost of governance

and when is it not worth it? / How do you migrate a legacy ad-hoc metric into a governed definition?

Q48. How do you version and evolve a metric definition without breaking trust in historical numbers?

Ideal answer. Metric definitions *must* change (new business logic, fixed bugs, new exclusions), and the danger is silently changing a definition so that a chart's history shifts underneath everyone – last quarter's number no longer matches last quarter's deck. I handle it like code: definitions are version-controlled with semantic versioning, changes go through review with a documented rationale, and a breaking redefinition gets a *new* metric version (purchase_rate_v2) rather than mutating v1 in place. I keep a changelog mapping when each version was active, and where possible I backfill both versions during a transition so dashboards can show the discontinuity explicitly rather than hide it. The principle: a metric's history should be reproducible, and any redefinition should be announced and auditable, never silent. **Why Helios demonstrates it.** Because Helios's metrics live in a git-versioned semantic_layer.yaml under CI, any definition change is a reviewable diff gated by the offline benchmark – the same PR that changes a metric must keep decomposition MAPE $\approx 10\%$ and 0 hallucinated columns, so a redefinition can't silently regress historical correctness. **Follow-ups.** When do you version-bump a metric vs edit it in place? / How do you communicate a metric redefinition to dashboard consumers? / How would you reproduce a number computed under an old metric version?

Q49. Design a metric for "checkout health" from scratch – talk me through your reasoning.

Ideal answer. I'd start from the decision the metric serves: is checkout *working*, and where does it leak? A single headline rate (begin_checkout → purchase) is the lagging summary, but health needs the step-level breakdown – add_shipping_info, add_payment_info, purchase – because "checkout is broken" is meaningless without the leaking step. So I'd define checkout health as the conversion rate of each downstream step with the *same* denominator discipline (sessions that reached the prior step), surfaced alongside a guardrail (payment-error rate) and segmented by device and channel since checkout friction is wildly device-dependent. I'd explicitly choose session-grain over user-grain given the data, state the time window, and exclude refunds from revenue. The output isn't one number – it's a small governed metric set that localizes the leak and reconciles to the funnel. **Why Helios demonstrates it.** This mirrors Helios's funnel marts exactly: begin_checkout → add_shipping_info → add_payment_info → purchase as monotonic step flags in int_ga4_funnel_steps, each step a governed metric in the semantic layer with consistent denominators, so "checkout health" is decomposable to the exact leaking step and segment. **Follow-ups.** Why a step-level set rather than one composite checkout score? / How do you pick the denominator for each checkout step? / Which guardrail would you pair with checkout conversion and why?

Q50. How do you handle metric definitions that must reconcile across many consumers and stay correct over time?

Ideal answer. Reconciliation is an engineering problem, not a willpower problem: I make one definition the source of truth, generate all downstream SQL from it, and add automated checks that the same metric computed two ways agrees within a tolerance. Concretely that means a semantic registry, dry-run cost gates so a query can't silently scan the wrong (more expensive, differently-scoped) tables, and reconciliation tests in CI that fail the build if a metric drifts beyond, say, 0.5%. I also pin the *byte/scope* of each query so "the same metric" can't accidentally read a different partition. The mature recognition is that correctness isn't a one-time verification – it's a continuously-enforced invariant, because data, definitions, and consumers all change, and the only durable defense is automation that catches drift on every change. **Why Helios demonstrates it.** Helios enforces exactly this: warehouse-mcp's mandatory `dry_run` + `run_query` byte-budget gate with `reconcile-to=±0.5%`, the governed semantic layer as the single SQL source, and CI gates that block any PR regressing the offline benchmark – so metric correctness is a continuously-enforced invariant, not a hope. **Follow-ups.** What tolerance do you set for reconciliation and why? / How does a cost/byte gate also protect metric correctness? / What CI checks would you add to catch metric drift before it ships?

Section 4: Growth Analytics Interview

These answers are deliberately framework-based: every response leans on the canonical growth toolkit - AARRR (Acquisition, Activation, Retention, Referral, Revenue), growth loops, the North Star metric, and prioritization scoring (ICE/RICE) - rather than ad-hoc opinion. The thread tying them together is that growth analytics is about *finding the lever that moves a system*, and Helios is the concrete proof: it diagnoses *why* an e-commerce funnel moved by decomposing the AARRR stages it can observe (acquisition mix and activation funnel) into mix-shift vs rate-change.

Acquisition

Q1. Walk me through how you'd evaluate the channel mix of an e-commerce business. What does a healthy mix look like?

Ideal answer. Channel mix is the share of sessions (and, more importantly, revenue) coming from each acquisition channel, and I evaluate it on three axes at once: volume share, conversion quality, and cost. A healthy mix is *diversified and not fragile* - no single paid channel should carry the majority of revenue, because that creates a single point of failure when CPMs spike or an algorithm changes - and it skews toward channels with a self-reinforcing growth loop (organic search, direct, referral) rather than purely paid pumps. Concretely I'd group every session into the

10 GA4 default channel groups (Direct, Organic Search, Paid Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other) and compute `sessions`, `session_conversion_rate`, `revenue`, and `revenue_per_session` per group, then look at the *gap* between volume share and revenue share - a channel that is 30% of sessions but 10% of revenue is a quality problem, not a volume win. The senior move is to never average per-channel conversion rates; you compute mix-level rates as $\text{SUM}(\text{purchasing_sessions}) / \text{SUM}(\text{sessions})$ after grouping, otherwise Simpson's paradox makes a healthy-looking blended number hide a deteriorating channel. **Why Helios demonstrates it.** Helios pivots every funnel metric across `channel_group` (one of its 16 governed dimensions) sourced from the single `channel_group_case()` macro, so channel-mix analysis is a first-class query, not a one-off; the Decompose agent then splits any channel-level movement into mix vs rate effects. **Follow-ups.** How would you weight volume vs quality when one channel is high-volume/low-quality? / What blended-mix metric would you put on an executive dashboard? / How does channel mix change the interpretation of a flat overall conversion rate?

Q2. Define CAC. What's the difference between blended CAC and paid CAC, and when does each mislead?

Ideal answer. CAC (customer acquisition cost) is total acquisition spend over a period divided by the number of *new* customers acquired in that period. Blended CAC divides total spend by *all* new customers including the organically acquired ones, while paid CAC divides paid spend only by customers attributable to paid channels - blended always looks better because organic "free" customers dilute the denominator. Blended CAC misleads when you're trying to decide whether to *scale a paid channel*, because the marginal customer from scaling spend costs the paid CAC (often the marginal CAC, which is higher still), not the flattering blend. Paid CAC misleads in the opposite direction - it ignores that paid spend often *seeds* organic and referral loops (brand search, word-of-mouth), so judging a channel purely on its isolated paid CAC undercounts its halo. The mature framing is to report both, watch the CAC:LTV ratio, and treat blended CAC as the board-level efficiency number and channel-level (marginal) CAC as the budget-allocation number. **Why Helios demonstrates it.** Helios runs on an observational GA4 dataset with no spend data, so it is honest that it cannot compute true CAC; instead it quantifies *revenue-at-risk in dollars* per channel and sizes experiments, which is the half of the unit-economics equation the data actually supports - exactly the kind of scope honesty that reads as maturity. **Follow-ups.** Why is marginal CAC usually higher than average CAC? / How would payback period change your read on a high-CAC channel? / How do you attribute organic customers that paid spend actually seeded?

Q3. A channel doubled its session volume but overall revenue barely moved. How do you diagnose this?

Ideal answer. This is the canonical channel-quality-vs-volume problem, and the first thing I'd check is whether the new volume converts at all - I'd decompose the change in `revenue` (or `revenue_per_session`) into a mix-shift component and a rate-change component. If a low-converting channel (say Display or a broad Paid Social campaign) doubled, the *mix* effect alone can move blended `session_conversion_rate` down even though every individual channel's conversion rate is unchanged - that's pure composition. The diagnostic SQL is to pull volume share and per-channel conversion for both periods:

```
SELECT channel_group,
       SUM(sessions) AS sessions,
       SUM(purchasing_sessions) / SUM(sessions) AS conv_rate,
       SUM(revenue) / SUM(sessions) AS rev_per_session
FROM helios.marts.fct_daily_funnel
WHERE day BETWEEN @t0_start AND @t1_end
GROUP BY channel_group, period
```

If the doubled channel's `rev_per_session` is far below the others, the volume is real but low-quality and you've simply bought traffic that doesn't buy. The trap to avoid is concluding "conversion is broken" (a rate story) when it's actually a mix story - those have opposite fixes.

Why Helios demonstrates it. This is the exact scenario Helios was built to disambiguate: `stats-mcp.decompose_change` computes $mix = \sum w_i \cdot r_i(t_0)$, $rate = \sum w_i(t_0) \cdot r_i$, and *interaction*, so a "volume doubled, revenue flat" case is resolved deterministically into "mix effect dominates, rates held" rather than a hand-waved guess. **Follow-ups.** What would the interaction term tell you here? / How would you size the revenue you'd recover by reallocating spend away from the low-quality channel? / When is buying low-converting volume actually rational (e.g., upper-funnel)?

Q4. Explain first-touch, last-touch, and last-non-direct attribution. Which would you default to and why?

Ideal answer. Attribution assigns credit for a conversion to the touchpoints in the journey: first-touch gives 100% credit to the channel that first acquired the user, last-touch to the final channel before conversion, and last-non-direct (GA4's default) is last-touch but skips direct visits, reassigning credit to the last *campaignable* channel on the theory that "direct" is usually a return visit, not a discovery. I'd default to last-non-direct for day-to-day reporting because it's GA4's native model, it's reproducible, and it avoids over-crediting Direct (which is a junk-drawer bucket - bookmarks, dark social, untagged links). But I'd be explicit that no single-touch model is "correct"; first-touch over-credits discovery channels and starves closers, last-touch does the reverse, and both are wrong for multi-session journeys. For budget decisions I'd want data-driven or position-based multi-touch attribution, and ideally incrementality testing (geo holdouts), because attribution answers "who to credit" while only experiments answer "what was caused." **Why**

Helios demonstrates it. Helios reports against GA4's channel-group conventions and is explicit about Direct being a residual bucket; because the dataset is observational, Helios *designs and sizes* quasi-experimental readbacks (pre/post, difference-in-differences) rather than claiming a single attribution model is causal truth. **Follow-ups.** Why does Direct inflate under last-touch, and what does that hide? / When does first-touch actually beat last-non-direct? / How would you validate an attribution model against an incrementality test?

Q5. There's a well-known GA4 traffic_source gotcha that trips up acquisition analysis. What is it and how do you handle it?

Ideal answer. The gotcha is that the event-level `traffic_source` record in the GA4 BigQuery export is the user's *first-touch* source/medium/campaign - it's stamped once at user acquisition and repeats on every event for that user forever - so if you naively `GROUP BY traffic_source.medium`, you are not analyzing *session* acquisition, you're analyzing *user* acquisition, and a user first acquired via Paid Search will have every later organic or direct session mislabeled as Paid Search. This systematically over-credits whatever channel first acquired your users and makes returning-visit channels (Direct, Email) look smaller than they are. The fix is to derive session-scoped source/medium from the `event_params` (the `source`, `medium`, `campaign`, and `gclid` params attached to the session's `session_start` / `page_view` events) and only fall back to the event-level `traffic_source` when the session-scoped params are null. That's the difference between "where did this user originally come from" and "what brought them *this* time," and channel-mix and conversion-by-channel analysis need the latter. **Why Helios demonstrates it.** This is codified as a hard rule in Helios: it derives session-scoped `source / medium` from `event_params` and falls back to first-touch `traffic_source` only when null, so its `channel_group` dimension reflects session acquisition, not lifetime first-touch - removing a silent, systematic bias from every acquisition diagnosis. **Follow-ups.** How would first-touch labeling bias a returning-user cohort analysis? / How do you sessionize source when params are missing mid-session? / What does this gotcha do to a last-non-direct attribution built on top of it?

Q6. How do you compare paid vs organic acquisition beyond just cost?

Ideal answer. Cost is only one axis; I compare paid vs organic on quality, durability, and scalability. Quality means conversion rate, AOV, and downstream activation/retention of the cohort each channel brings - paid traffic from a broad prospecting campaign often converts and retains worse than organic-search intent traffic, so a "cheaper" paid customer can be more expensive on an LTV basis. Durability is the growth-loop question: organic search, direct, and referral are *compounding* (content and brand keep paying off, a flywheel), while paid is a *pump* that stops the instant you stop spending - so a business overweight on paid has a structurally fragile growth engine. Scalability cuts the other way: paid scales fast and predictably (turn up

budget), organic scales slowly. So the framework is to map each channel onto a 2x2 of unit economics (CAC:LTV) vs durability (loop vs pump), favor compounding channels for the base, and use paid for controllable, incremental growth on top - always validated with incrementality, since organic and direct are partly *caused* by paid. **Why Helios demonstrates it.** Helios segments the funnel and revenue by `channel_group` and computes `revenue_per_session` per channel, giving the quality (not just volume) read for each; its honesty about lacking spend data keeps the paid/organic comparison to what the observational dataset can defend. **Follow-ups.** How do you isolate the incremental value of organic that paid actually drove? / Which channel would you cut first in a downturn and why? / How does the loop-vs-pump distinction change a forecast?

Q7. What is an acquisition cohort and what questions does it answer that a cross-sectional view can't?

Ideal answer. An acquisition cohort groups users by *when* (and often *how*) they were acquired - e.g., "users first seen in week of 2020-12-07 via Paid Search" - and then tracks that fixed group's behavior forward over time. The questions it answers that a cross-sectional snapshot cannot are about *trajectory and quality drift*: is the December cohort activating and converting better or worse than the November cohort, and is a channel's customer *quality* degrading even while its volume grows? A cross-sectional "conversion rate this month" blends new and old users and hides the fact that, say, a scaled Paid Social channel is bringing progressively worse cohorts. Cohorting also separates acquisition effects from retention effects: a falling overall active-user count can be a *new-cohort acquisition* problem or an *old-cohort retention* problem, and only a cohort table (acquisition period on the rows, age on the columns) tells you which. The frequent mistake is comparing cohorts of different ages as if they were comparable - you must align on cohort *age*, not calendar date. **Why Helios demonstrates it.** Helios's `stats-mcp.cohort_retention` builds these triangles deterministically, and because identity is cookie-grain (`user_pseudo_id`, since `user_id` is almost always null), it is explicit that cohorts are cookie-based approximations - a stated assumption rather than a hidden one. **Follow-ups.** How would you tell an acquisition problem from a retention problem in a declining MAU? / Why must you compare cohorts at equal age? / What does cookie-grain identity do to cohort sizes over time?

Q8. Define a growth loop and contrast it with a funnel. Give an acquisition-loop example.

Ideal answer. A funnel is a *linear, one-way* model - traffic enters at the top and a shrinking fraction reaches the bottom - whereas a growth loop is *circular*: the output of the loop feeds back as input to the next turn, so growth compounds instead of requiring fresh top-of-funnel spend each time. The classic acquisition loop is content/SEO: a user lands on a page, that engagement and any user-generated content strengthen the page's ranking, which ranks for more queries, which acquires more users - the output (users + content) becomes the input (better ranking). A

referral loop is even cleaner: acquired users invite others who become acquired users. The strategic point is that loops have *defensible, compounding* economics while funnels are *leaky and need constant refilling*, so a great growth team instruments the loop's cycle time and amplification factor (how many new users each user generates), not just the funnel's step rates. Funnels and loops are complementary - you still optimize the funnel *inside* each turn of the loop. **Why Helios demonstrates it.** Helios models the GA4 macro funnel (session_start -> view_item -> add_to_cart -> begin_checkout -> add_shipping_info -> add_payment_info -> purchase) as its diagnostic spine while framing channel mix in loop terms - it can flag when a compounding channel (Organic Search) is losing share to a pump channel, which is a loop-health signal a pure funnel view misses. **Follow-ups.** What two metrics define a loop's health? / How would you instrument referral-loop amplification in GA4? / When is a funnel the right model and a loop the wrong one?

Q9. Acquisition volume is up but the overall funnel conversion rate dropped. Is the funnel broken? How do you prove it isn't (or is)?

Ideal answer. Not necessarily - this is the textbook mix-shift-masquerading-as-rate-change situation, and you prove which it is with a decomposition, not an eyeball. If acquisition grew by adding sessions from a low-converting channel or device, the *blended session_conversion_rate* falls purely because the weights *w_i* shifted toward low- *r_i* segments, even if every segment's own conversion rate is flat or improving (Simpson's paradox). To prove it, I decompose $\hat{I}(\text{conversion_rate})$ into $\text{mix} = \hat{I} \sum w_i \hat{A} \cdot r_i(t_0)$ (composition moved), $\text{rate} = \hat{I} \sum w_i(t_0) \hat{A} \cdot \hat{I} r_i$ (behavior moved), and *interaction*. If the mix term explains nearly all of the drop and per-segment rates are stable, the funnel is *not* broken - you just changed who's walking into it. If the rate term dominates, then a real behavioral/UX problem exists and I'd drill the worst-affected segment. The discipline is to *always drill rate effects before mix effects*, because rate effects are real behavior change you can fix, while mix effects are composition artifacts you manage at the acquisition layer. **Why Helios demonstrates it.** This is Helios's reason to exist: the Decompose agent calls `stats-mcp.decompose_change` to split the movement, and the Diagnose agent prioritizes rate effects over mix effects in its hypothesis tree - so "is the funnel broken?" gets a deterministic, dollar-quantified answer instead of a debate. **Follow-ups.** What does a large interaction term imply about the two periods? / If mix dominates, what's the recommended action vs if rate dominates? / How do you guard the decomposition against a tiny-sample segment dominating the math?

Q10. How do you detect and quantify a mix-shift in acquisition channels specifically?

Ideal answer. A mix-shift in acquisition is a change in the *volume share* *w_i* of channels between two periods, independent of how each channel converts. I detect it by computing each `channel_group`'s session share in *t0* and *t1*, flagging channels whose share moved beyond a noise threshold, and then quantifying the *business impact* of that shift by holding each channel's

own conversion/ `revenue_per_session` fixed and asking: how much did blended revenue move purely because the weights changed? That isolated number is the mix effect, $\hat{E} \hat{w}_i \cdot r_i(t_0)$. Practically:

```
SELECT channel_group, period,
       SUM(sessions) / SUM(SUM(sessions)) OVER (PARTITION BY period) AS volume_share,
       SUM(revenue) / SUM(sessions) AS rev_per_session
FROM helios.marts.fct_daily_funnel
GROUP BY channel_group, period
```

Then the mix effect is the share deltas weighted by baseline `rev_per_session`. The key sophistication is reporting the mix-shift *in dollars*, not just "Display share rose 6 points," because a share shift toward a high-RPS channel is good news and toward a low-RPS channel is a quantifiable revenue drag - same direction of share change, opposite business meaning. **Why Helios demonstrates it.** Helios's offline benchmark literally injects *volume perturbations* (change `w_i` only, holding rates fixed) as labeled mix-shift scenarios, and grades whether the pipeline rediscovers them; `stats-mcp.decompose_change` returns the mix effect in the metric's units so it can be priced as revenue-at-risk. **Follow-ups.** How big a share change is "signal" vs seasonal noise? / Why can a mix-shift toward a high-RPS channel still be a risk? / How would you separate a genuine acquisition mix-shift from a tracking/tagging change?

Q11. Where does acquisition sit in AARRR, and what's the single metric you'd watch for acquisition health?

Ideal answer. Acquisition is the first A in AARRR - the top of the pirate-metrics funnel - and it measures whether you can reliably bring qualified strangers to the product. The single metric depends on the business model, but for an e-commerce store I'd watch *qualified new sessions by channel* (or new-user sessions) as the volume pulse, paired with a quality guardrail like `session_conversion_rate` or `revenue_per_session` of the *newly acquired* traffic - because volume without quality is vanity. If forced to one composite, I'd lean toward CAC-payback or new-customer revenue contribution, since it bakes cost and quality together. The framework discipline is that acquisition should never be optimized in isolation: a North Star metric (e.g., weekly purchasing users, or revenue per active user) sits above AARRR and forces acquisition to be judged by the *quality* of users it feeds into Activation and Retention, not raw traffic. Optimizing acquisition volume against a North Star prevents the classic failure of celebrating a traffic spike that never converts. **Why Helios demonstrates it.** Helios treats the acquisition layer as the *source of weights* `w_i` feeding the funnel and always pairs a volume read with `revenue_per_session` per `channel_group`, so acquisition is judged by downstream quality - the AARRR-into-North-Star linkage made concrete. **Follow-ups.** Why is raw session volume a vanity metric? / How would you design a North Star that keeps acquisition honest? / What guardrail metric stops acquisition from gaming the North Star?

Q12. How would you prioritize which acquisition channel to invest in next? Use a scoring framework.

Ideal answer. I'd use a RICE or ICE scoring pass over candidate channel bets so prioritization is transparent and arguable on inputs, not vibes. ICE scores each option on Impact (expected revenue lift if it works), Confidence (how sure I am, grounded in cohort quality and prior data), and Ease (effort/cost to test), and I multiply or average them; RICE adds Reach (how many users the bet touches) and divides by Effort: $RICE = (Reach \times Impact \times Confidence) / Effort$. For acquisition specifically, Impact should be the *dollar* opportunity (e.g., the revenue-at-risk or revenue-upside a decomposition surfaced for that channel), Confidence should be pulled down for channels where the data is observational or the cohort is small, and Effort includes both creative/ops cost and the experiment runtime needed to read a result. The senior nuance is that the highest raw-ICE channel isn't always the pick - you weight toward *compounding* (loop) channels for long-run defensibility and reserve fast paid bets for filling near-term gaps. The output is a ranked, justifiable backlog, not a single hero bet. **Why Helios demonstrates it.** Helios's Prescribe agent emits a *prioritized, powered* experiment backlog - each card carries the dollar impact (from the decomposition), a confidence grounded in significance testing, and a runtime estimate from `experiment-mcp ( power_analysis / runtime_estimate )` - which is exactly the Impact/Confidence/Effort triad RICE formalizes. **Follow-ups.** When does the top-RICE item not get built? / How do you keep Confidence honest on observational data? / How does experiment runtime feed the Effort term?

Q13. How do seasonality and one-off campaigns confound acquisition trend analysis, and how do you control for them?

Ideal answer. Acquisition traffic is highly seasonal and event-driven - Black Friday, Cyber Monday, holiday gifting, a product launch, or a one-day email blast can swing channel volumes and mix dramatically - so a naive period-over-period comparison will mislabel a calendar effect as a real change. I control for this three ways: compare against the right baseline (year-over-year or the same weekday/week-of-season, not just last week), maintain a *known-events calendar* so a spike on a launch date isn't flagged as an anomaly, and use a forecast with prediction intervals (e.g., a seasonal model) so I'm comparing actuals against an expected band rather than a flat line. The decomposition then runs on the *residual* - the part of the movement the seasonal expectation didn't already explain. The failure mode to avoid is "seasonality theater," where every real regression gets dismissed as "probably just seasonal"; the discipline is to quantify the seasonal expectation explicitly and require the residual to clear a significance bar before raising an alarm. **Why Helios demonstrates it.** Helios's `stats-mcp.forecast` (Prophet/statsmodels) sets the expected band, its memory store holds a seasonality/launch calendar, and the Critic agent runs an explicit *seasonality* refutation that can DOWNGRADE or DROP a finding - and the offline benchmark includes deliberate "seasonality decoy" scenarios to ensure Helios doesn't fire on

calendar effects. **Follow-ups.** What baseline do you pick when there's no prior-year data? / How does a launch-calendar entry change anomaly scoring? / How do you avoid over-suppressing real regressions as "seasonal"?

Activation

Q14. Define activation. How is it different from acquisition and from conversion?

Ideal answer. Activation is the second A in AARRR: the moment a newly acquired user first experiences the product's core value - the point at which they "get it" and become meaningfully more likely to stick. It's distinct from acquisition (which only got them in the door - a `session_start`) and from a one-time conversion (which is a transaction, a Revenue-stage event); activation is specifically about *first realized value*, which for a SaaS product might be "completed setup and ran a first report" and for e-commerce is closer to "found a relevant product and engaged with it" (a `view_item` or `add_to_cart`), with purchase being the eventual conversion downstream. The reason the distinction matters operationally is that the *fix* differs: low acquisition is a marketing problem, low activation is an onboarding/first-experience problem, and low conversion can be a pricing/checkout problem. Conflating them sends you optimizing the wrong stage. A precise activation definition is the prerequisite for an activation *rate*, which is the lever between traffic and retention. **Why Helios demonstrates it.** Helios's funnel separates the acquisition entry (`session_start`), the activation/engagement steps (`view_item` , `add_to_cart`), and the conversion steps (`begin_checkout` -> `purchase`) as distinct, separately-decomposable stages - so a drop can be localized to activation rather than blamed on acquisition or checkout. **Follow-ups.** What's the e-commerce equivalent of a SaaS "aha moment"? / Why does conflating activation and conversion mislead the fix? / Where does `engagement_rate` fit relative to activation?

Q15. What is the "aha moment" and how would you find it empirically?

Ideal answer. The aha moment is the specific early action (or set of actions) that correlates most strongly with a user becoming a retained, valuable customer - Facebook's "7 friends in 10 days" and Slack's "2,000 messages" are the canonical examples. I find it empirically by taking an acquisition cohort, enumerating candidate early behaviors (e.g., viewed N items, added to cart, used search, returned within X days), and measuring which behavior best *separates* retained/converted users from churned ones - i.e., the action where the gap in downstream retention between users who did it and didn't is largest and the curve flattens (a threshold effect). I'm careful to frame this as *correlational* and then validate causality: the right test is whether *driving* more users to that action via onboarding actually lifts retention (a quasi-experiment or A/B), because otherwise I might just be describing what engaged users do anyway, not what *causes* engagement. The output is a crisp, measurable activation definition the whole team can

rally around. **Why Helios demonstrates it.** Helios is explicit that its dataset is observational, so it surfaces *candidate* leading indicators by decomposing the activation funnel and then has the Prescribe agent *design and size the experiment* to test whether driving that behavior is causal - it never claims correlation is causation, which is the exact discipline aha-moment analysis requires.

Follow-ups. How do you avoid mistaking a correlate of engagement for a cause? / What sample size do you need to trust the threshold? / How would you A/B test driving users to the aha action?

Q16. Define activation rate precisely and write how you'd compute it.

Ideal answer. Activation rate is the share of newly acquired users (or sessions) who reach the activation milestone within a defined window - precision requires nailing the numerator (the activation event), the denominator (which users, over what acquisition window), and the time bound (within N days of acquisition). For the GA4 store, a defensible session-grain definition is "sessions that reached the engagement/value step out of all sessions," which maps to a funnel-step rate:

```
SELECT
  channel_group,
  SUM(view_item_sessions)      AS activated_sessions,  -- reached value step
  SUM(sessions)                AS sessions,
  SUM(view_item_sessions) / SUM(sessions) AS activation_rate
FROM helios.marts.fct_daily_funnel
GROUP BY channel_group
```

Two non-negotiables: compute the rate as `SUM(numerator)/SUM(denominator)` after grouping (never the average of per-segment ratios - that reintroduces Simpson's paradox), and use the monotonic "reached" definition so the numerator counts a session that reached the value step or *any later step*, keeping the rate bounded by 1. The exact milestone is a product choice; the rigor is in fixing the cohort and window so the rate is comparable over time. **Why Helios demonstrates it.** Helios's `int_ga4__funnel_steps` uses `reached_*` max-downstream monotonic flags (guaranteeing `sessions ≥ reached_view_item ≥ ... ≥ reached_purchase`, so every step rate is `≤ 1`), and the semantic layer computes rates as `SUM(num)/SUM(den)` - so an activation-rate metric is governed and Simpson-safe by construction. **Follow-ups.** Why use a monotonic "reached" flag instead of a per-step "did" flag? / How does the activation window length change the rate? / Session-grain vs user-grain activation - which and why here?

Q17. Walk me through how you'd map and diagnose an onboarding (activation) funnel.

Ideal answer. I map the onboarding funnel as the ordered sequence of steps from entry to first value, measure the *step-to-step* conversion at each, and look for the biggest drop-off step relative to a benchmark - that's where the leak is. For the store the activation segment of the funnel is

`session_start` → `view_item` → `add_to_cart` , and I'd compute `view_to_cart_rate` and the step rates around it, then segment the worst step by device, channel, and landing page to localize the problem (e.g., the drop is concentrated on mobile, or on traffic from one channel landing on a slow page). Diagnosing *why* a step fell requires the mix-vs-rate split again: did the step rate genuinely drop (a real UX/onboarding problem) or did the *composition* of users hitting that step shift toward a segment that always converted worse? Only after isolating a real rate drop in a real segment do I form a hypothesis (e.g., a regression on the mobile product page) and prescribe a fix. The framework is: ordered steps -> biggest leak -> segment the leak -> decompose mix vs rate -> hypothesis -> sized experiment. **Why Helios demonstrates it.** Helios's Decompose agent runs on each funnel step (`view_to_cart_rate` , `cart_to_checkout_rate` , `checkout_to_purchase_rate`) and the Diagnose agent runs a best-first hypothesis-tree RCA that's SQL-verified at every node - so an onboarding-funnel leak is localized to a step *and* a segment *and* attributed to rate-not-mix, then handed to Prescribe. **Follow-ups.** How do you pick the benchmark for a "good" step rate? / What if the leak is split across two steps? / How do you tell a real step regression from a composition shift at that step?

Q18. What is time-to-value (TTV) and why does it matter for activation?

Ideal answer. Time-to-value is the elapsed time from a user's first contact (acquisition) to the moment they first experience core value (activation) - shorter TTV means users hit the aha moment before they lose interest, so it's one of the strongest levers on activation rate and, downstream, on retention. It matters because attention decays fast: every extra step, form field, or minute before first value bleeds users, and reducing TTV often lifts activation more cheaply than buying more traffic. I'd measure it as the median (not mean, because it's right-skewed) time or step-count from `session_start` to the activation event, segment it (mobile TTV is usually worse), and treat a long TTV at a specific step as a prioritized onboarding fix. For e-commerce, TTV maps to "how fast can a visitor find a relevant product" - search quality, navigation, and page speed - so it ties directly to the early funnel steps. The framework link is that TTV is the *speed* dimension of activation, while activation rate is the *completion* dimension; you want both high-completion and low-time. **Why Helios demonstrates it.** Helios's sessionized data carries event timestamps, so the time and step distance from `session_start` to the value step (`view_item` / `add_to_cart`) is derivable and decomposable by segment - letting a TTV regression be localized (e.g., mobile) the same way a rate regression is. **Follow-ups.** Why median over mean for TTV? / How would you trade off TTV against richness of the first experience? / How does TTV differ on mobile vs desktop and why?

Q19. What are leading indicators of retention, and how do they relate to activation?

Ideal answer. Leading indicators of retention are *early*, observable behaviors that predict whether a user will stick around long before the retention itself is measurable - they're the bridge between

activation and the Retention stage of AARRR, and activation milestones are usually the strongest ones. Examples: reaching the aha action within the first session/week, returning within N days of first visit, breadth of early engagement (viewed multiple categories, used search), and for e-commerce, adding to cart or completing a first purchase. They matter because retention is a *lagging* metric - you can't wait 90 days to learn a cohort is bad - so you instrument leading indicators to get a same-week read on cohort quality and to know which onboarding changes are working. The analytical method is the same as finding the aha moment: regress/segment early behaviors against eventual retention, pick the ones with the strongest, most stable separation, and validate causally. The mature caveat is that a leading indicator must be validated as *predictive out-of-sample*, not just fit in-sample, or you'll chase a spurious early signal. **Why Helios demonstrates it.** Helios is honest that its ~3-month window (Nov 2020-Jan 2021) defers true long-horizon retention/LTV, so it reports *short-horizon retention proxies with stated assumptions* and uses `stats-mcp.cohort_retention` to relate early activation behaviors to the near-term return signal - leading indicators done with explicit limitation-flagging. **Follow-ups.** How do you validate a leading indicator is predictive, not just correlated? / What proxy would you use when you only have 90 days of data? / How would activation rate itself serve as a leading indicator?

Q20. Activation rate dropped this week. Walk me through your diagnosis before you alarm anyone.

Ideal answer. First I'd check data quality - a tracking break, a tag misfire, or a schema change can fake an activation drop, and shipping a false alarm burns credibility - so I reconcile the event volumes against canonical totals before trusting the number. Then I'd ask whether the drop clears the noise/seasonal band: compare against the right baseline and a forecast interval, not last week's point. If it's real, I decompose $\hat{I}(\text{activation_rate})$ into mix vs rate: a shift in *who* arrived (e.g., a surge of low-intent Display traffic) can drop blended activation with every segment's own rate intact (mix), versus a genuine in-segment drop (rate) that signals an onboarding/UX regression. If rate dominates, I segment to localize it (device, channel, landing page), form a hypothesis, run a significance test to confirm the affected-segment drop isn't sampling noise, and only then quantify the dollar impact and raise it - with a recommended, sized experiment to fix it. The order is deliberate: data-quality -> seasonality -> mix-vs-rate -> segment -> significance -> dollars -> action. **Why Helios demonstrates it.** This is the Helios pipeline end to end: `warehouse-mcp.reconcile` ($\leq 0.5\%$ drift) guards data quality, `stats-mcp.forecast` sets the band, `decompose_change` splits mix/rate, `significance_test` confirms the segment, and the Critic adversarially attacks the finding (data-quality / seasonality / insufficient-sample / mix-confound) before the Narrator writes the Decision Brief. **Follow-ups.** Why is data-quality the first check, not the decomposition? / What significance bar would you require on the affected segment? / How do you price an activation drop in dollars when activation isn't a purchase?

Q21. How would you design an experiment to improve activation, and how do you size it?

Ideal answer. I'd start from a diagnosis (which step leaks, in which segment, mix vs rate) so the experiment targets a real, sized opportunity, then write a hypothesis with one primary metric (the activation rate at the leaking step) and a guardrail (e.g., downstream purchase rate, so I don't lift activation at the cost of conversion). Sizing is a power-analysis problem: given the baseline rate, the minimum detectable effect (MDE) worth shipping, and target power (0.8) and significance (0.05), I compute the required sample per arm and then translate it into runtime given the segment's daily traffic - a low-traffic segment may need weeks, which itself is a prioritization input. I'd guard against the classic mistakes: peeking (fixing the horizon up front or using sequential methods), under-powering (calling a flat result a "no effect" when the test couldn't detect the MDE), and ignoring the guardrail. If a live A/B isn't possible, I design a quasi-experiment - a pre/post or difference-in-differences readback against a comparable unaffected segment as the control. **Why Helios demonstrates it.** Helios's `experiment-mcp` does exactly this - `power_analysis` -> `runtime_estimate` -> `design_experiment` - and because the GA4 data is observational, Prescribe emits quasi-experimental (pre/post, diff-in-diff) readback designs, with each card carrying the dollar impact and runtime that feed the RICE-style backlog ranking.

Follow-ups. What guardrail metric stops you gaming activation? / How does low segment traffic change your design? / Why is peeking at the result dangerous and how do you prevent it?

Q22. How does the mix-shift-vs-rate-change distinction apply inside the activation funnel, not just acquisition?

Ideal answer. It applies at every rate in the funnel, including each activation step, because any aggregate step rate is $R = \sum w_i \cdot r_i$ over segments - so a change in `view_to_cart_rate` can be driven by a shift in *who* reached `view_item` (mix) or by a genuine change in how each segment behaves at that step (rate), exactly as at the acquisition layer. The practical consequence is that an "activation is dropping" alarm must be decomposed before it's actioned: if mobile traffic share rose and mobile always carries a lower view-to-cart rate, blended activation falls with no real UX regression (mix), and the lever is upstream acquisition targeting, not the product page. If instead the desktop view-to-cart rate itself fell, that's a real activation regression (rate) and the lever is the onboarding/page experience. Same number, opposite owner and opposite fix - which is precisely why you never act on a blended funnel rate without splitting it. This is also how you dissolve Simpson's paradox *inside* activation, where overall activation can drop while every segment improves. **Why Helios demonstrates it.** Helios's Decompose agent runs `decompose_change` on the funnel-step rates (`view_to_cart_rate`, `cart_to_checkout_rate`, etc.), not just on acquisition volume - so the mix/rate/interaction identity is applied throughout activation, and the offline benchmark's rate vs volume perturbations test both layers. **Follow-ups.** Give a Simpson's-paradox example inside activation. / How do you decide which segmentation dimension to decompose by? / If interaction is large at a step, what does that say?

Q23. What activation metric would you nominate as (or feed into) a North Star, and why?

Ideal answer. A North Star metric should capture *delivered customer value* and predict long-term growth, so for an e-commerce store I'd nominate something like *weekly purchasing users* (or revenue per active user), and the *activation* metric that feeds it is "share of new users who reach first value within their first session/week." The logic is the AARRR chain: activation is the leading lever that converts acquired traffic into the retained, revenue-generating users the North Star counts, so an activation milestone is a high-quality *input* metric to the North Star rather than the North Star itself. I'd avoid making raw activation events the North Star, because they can be gamed (lower the value bar and "activation" rises while real value doesn't) - which is why the North Star sits above AARRR and forces every stage, including activation, to be judged by downstream value. The test of a good North Star is that moving it reliably moves revenue and retention; activation rate earns its place because it demonstrably leads both. **Why Helios demonstrates it.** Helios's metric registry (28 metrics, including `revenue_per_user` /ARPU and `revenue_per_session`) lets activation milestones be evaluated against downstream value metrics in the same governed query - so the activation-to-North-Star linkage is computed on one source of truth, not stitched across tools. **Follow-ups.** Why not make the activation event itself the North Star? / How would you detect if activation is being gamed? / What guardrails sit beside a purchasing-users North Star?

Q24. How do you connect activation back to acquisition channel quality?

Ideal answer. Activation is the cleanest *quality* signal for an acquisition channel, because it measures whether the users a channel brings actually engage with core value, not just whether they showed up - so I cross-tabulate activation rate by `channel_group` (and campaign) on the *new-user* cohort and compare. A channel with high session volume but low activation rate is bringing low-intent traffic, and that's a budget-reallocation signal even before you get to purchase - it tells you the problem is upstream targeting, not the product. This closes the AARRR loop between the two stages: you judge acquisition not by volume but by the activation (and eventual retention) of the cohorts it delivers, which is exactly how you avoid scaling a channel that pumps in traffic that never activates. The analytical care is to compare channels at equal cohort age and to decompose so you don't confuse a channel's intrinsic quality with a temporary mix-shift in *which* of its sub-segments arrived. **Why Helios demonstrates it.** Because Helios pivots both the activation funnel steps and `revenue_per_session` by `channel_group` , it can directly attribute a low activation rate to a specific channel and price the reallocation opportunity in dollars - making "this channel's traffic doesn't activate" a quantified finding rather than an assertion. **Follow-ups.** What does a high-volume/low-activation channel tell you to do? / How do you separate channel quality from a within-channel mix-shift? / Why compare activation at equal cohort age across channels?

Q25. Tie it together: how do AARRR, growth loops, and the mix-vs-rate lens combine into one operating model for diagnosing growth?

Ideal answer. AARRR gives the *map* - five stages (Acquisition, Activation, Retention, Referral, Revenue) where growth can be won or lost - and the first diagnostic move is always to localize a movement to a stage so you fix the right thing. Growth loops give the *engine* view layered on top: which stages feed back into each other (referral and content loops turn Revenue/Activation back into Acquisition), so you optimize for compounding, not just plugging leaks in a linear funnel. The mix-vs-rate lens is the *microscope* applied at every stage and every rate: any aggregate metric is $\sum w_i \cdot r_i$, so before acting you decompose each movement into composition change (mix), behavior change (rate), and interaction - because they have different owners and opposite fixes, and conflating them (Simpson's paradox) is the single most common analytics error. Put together: AARRR tells you *where*, the decomposition tells you *why* (mix or rate), significance tells you *whether it's real*, dollarization tells you *how much it matters*, and a powered experiment tells you *what to do* - and the loop view keeps you investing in compounding stages. That sequence is a defensible operating model, not a dashboard. **Why Helios demonstrates it.** Helios is this operating model instantiated end to end: Monitor localizes the movement (AARRR stage), Decompose runs the mix/rate identity, the Critic refutes it, Prescribe sizes the experiment, and the Narrator ships a dollar-quantified Decision Brief - grounded in governed SQL (semantic-mcp), deterministic math (stats-mcp), and graded by an 85%-vs-45% offline benchmark. **Follow-ups.** Where would you start a diagnosis if you could only look at one AARRR stage? / How does the loop view change which leak you fix first? / What makes this an operating model rather than just reporting?

Retention

Q26. Why do experienced growth people say retention is the most important part of the AARRR funnel, not acquisition?

Ideal answer. Acquisition is a leaky-bucket problem: if retention is poor, every dollar of paid acquisition pours into a bucket that drains, so you scale spend to stand still and CAC payback never closes. Retention is the only stage that compounds - it simultaneously lifts LTV (you monetize a user over more periods), funds acquisition (a higher LTV/CAC ratio lets you bid more aggressively than competitors), and feeds Referral (retained users are the only ones around long enough to invite others). I treat retention as the multiplier on all four other AARRR stages rather than a co-equal stage, which is why the North Star for most subscription and marketplace businesses is a retained-usage metric (WAU, retained GMV), not a top-of-funnel count. The honest nuance is that for a low-frequency transactional business, "retention" reads as repeat-purchase rate and reactivation, not daily engagement, so the framing changes but the compounding logic does not. **Why Helios demonstrates it.** Helios models retention via `stats-mcp.cohort_retention` and `rfm_segment`, treating repeat-purchase/return-visit decay as a first-class diagnosable surface rather than a vanity acquisition count, consistent with the leaky-bucket

view. **Follow-ups.** When would acquisition legitimately outrank retention as the priority? / How does a strong LTV/CAC ratio mechanically let you out-bid competitors on acquisition? / What North Star would you pick for a low-frequency e-commerce store specifically?

Q27. Define a retention curve and explain what its shape tells you about product-market fit.

Ideal answer. A retention curve plots the fraction of a cohort still active in period N against N (D1, D7, D30, or week/month buckets), anchored to a defined "active" event. The diagnostic signal is whether the curve **flattens** to a non-zero asymptote or **decays to zero**: a flat tail means you have a stable, habitual core of users who will never churn for organic reasons, which is the empirical fingerprint of product-market fit; a curve that asymptotes at zero means you have no durable base and are running pure acquisition arbitrage. I read three numbers: the D1/W1 drop (onboarding and first-value delivery), the slope of the middle (the habit-forming zone), and the height of the flattened tail (the size of your retained core). The classic mistake is comparing only headline retention rates between cohorts without aligning the "active" definition and the cohort-entry event, which makes two curves non-comparable. **Why Helios demonstrates it.** `stats-mcp.cohort_retention` computes period-N survival per cohort deterministically, so the curve shape (and any flattening) is a tool output, never an eyeballed estimate, satisfying the determinism-where-it-matters principle. **Follow-ups.** How do you decide the "active" event for a transactional store vs a SaaS app? / What does a smiling (upward-sloping tail) retention curve indicate and how is it possible? / How would you statistically test whether two cohorts' tails differ?

Q28. Walk me through the leaky-bucket framing and how you'd quantify the leak.

Ideal answer. The leaky bucket models your active base as a tank: acquisition is the inflow, churn is the outflow through holes, and the water level is your active users. The level grows only when inflow exceeds outflow, so the first thing I quantify is the **net retention dynamic**: $\text{new} + \text{resurrected} - \text{churned per period}$. I size the leak as $\frac{\text{churned_users}}{\text{active_users_start_of_period}}$ and immediately decompose it - new-user churn (a first-value/onboarding hole) behaves differently from established-user churn (a value-erosion hole), and they get different fixes. The framing forces the right prioritization question: a 5-point reduction in churn is usually cheaper and more compounding than the acquisition spend required to offset the same leak, because plugging the hole improves every future cohort, not just this one. **Why Helios demonstrates it.** Helios's funnel marts and `cohort_retention` let it attribute a level-change to inflow vs outflow per segment, and `decompose_change` separates whether a retention dip is a mix artifact (more low-retention traffic acquired) or a true rate leak (existing users churning faster). **Follow-ups.** How would you distinguish a leak caused by worse acquisition quality from one caused by product degradation? / What's the formula for the equilibrium active-user level given constant inflow and a constant churn rate? / Why might plugging the leak still not grow the base?

Q29. How would you build a churn-prediction model, and what's the trap most people fall into?

Ideal answer. I'd frame it as supervised classification: define churn precisely (e.g. no purchase in the next 90 days), build a feature set from RFM (recency, frequency, monetary), engagement trajectory, and tenure, then train a calibrated model (logistic regression or gradient-boosted trees) optimized for ranking quality - PR-AUC over accuracy, because churn is class-imbalanced. The pervasive trap is **target leakage**: including features that are mechanically downstream of churn (e.g. "days since last activity" measured into the prediction window) so the model "predicts" the outcome it's secretly observing - it scores beautifully offline and is useless in production. The second trap is optimizing AUC instead of **intervenable** churn: a model that flags users who were always going to leave has no business value; you want to surface the persuadable segment where an intervention changes the outcome, which is an uplift-modeling problem, not a pure-prediction one. **Why Helios demonstrates it.** Helios stays deliberately on the diagnostic side: `rfm_segment` produces interpretable, intervenable segments (e.g. "at-risk high-value: high monetary, decaying recency") that map directly to a prescribed action, rather than a black-box churn score with no lever attached - reflecting principle 4, every finding carries an action. **Follow-ups.** What's the difference between a propensity-to-churn model and an uplift model? / How do you set the prediction-window length without leaking the label? / Why might a high-AUC churn model still lose money in production?

Q30. What is resurrection (reactivation) and how do you measure whether a resurrection program is actually working?

Ideal answer. Resurrection is bringing a churned/dormant user back to active status; in AARRR accounting it's a third inflow alongside new acquisition, and it's usually the cheapest growth dollar because the user already knows the product. I measure it as a cohort: take everyone dormant as of date T, apply the intervention, and track the **resurrection rate** = reactivated / dormant-targeted, but the number that matters is **second-life retention** - do resurrected users stick, or do they bounce back to dormant within a period? A program that resurrects 10% who all re-churn in 30 days is theatre. The correct evaluation is quasi-experimental: a holdout of dormant users who get no intervention, so you measure incremental resurrection (organic dormant users do reactivate on their own), and you net out that baseline before claiming credit. **Why Helios demonstrates it.** Helios's `rfm_segment` isolates the "lapsed" cells, and because the GA4 data is observational, Helios's `experiment-mcp` would **design and size** a resurrection holdout (power_analysis + runtime_estimate) and read it back as a pre/post difference-in-differences rather than asserting a raw before/after lift. **Follow-ups.** Why is a dormant-user holdout essential rather than a simple before/after? / How do you set the dormancy threshold that defines "resurrectable"? / What's the risk of over-messaging dormant users to resurrect them?

Q31. Distinguish retention, churn, and engagement - candidates conflate these constantly.

Ideal answer. Retention and churn are complements measured on a cohort over time - retention is the fraction still active in period N, churn is $1 - \text{retention}$ for that period, and both require a fixed cohort and an "active" definition. Engagement is an **intensity** measure within the active population - sessions per user, items viewed, DAU/MAU stickiness ratio - and a user can be retained but disengaged (logs in but does little) or highly engaged right before churning (a usage spike can precede cancellation). The practical consequence is that engagement is a **leading indicator** of future retention but is not retention itself; I use engagement trends to forecast where the retention curve is heading and to find at-risk users before the churn event fires, but I never report rising engagement as if it were proof of improved retention. **Why Helios demonstrates it.** Helios tracks `engaged_sessions` and `engagement_rate` as distinct metrics from session/return-visit retention in the semantic registry, keeping intensity and survival separate so the Diagnose agent can flag "engagement falling ahead of retention" as a leading-indicator finding. **Follow-ups.** Give an example where engagement rose but retention fell - what's the explanation? / Why is DAU/MAU a stickiness measure and not a retention measure? / How would you use an engagement leading indicator to trigger an intervention?

Q32. How do you choose retention period buckets (D1/D7/D30 vs W1/W4/M3) for a given product?

Ideal answer. The bucketing must match the product's **natural usage frequency** - the cadence at which a satisfied user would naturally return. A daily-habit product (social, news) uses D1/D7/D30 because a day is a meaningful unit; a weekly product (grocery, fitness) uses week buckets; a low-frequency transactional store like the GA4 Merch Store uses month or even quarter buckets because expecting a D1 return is nonsensical and would manufacture an artificially brutal curve. The test I apply: pick the bucket where the median active user's inter-event gap fits comfortably inside one bucket, so a single skipped period isn't immediately misread as churn. Getting this wrong is a silent error - too-tight buckets make a healthy low-frequency product look like it's hemorrhaging, and too-loose buckets hide a real daily-product problem until it's catastrophic. **Why Helios demonstrates it.** On the GA4 Merch Store's transactional, low-frequency pattern Helios uses return-visit / repeat-purchase windows appropriate to that cadence rather than DAU-style daily retention, and is explicit (a maturity point) that the ~3-month dataset window caps how long a bucket horizon it can honestly report. **Follow-ups.** What inter-event-gap distribution would justify switching from weekly to monthly buckets? / How does the 3-month GA4 window constrain your bucket choice? / How do you handle a product with two distinct usage frequencies (power users vs casual)?

Q33. A cohort's retention looks flat overall but is collapsing in one segment. How does Simpson's paradox apply, and how do you catch it?

Ideal answer. Aggregate retention $R = \sum w_i \cdot r_i$ is a volume-weighted blend of per-segment retention rates, so a severe collapse in one segment can be masked at the top line by a

simultaneous **mix shift** toward a higher-retaining segment - the classic Simpson's-paradox structure. To catch it I never trust the aggregate alone; I decompose the change into a **mix effect** ($\sum w_i \cdot r_i$, composition moved) and a **rate effect** ($\sum w_i \cdot \hat{r}_i$, in-segment behavior moved), and I drill the rate effects because those are the real behavior change. A flat aggregate with a large negative rate effect in one segment offset by a positive mix effect is exactly the failure that a dashboard's single retention number hides, and it's the situation that costs analysts a day of manual slicing. **Why Helios demonstrates it.** This is the literal centerpiece: `stats-mcp.decompose_change` splits $\hat{R}$ into mix/rate/interaction so a hidden segment collapse surfaces as a rate effect even when the top-line is flat - the exact Simpson's-paradox dissolution Helios was built around. **Follow-ups.** Write the three decomposition terms and state which one you'd drill first. / How do you guard against computing the aggregate rate as a mean of per-segment ratios? / What does a large interaction term tell you mechanically?

Q34. How would you set a retention target and tie it to a North Star metric without gaming it?

Ideal answer. I anchor retention to a North Star that captures **delivered value**, not activity - retained-usage or retained-revenue, something a user would only do if they got real value (e.g. "monthly active purchasers" rather than "monthly logins"). The target is set off the retention curve's flattening point: I commit to lifting the **asymptotic tail** (the durable core) by N points, not just the easily-gamed D1 number, because early-period retention can be juiced with notification spam that erodes the tail. To prevent gaming I pair the headline target with a guardrail metric - e.g. unsubscribe/complaint rate or per-user value - so a "win" that's actually annoyance-driven is caught. Frameworks: North Star + input metrics tree, and I prioritize the candidate retention levers with ICE/RICE so effort goes to high-confidence, high-reach interventions first. **Why Helios demonstrates it.** Helios's Prescribe agent emits **ICE/RICE-scored, powered** experiment cards for retention levers and the Critic refutes findings that would game a metric (insufficient sample, seasonality, data-quality), so a target is only "moved" by a defensible, guardrail-aware intervention. **Follow-ups.** What guardrail metric pairs with a D1-retention target? / Why is the tail height a better target than the D1 drop? / How would RICE rank a notification change vs an onboarding redesign?

Referral

Q35. Define the K-factor (viral coefficient) and explain what value range actually matters in practice.

Ideal answer. K-factor = $i \cdot c$, where i is the average number of invites each user sends and c is the conversion rate of an invite into a new active user; it's the expected number of new users each existing user begets through referral. The thresholds: $K \geq 1$ means self-sustaining exponential growth (each cohort more than replaces itself virally), which is extraordinarily rare and

usually short-lived; the realistic and still-valuable regime is **$K < 1$ but meaningful** (say 0.4-0.7), where virality acts as an **acquisition multiplier** on every other channel - a paid user who refers 0.5 others cuts your effective blended CAC by a third. The number people forget is the **viral cycle time**: a K of 0.5 with a one-day loop massively outperforms K of 0.7 with a 30-day loop, because growth compounds on cycles, not on K alone. **Why Helios demonstrates it.** Helios frames referral as a measurable loop with **i** and **c** as governed metrics in the funnel, so a referral diagnosis decomposes a K-factor move into "fewer invites sent" vs "lower invite conversion" - the same numerator/denominator discipline as the rest of the funnel. **Follow-ups.** Why is a sustained $K \geq 1$ almost never observed in real products? / How does viral cycle time change the prioritization between lifting i vs c? / How would you measure K when invites and signups are attributed imperfectly?

Q36. Walk through the standard growth loops and how a referral loop differs from a viral loop.

Ideal answer. A growth loop is a closed system where the output of one cycle becomes the input to the next, so growth reinvests in itself rather than relying on linear top-of-funnel spend; the canonical types are viral/referral loops, content/SEO loops, and paid loops (revenue funds acquisition that generates more revenue). The distinction people blur: a **viral loop** is intrinsic - the product is more valuable when shared, so usage itself generates invites (a shared doc, a multiplayer game), and the invite is the product; a **referral loop** is extrinsic - you bolt an incentive ("give \$10, get \$10") onto a product that works fine solo, and the loop runs on the incentive economics, not the product mechanic. The consequence is that viral loops are cheap and durable but only available to inherently-social products, whereas referral loops are bolt-on-able to almost anything but cost margin and degrade when the incentive stops. **Why Helios demonstrates it.** Helios itself runs on a loop framing - the action-tracking memory closes a "did-the-fix-work" loop where each diagnosis's prescribed action feeds back as a measured input to the next run, mirroring how a growth loop reinvests output as input. **Follow-ups.** Which loop type would you prioritize for a single-player productivity tool? / How do you detect when a referral loop's incentive is being arbitrated/fraudulently exploited? / Why do paid loops eventually saturate while content loops can compound longer?

Q37. How do you connect NPS to actual referral behavior - and why is NPS alone insufficient?

Ideal answer. NPS measures stated willingness to recommend (promoters minus detractors on a 0-10 scale), but stated intent is not realized behavior, so I treat NPS as a **leading proxy** and validate it against the actual referral funnel: do self-identified promoters in fact send more invites with higher conversion? The right pipeline is to instrument the **behavioral** referral funnel - invites sent, invites accepted, referred-user activation - and correlate it back to survey segments, because the gap between "would recommend" and "did recommend" is where the real lever lives (often a missing or high-friction sharing mechanic, not a satisfaction problem). NPS alone is insufficient

because it's sample-biased (who responds), it's a lagging point-in-time read, and a high NPS with no sharing affordance produces zero actual referrals; conversely you can engineer referrals from passives with good incentives. I use NPS to size the *latent* promoter pool and the behavioral funnel to find why that pool isn't converting to invites. **Why Helios demonstrates it.** Helios privileges behavioral funnel evidence over stated metrics throughout - every finding must be SQL-verified against the governed event funnel and carry a significance test, so a "promoters drive referral" claim would have to be backed by the invite-conversion data, not the survey alone. **Follow-ups.** How would you statistically test that promoters refer more than passives? / What product change closes a high-NPS, low-referral gap? / Why is NPS response bias dangerous when sizing the promoter pool?

Q38. A referral program's signups jumped but revenue didn't. How do you diagnose it?

Ideal answer. This is a classic quality-vs-quantity referral failure, and I'd decompose revenue-per-referred-user into its parts: $\text{revenue} = \text{referred_signups} \cdot \text{activation_rate} \cdot \text{purchase_rate} \cdot \text{AOV}$. A signup spike with flat revenue means the gain is concentrated upstream and lost downstream - typically the incentive attracted **incentive-seekers** (low purchase intent) or fraud, so activation and purchase rates collapsed in the referred cohort even as the count rose. I'd cohort referred vs organic users on downstream conversion and check whether the new referred mix is dominated by a low-value segment - i.e. is this a **mix shift** toward low-quality traffic (a composition artifact) rather than a real behavior change? The fix follows the diagnosis: if it's incentive-seeker mix, restructure the incentive to reward a *purchase* not a *signup*; if it's fraud, add verification gates. **Why Helios demonstrates it.** Helios's `decompose_change` would split the referred-cohort revenue move into mix (more low-value referred traffic) vs rate (referred users converting worse) - so it distinguishes "we acquired junk" from "the product broke for referred users," and prices the gap as revenue-at-risk in dollars. **Follow-ups.** What incentive structure reduces incentive-seeker abuse? / How do you separate referral fraud from genuinely low-intent referred users? / If it's a mix shift, why might the program still be worth keeping?

Q39. What metrics define a healthy referral funnel, and where do referral programs most commonly leak?

Ideal answer. The referral funnel is its own AARRR mini-funnel: share-prompt impressions -> invites sent (`i`) -> invites delivered -> invites accepted/converted (`c`) -> referred-user activation -> referred-user retention. I'd track step-to-step rates and the end-to-end **referred-user value** so the program is judged on retained revenue, not raw signups. The most common leaks are early: the share prompt is buried or mistimed (low invites-sent), or the invite landing experience is generic and high-friction (low conversion). The subtler leak is at the tail - referred users activate but don't retain because the incentive, not the value proposition, drove them in, so they never reach the habit zone. Healthy looks like a non-trivial invites-per-user, double-digit invite

conversion, and referred-user retention at parity with or above organic. **Why Helios demonstrates it.** Helios treats every funnel as step-to-step governed rates with `reached_*` monotonic flags, so a referral funnel slots into the same machinery - the Diagnose agent drills the lowest-yielding step and the Critic checks the referred-cohort sample is large enough to trust. **Follow-ups.** Which referral-funnel step would you instrument first if you could only add one event? / Why judge a referral program on referred-user retention rather than signups? / What does referred-user retention below organic tell you?

Q40. How do you attribute a referred user's value back to the referrer, and why is double-counting a risk?

Ideal answer. Attribution needs a referral edge that links referrer -> referred at signup (a referral code/link), then I credit the referred user's downstream value to the referrer's account for program ROI, while being careful that the **referred user's own value is counted once** in company-level revenue - the referrer credit is a *program-accounting* overlay, not additional revenue. Double-counting bites two ways: counting the referred user's revenue both as "organic" and "referral-driven" inflates total acquisition impact, and in multi-touch journeys a user touched by paid *and* referral can be claimed fully by both channels, summing attributed revenue above 100%. I enforce a single attribution model (e.g. first-touch or a fractional model) consistently across channels so the channel-level numbers reconcile to the company total. **Why Helios demonstrates it.** Helios's reconciliation gate (`warehouse-mcp.reconcile` , drift > 0.5% fails the finding) is exactly the defense against attribution double-counting - any channel-level revenue breakdown must sum back to the canonical total, so an over-100% attribution would fail before it ships. **Follow-ups.** First-touch vs fractional attribution for referral - when does each mislead? / How does the GA4 first-touch traffic_source gotcha complicate referral attribution? / How would you reconcile channel-attributed revenue to the company total?

Q41. When is a referral program the wrong growth lever to invest in?

Ideal answer. Referral is the wrong lever when the **prerequisite isn't met**: you need a retained, satisfied core first, because referral amplifies whatever you have - a great product spreads, a mediocre one spreads its dissatisfaction, and incentivizing referrals of a leaky product just accelerates churn-on-arrival. It's also wrong for low-frequency, low-emotional-salience purchases where users have few natural moments to share and little reason to (most people don't evangelize a one-off commodity purchase), and for products where the unit economics can't absorb a two-sided incentive without going underwater on referred-user LTV. The framework check: confirm retention/PMF first (flattening retention curve), confirm a natural sharing moment exists, and confirm referred-user LTV exceeds the incentive cost - fail any and the spend belongs in retention or a different acquisition channel. **Why Helios demonstrates it.** Helios's maturity stance applies here - it would refuse to prescribe a referral push as a top experiment if the diagnosis shows the

leak is in-funnel rate decay (a retention/conversion problem), routing effort to the highest-impact lever via ICE/RICE rather than defaulting to a fashionable tactic. **Follow-ups.** What retention threshold would you require before launching referral? / Why can referral accelerate the death of a poor product? / How do you sanity-check that referred-user LTV exceeds the two-sided incentive cost?

Q42. How would you design an experiment to measure the true incremental impact of a referral incentive?

Ideal answer. I'd run a randomized holdout: split eligible users into a treatment group that sees the incentive and a control that doesn't, then measure incremental referred-user acquisition and, crucially, incremental **retained revenue** net of incentive cost - because some referrals would have happened organically and naive before/after over-credits the program. I'd size it with a power analysis on the expected lift and baseline variance to get a minimum detectable effect and a runtime, and I'd pre-register the success metric and guardrails (referred-user retention parity, fraud rate) to avoid post-hoc cherry-picking. Where a clean randomized split is impossible, I fall back to a quasi-experiment - difference-in-differences against a comparable un-incentivized segment - and state the parallel-trends assumption explicitly. **Why Helios demonstrates it.** This is precisely `experiment-mcp`'s job: `power_analysis` + `runtime_estimate` + `design_experiment` produce a powered card, and because the GA4 data is observational, Helios is honest that it **designs and sizes** the test and reads it back quasi-experimentally (pre/post, diff-in-diff) rather than claiming to have run a live A/B. **Follow-ups.** Why does a before/after comparison over-credit the incentive? / How do you pick the minimum detectable effect for the power analysis? / What's the parallel-trends assumption and how would you check it?

Revenue

Q43. Walk me through revenue-per-session as a decomposition - why is it more useful than tracking revenue directly?

Ideal answer. Revenue-per-session decomposes cleanly as $RPS = session_conversion_rate \hat{\cdot} AOV$ - the probability a session buys, times the average dollars when it does. This is far more useful than a raw revenue line because revenue conflates volume, conversion, and basket size, so a revenue dip is ambiguous; RPS isolates **yield per unit of traffic**, and the two-factor split tells you immediately whether a movement is a **conversion** problem (fewer sessions buying) or a **monetization** problem (buyers spending less), which have completely different fixes. I extend it with $AOV = items_per_transaction \hat{\cdot} price_per_item$ when I need to know whether basket size or unit price moved. The discipline is to always compute the rate as `SUM(revenue)/SUM(sessions)` after grouping, never an average of per-segment RPS, or Simpson's paradox creeps back in. **Why Helios demonstrates it.** `revenue_per_session` is a first-class governed metric in Helios's semantic registry, and the $RPS = conversion \times AOV$ identity

is exactly the kind of multiplicative decomposition the Decompose agent runs so a revenue move is attributed to conversion vs basket before any narrative is written.

```
-- governed shape: RPS = conversion_rate * AOV, computed as ratios of sums
SELECT
  channel_group,
  SAFE_DIVIDE(SUM(revenue), SUM(sessions)) AS revenue_per_session,
  SAFE_DIVIDE(SUM(purchasing_sessions), SUM(sessions)) AS session_conversion_rate,
  SAFE_DIVIDE(SUM(revenue), SUM(transactions)) AS aov
FROM helios.marts.fct_daily_funnel
GROUP BY channel_group
```

Follow-ups. If RPS fell, how do you tell a conversion problem from a basket problem? / Why compute the rate as a ratio of sums rather than averaging segment RPS? / How would you decompose AOV one level further?

Q44. Define ARPU, AOV, and LTV and explain how they relate - candidates mix these up constantly.

Ideal answer. AOV (average order value) = $\text{revenue} / \text{transactions}$ - a per-**order** measure, blind to how often a user orders. ARPU (average revenue per user) = $\text{revenue} / \text{users}$ over a period - a per-**user** measure that folds in purchase frequency, so $\text{ARPU} \approx \text{AOV} \cdot \text{orders_per_user}$. LTV (lifetime value) is the **cumulative discounted** revenue (or margin) a user generates over their entire relationship - effectively ARPU integrated over the retention curve, $\text{LTV} \approx \text{ARPU_per_period} \cdot \text{expected_lifetime}$, where expected lifetime is driven by retention/churn. The relationship chain is the key insight: AOV is a single-transaction lens, ARPU adds frequency, and LTV adds longevity - so to grow LTV you can lift basket (AOV), buy more often (frequency), or churn less (retention), and retention is usually the highest-leverage of the three because it multiplies the whole stream. **Why Helios demonstrates it.** Helios carries `aov` and `revenue_per_user` (ARPU) as distinct governed metrics and is explicit (a maturity point) that true LTV is **deferred** - the ~3-month GA4 window can't observe a real lifetime - so it reports short-horizon ARPU-based proxies with stated assumptions rather than fabricating an LTV number. **Follow-ups.** Why is AOV alone a misleading health metric? / How does the discount rate enter the LTV formula and why does it matter? / Which of basket, frequency, or retention would you prioritize to grow LTV and why?

Q45. Why is Helios honest that LTV is "deferred," and how would you compute a defensible LTV proxy on a 3-month window?

Ideal answer. LTV requires observing a full customer lifetime, but the GA4 Merch Store dataset spans only ~Nov 2020-Jan 2021, so any "LTV" computed on it is really a 90-day cumulative revenue truncated long before the retention curve flattens - reporting it as LTV would massively

understate value and bake in a hidden truncation bias. The defensible move is to report a clearly-labeled **short-horizon proxy** - e.g. cumulative revenue-per-acquired-cohort at 30/60/90 days - and state the assumption explicitly, or fit a model (BG/NBD + Gamma-Gamma for transactional, or a survival-curve extrapolation) and surface the **confidence interval and the extrapolation assumption** rather than a false point estimate. The senior signal is naming the limitation: a fabricated LTV is worse than an honest 90-day proxy because it drives real CAC-payback decisions off a number that can't be supported. **Why Helios demonstrates it.** Refusing to fabricate LTV from a 3-month window is one of Helios's explicit honesty/maturity points - it reports short-horizon proxies with stated assumptions, which reads as senior judgment, not a gap. **Follow-ups.** What bias does truncating the retention curve introduce into a naive 90-day "LTV"? / When would a BG/NBD model be appropriate vs a simple cohort cumulative? / How would you communicate the proxy's uncertainty to a non-technical exec?

Q46. What is expansion revenue and net revenue retention, and why do investors weight NRR so heavily?

Ideal answer. Expansion revenue is additional revenue from **existing** customers - upsell, cross-sell, usage growth, larger baskets over time - as opposed to new-logo revenue. Net revenue retention (NRR) = $(\text{starting_revenue} + \text{expansion} - \text{contraction} - \text{churn}) / \text{starting_revenue}$ for a cohort; NRR > 100% means your existing base grows revenue even with zero new acquisition, which is the holy grail because it means the leaky bucket actually *fills itself*. Investors weight NRR heavily because it's the cleanest single read on durable, compounding, capital-efficient growth: a company at 120% NRR has a built-in growth engine and predictable revenue, whereas one relying entirely on new logos to offset churn is on a treadmill. For e-commerce specifically, expansion reads as rising repeat-purchase frequency and basket growth within a retained cohort rather than seat/usage upsell. **Why Helios demonstrates it.** Helios cohorts revenue with `cohort_retention` and `rfm_segment`, so it can in principle measure within-cohort revenue growth (expansion via rising frequency/AOV) vs contraction - the building blocks of an NRR read - though it's honest the short window limits the horizon over which expansion can be observed. **Follow-ups.** Why can NRR exceed 100% even while logo churn is positive? / What does expansion revenue look like for a transactional e-commerce store vs SaaS? / How does the 3-month window constrain measuring expansion?

Q47. How would you approach a pricing change analysis, and what's the elasticity trap?

Ideal answer. Pricing is the highest-leverage revenue lever because it flows almost entirely to margin, but it's the most dangerous because it moves demand. I'd estimate **price elasticity of demand** ($\% \Delta \text{ quantity} / \% \Delta \text{ price}$) per segment, because elasticity is rarely uniform - price-sensitive segments shrink while inelastic ones don't - and revenue impact = the net of the price lift times the volume contraction. The elasticity trap is treating it as a single global constant and

extrapolating linearly: elasticity is local (valid near the current price, not for a 2x move), it's confounded by promotions/seasonality/competitor moves, and a naive before/after read attributes seasonal demand swings to the price change. The correct approach is a controlled price test (randomized by cohort or geo) or, failing that, a quasi-experimental diff-in-diff with an explicit control, and I'd report the revenue and margin impact with the volume-contraction risk, not just the per-unit gain. **Why Helios demonstrates it.** Helios separates seasonality from real change via the Critic's seasonality-decoy refutation and Decompose's mix/rate split, so a "price change worked" claim is attacked for whether the revenue move is just a seasonal swing - directly defusing the elasticity-confounding trap. **Follow-ups.** Why is global constant elasticity a dangerous assumption? / How would you isolate the price effect from a concurrent seasonal demand swing? / What guardrail tells you a price rise is destroying long-run value despite short-run revenue gains?

Q48. Revenue dropped 12% week-over-week. Give me the structured framework you'd use to diagnose it.

Ideal answer. I'd decompose top-down, never guess. First the **factor split**: revenue = sessions x conversion_rate x AOV, so I attribute the 12% to traffic volume, conversion, or basket - that alone usually halves the search space. Then within whichever factor moved, a **segment decomposition** via mix/rate: is conversion down because traffic shifted toward a low-converting channel/device (mix effect, a composition artifact) or because a specific segment's conversion genuinely fell (rate effect, real behavior)? I drill the **rate effects first** because those are the actionable behavior changes, and I check seasonality and data-quality (a tracking break or a duplicated/lost shard can fake a 12% drop) before declaring a behavioral cause. Finally I **price it** in dollars of revenue-at-risk and attach a significance test so I'm not chasing noise, then prescribe the highest-ICE fix. **Why Helios demonstrates it.** This is literally the Helios pipeline end-to-end: Monitor detects the anomaly, Decompose runs the factor and mix/rate split, Diagnose does best-first RCA, Critic refutes seasonality/data-quality/sample confounds, and Narrator ships a dollar-quantified Decision Brief - the structured framework executed autonomously. **Follow-ups.** Which factor (traffic/conversion/AOV) would you check first and why? / How do you rule out a tracking/data-quality artifact before blaming behavior? / How does pricing the drop in revenue-at-risk change the prioritization?

Q49. What is revenue-at-risk and why insist every finding carries a dollar figure?

Ideal answer. Revenue-at-risk is the dollar magnitude of a diagnosed movement - the counterfactual revenue you'd have had absent the anomaly minus the actual, summed over the affected window (and projected forward if the cause persists). I insist on it because a finding without a dollar figure can't be **prioritized**: "checkout conversion dropped in Paid Search mobile" is interesting, but "that's costing \$14k/week and growing" is what decides whether it jumps the

backlog. Dollarizing also forces honesty - it converts a statistically-significant-but-tiny effect into a visibly trivial number, killing the trap of chasing significant-but-immaterial findings, and it makes the prioritization framework (ICE/RICE) operate on real impact rather than vibes. The figure must be derived deterministically from reconciled revenue, not estimated in prose, or it's just a confident-sounding guess. **Why Helios demonstrates it.** Principle 4 - "every finding carries a significance test, a dollar impact, and an action" - is non-negotiable in Helios; revenue-at-risk is computed from governed, reconciled revenue (drift > 0.5% fails) and is the field that lets Prescribe rank the experiment backlog by real money. **Follow-ups.** How do you compute the counterfactual baseline for revenue-at-risk? / Why does dollarizing protect against chasing statistically-significant-but-trivial effects? / Should revenue-at-risk be the realized loss or the projected-forward loss, and when?

Q50. How do conversion-rate optimization and AOV optimization trade off, and how do you decide which to pursue?

Ideal answer. Both lift RPS (since $RPS = conversion \times AOV$) but through opposite mechanics that can fight each other: conversion optimization removes friction and lowers the bar to buy (simpler checkout, lower entry price, free shipping) which can *drag AOV down* by pulling in smaller baskets; AOV optimization (upsells, bundles, minimum-for-free-shipping thresholds) adds steps or raises the bar and can *suppress conversion*. So I never optimize one in isolation - the objective function is **revenue-per-session**, and I evaluate any change on RPS net, watching the second factor as a guardrail. To decide which to pursue, I look at the decomposition and elasticity: if conversion is far below benchmark and baskets are healthy, the marginal dollar is in conversion; if conversion is near-ceiling and baskets are thin, it's in AOV - and I size each candidate's RPS impact and rank with ICE/RICE rather than following a default playbook. **Why Helios demonstrates it.** Because Helios decomposes **revenue_per_session** into conversion x AOV, a finding can say *which* factor is the binding constraint and Prescribe can target the lever with the larger RPS upside - and the Critic guards against a "conversion win" that's actually cannibalizing AOV by demanding the net RPS effect, not a single-factor lift. **Follow-ups.** Give a concrete change that lifts conversion but lowers AOV - is it net positive? / Why is revenue-per-session the right objective function rather than either factor alone? / How would you size the RPS impact of a free-shipping-threshold change before shipping it?

Section 5: Data Analytics Interview

This section drills the analyst craft that sits under Helios: writing correct GA4 BigQuery SQL (UNNEST, sessionization, dedup, funnels, cohorts, mix-vs-rate aggregation) and designing metrics that survive scrutiny (ratio discipline, governed definitions, reconciliation, denominators,

guardrails). The through-line is that in Helios the analyst writes none of this SQL at runtime â€” `semantic-mcp` emits it from a governed registry â€” so every answer doubles as a spec for what that registry and the dbt models must encode correctly once, forever.

SQL

Q1. The GA4 BigQuery export stores `event_params` as a repeated record. How do you pull a single parameter like `ga_session_id` out of it, and why is the syntax shaped that way?

Ideal answer. `event_params` is an `ARRAY<STRUCT<key STRING, value STRUCT<string_value, int_value, float_value, double_value>>>`, so a parameter is one row inside a nested array, not a column. I extract it with a correlated subquery over `UNNEST`, picking the right typed slot: `(SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key = 'ga_session_id')` AS `ga_session_id`. The subquery form is preferable to a top-level `CROSS JOIN UNNEST` when I want many params side-by-side on one event row, because each subquery returns a scalar and avoids fanning the event out into one row per param. The cardinal mistake is reading the wrong value slot (e.g. `string_value` for a numeric `ga_session_id`, which yields `NULL`), so I always match the type to the parameter. **Why Helios demonstrates it.** This exact pattern lives in `stg_ga4__event_params`, which unnests `event_params` once into a long key/value table so no downstream model re-implements `UNNEST`; the `get_event_param` macro centralizes the typed-slot extraction. **Follow-ups.** When would you prefer `CROSS JOIN UNNEST` over the scalar subquery? / How does `UNNEST(event_params) WITH OFFSET` help and when do you need it? / What happens to the event row if a param key is genuinely absent?

Q2. Define a GA4 session in SQL. What is the session key and why isn't `ga_session_id` alone enough?

Ideal answer. `ga_session_id` is only unique within a single user, so the session grain is the pair `(user_pseudo_id, ga_session_id)`; using `ga_session_id` alone collides sessions across users. I derive a stable single-column key by hashing the pair: `TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`, then count sessions as `COUNT(DISTINCT session_key)` rather than `COUNT(*)`, which would count events. I deliberately use `MD5`, not `FARM_FINGERPRINT`, for a stable cross-environment key, and I cast `ga_session_id` (an `INT64`) to `STRING` before concatenation so the delimiter actually separates the parts. **Why Helios demonstrates it.** `int_ga4__sessionized` is built on exactly `(user_pseudo_id, ga_session_id)` with the canonical `session_key = TO_HEX(MD5( ... ))` expression, and `sessions = COUNT(DISTINCT session_key)` is the one allowed definition repo-wide â€” it is a keystone that fails silently if wrong. **Follow-ups.** Why `MD5` over `FARM_FINGERPRINT` for a key you persist? / How do you handle events where `ga_session_id` is `NULL`? / How would you bound a session by a 30-minute inactivity timeout if you couldn't trust `ga_session_id`?

Q3. Write SQL to roll raw GA4 events up to one row per session with session-scoped source/medium.

Ideal answer. I unnest the params I need per event, then aggregate to the session grain, taking the first non-null source/medium ordered by time. `ARRAY_AGG(x IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)]` cleanly returns the earliest populated value:

```
WITH ev AS (
  SELECT
    user_pseudo_id,
    (SELECT ep.value.int_value FROM UNNEST(event_params) ep WHERE ep.key='ga_session_id') AS
      ga_session_id,
    event_timestamp,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='source') AS
      p_source,
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='medium') AS p_medium
  FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*`
)
SELECT
  user_pseudo_id, ga_session_id,
  COALESCE(ARRAY_AGG(p_source IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)],
    '(direct)') AS session_source,
  COALESCE(ARRAY_AGG(p_medium IGNORE NULLS ORDER BY event_timestamp LIMIT 1)[OFFSET(0)],
    '(none)') AS session_medium
FROM ev
WHERE ga_session_id IS NOT NULL
GROUP BY user_pseudo_id, ga_session_id
```

The `IGNORE NULLS` + `LIMIT 1` is the trick: most events carry no source param, so a plain `MIN` or unfiltered `ARRAY_AGG` would surface a NULL or an arbitrary value. **Why Helios demonstrates it.** This is the spine of `int_ga4__sessionized`; the session-scoped source/medium it produces feeds the `channel_group` macro, which the Decompose and Diagnose agents lean on to explain funnel movement. **Follow-ups.** Why first-non-null and not last? / How do `'(direct)'` / `'(none)'` defaults affect downstream channel grouping? / What if two events share an `event_timestamp`?

Q4. The event-level `traffic_source` struct is right there. Why not just group sessions by it?

Ideal answer. In the GA4 export the event-level `traffic_source STRUCT<name, medium, source>` is *user first-touch attribution* – it's stamped from the user's first acquisition and copied onto every later event for that `user_pseudo_id`. A user acquired via Organic Search who returns months later via Email still carries `medium='organic'` on the new events, so grouping sessions by it systematically over-credits acquisition channels and under-credits re-engagement. Worse, it manufactures fake mix-shift that masquerades as rate-change. The correct source is the first non-null `source` / `medium` from `event_params` within the session itself; event-level `traffic_source` is acceptable only as a documented fallback when session params are entirely NULL, and as the

basis for a genuinely user-level `first_touch_channel`. **Why Helios demonstrates it.** Helios codifies this as a hard rule â€” "never use event-level `traffic_source` for session-scoped channel grouping" â€” and derives session source/medium from `event_params` in `int_ga4__sessionized`, precisely to stop a phantom Simpson's-paradox confound from polluting the Decompose agent. **Follow-ups.** When is event-level `traffic_source` actually the right thing to read? / How would `collected_traffic_source` change your derivation in newer exports? / How do you test that your session source matches GA4's own UI numbers?

Q5. Write SQL that flags whether a session reached each funnel step, guaranteeing the funnel never inverts.

Ideal answer. I compute per-session "reached" flags as *max-downstream* monotonic flags: a session reached `add_to_cart` if it fired `add_to_cart` or any later event. Doing it monotonically guarantees `sessions ≥ reached_view_item ≥ ... ≥ reached_purchase`, so every step rate is ≤ 1 by construction.

```
SELECT
  session_key,
  MAX(event_name = 'view_item'           OR reached_later) AS reached_view_item,
  MAX(event_name = 'add_to_cart'         OR reached_later) AS reached_add_to_cart,
  MAX(event_name IN ('begin_checkout', 'add_shipping_info', 'add_payment_info', 'purchase')) AS
    reached_begin_checkout,
  MAX(event_name = 'purchase')           AS reached_purchase
FROM events_with_session
GROUP BY session_key
```

In practice I implement the monotonicity by including each downstream event in the upstream flag's predicate (as in `reached_begin_checkout` above), which is why the names are `reached_*`, not `did_*`. The funnel is `session_start` → `view_item` → `add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase`. **Why Helios demonstrates it.** `int_ga4__funnel_steps` materializes exactly these monotonic `reached_*` flags (the retired `did_*` names are banned for this reason), making the funnel a keystone that downstream rates can trust. **Follow-ups.** Why are non-monotonic per-step flags dangerous for diagnosis? / How do you handle a session that purchases without an `add_to_cart` event? / Where do you store `session_revenue` so daily rollups keep dollar impact?

Q6. Compute step-to-step conversion rates and overall session conversion from the funnel flags.

Ideal answer. I aggregate the boolean flags with `COUNTIF` and divide adjacent stages, computing each rate as a ratio of summed counts at the group level â€” never an average of per-session ratios.

```

SELECT
  COUNTIF(reached_view_item) AS view_item_sessions,
  SAFE_DIVIDE(COUNTIF(reached_add_to_cart), COUNTIF(reached_view_item)) AS view_to_cart_rate,
  SAFE_DIVIDE(COUNTIF(reached_begin_checkout), COUNTIF(reached_add_to_cart)) AS
    cart_to_checkout_rate,
  SAFE_DIVIDE(COUNTIF(reached_purchase), COUNTIF(reached_begin_checkout)) AS
    checkout_to_purchase_rate,
  SAFE_DIVIDE(COUNTIF(reached_purchase), COUNT(DISTINCT session_key)) AS
    session_conversion_rate
FROM fct_funnel

```

`SAFE_DIVIDE` prevents divide-by-zero from blanking a whole row, and `COUNTIF(flag)` is cleaner than `SUM(CASE WHEN flag THEN 1 ELSE 0 END)`. `session_conversion_rate = purchasing_sessions / sessions` is the headline. **Why Helios demonstrates it.** These are canonical metrics (`view_to_cart_rate`, `cart_to_checkout_rate`, `checkout_to_purchase_rate`, `session_conversion_rate`) defined once in `semantic_layer.yaml`; the agent never types this SQL â€” `semantic-mcp.build_query` emits it from the registry. **Follow-ups.** Why `SAFE_DIVIDE` over `/` ? / What does `cart_abandonment_rate` equal in these terms? / How do micro-segment zero-denominators corrupt a naive average-of-ratios?

Q7. Dedup is a classic GA4 trap. How do you guarantee one row per transaction for revenue?

Ideal answer. GA4 can emit duplicate `purchase` events (retries, double-fires), so revenue must be deduped on `transaction_id`. I keep the first event per `transaction_id` with a window function, then sum:

```

WITH ranked AS (
  SELECT
    (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE ep.key='transaction_id') AS
      transaction_id,
    ecommerce.purchase_revenue_in_usd AS revenue,
    ROW_NUMBER() OVER (
      PARTITION BY (SELECT ep.value.string_value FROM UNNEST(event_params) ep WHERE
        ep.key='transaction_id')
      ORDER BY event_timestamp
    ) AS rn
  FROM `...events_*`
  WHERE event_name = 'purchase'
)
SELECT transaction_id, revenue FROM ranked WHERE rn = 1 AND transaction_id IS NOT NULL

```

I deliberately exclude NULL/empty `transaction_id` rows (which can't be a real order) and use `_in_usd` revenue so I never aggregate a non-USD twin column. **Why Helios demonstrates it.** `fct_orders` is grain-exactly one row per `transaction_id` with a `unique + not_null` dbt test on the key, and a reconciliation test asserts `SUM(revenue)` matches the source to the cent â€”

the verify-then-trust backbone. **Follow-ups.** `ROW_NUMBER` vs `QUALIFY` vs `DISTINCT` here “ when does each break? / How would you detect duplicate transactions before deduping? / Why exclude empty-string `transaction_id` ?

Q8. Show how `QUALIFY` simplifies window-function dedup and where it doesn't apply.

Ideal answer. `QUALIFY` filters on a window function without a wrapping subquery, so the Q7 dedup collapses to one statement: `SELECT ... FROM purchases QUALIFY ROW_NUMBER() OVER (PARTITION BY transaction_id ORDER BY event_timestamp) = 1`. It's BigQuery-specific sugar that runs after `SELECT / WINDOW`, the way `HAVING` runs after `GROUP BY`. It does *not* help when I need the deduped set as a CTE consumed multiple times, or when the dedup key itself is a heavy correlated subquery I'd rather compute once “ there I keep an explicit CTE for readability and to avoid recomputing the subquery. For “latest record wins” I just flip the `ORDER BY` to `DESC`. **Why Helios demonstrates it.** Helios uses `QUALIFY` for single-pass dedup in marts but keeps explicit CTEs in `int_ga4__sessionized` where the same param extraction feeds several columns “ readability and cost both matter under the 5 GiB byte budget. **Follow-ups.** What's the evaluation order of `QUALIFY` relative to `WHERE / GROUP BY / HAVING` ? / When does `ROW_NUMBER` vs `RANK` vs `DENSE_RANK` change your result? / How do ties in `ORDER BY` make dedup non-deterministic and how do you fix it?

Q9. Write a cohort-retention query: users grouped by first-active week, retained in later weeks.

Ideal answer. I anchor each user to a cohort by their first activity, then count distinct users active in each subsequent period offset, and divide by the cohort's size.

```
WITH first_seen AS (
  SELECT user_pseudo_id,
         MIN(DATE_TRUNC(DATE(TIMESTAMP_MICROS(event_timestamp)), WEEK)) AS cohort_week
  FROM `...events_*` GROUP BY user_pseudo_id
),
activity AS (
  SELECT DISTINCT user_pseudo_id,
                 DATE_TRUNC(DATE(TIMESTAMP_MICROS(event_timestamp)), WEEK) AS active_week
  FROM `...events_*`
)
SELECT f.cohort_week,
       DATE_DIFF(a.active_week, f.cohort_week, WEEK) AS week_offset,
       COUNT(DISTINCT a.user_pseudo_id) AS retained_users,
       SAFE_DIVIDE(COUNT(DISTINCT a.user_pseudo_id),
                   MAX(COUNT(DISTINCT a.user_pseudo_id)) OVER (PARTITION BY f.cohort_week)) AS
       retention_rate
FROM first_seen f JOIN activity a USING (user_pseudo_id)
GROUP BY 1, 2
```

The `MAX( ... ) OVER (PARTITION BY cohort_week)` grabs week-0 size (the largest cohort count) as the denominator, giving a clean triangle. **Why Helios demonstrates it.** Cohort logic is owned by `stats-mcp.cohort_retention` (the only math path), not hand-SQL'd in a prompt; the dataset's ~3-month window is exactly why Helios reports short-horizon retention proxies and explicitly defers true LTV. **Follow-ups.** Why is cohort retention only a proxy on a 90-day window? / How does cookie-grain `user_pseudo_id` (`user_id` mostly NULL) bias retention up or down? / How would you turn this into rolling vs classic retention?

Q10. Explain the mix-vs-rate aggregation rule. Why is `SUM(num)/SUM(den)` not the same as `AVG(rate)` ?

Ideal answer. A correct aggregate rate is `SUM(numerator)/SUM(denominator)` computed *after* grouping – equivalently a volume-weighted average of segment rates, $R = \sum_i w_i \cdot r_i$ where `w_i` is segment volume share. `AVG(per-segment rate)` weights every segment equally regardless of size, so a tiny segment with a wild rate distorts the headline and you get Simpson's paradox: each segment's rate can rise while the blended rate falls because traffic mix shifted toward low-rate segments. So I never average ratios:

```
-- right: SAFE_DIVIDE(SUM(purchasing_sessions), SUM(sessions))
-- wrong: AVG(SAFE_DIVIDE(purchasing_sessions, sessions))
```

This is also why I keep numerator and denominator as separate additive columns through every rollup, deferring the division to the very last step. **Why Helios demonstrates it.** This is Helios's core Simpson's-paradox defense, encoded as a rule ("compute rates as `SUM(num)/SUM(den)` after grouping, never an average of per-segment ratios") and operationalized by `stats-mcp.decompose_change`, which splits $\hat{R}$ into `mix + rate + interaction`. **Follow-ups.** Show the mix/rate/interaction decomposition algebra. / Why carry numerator and denominator separately to the top of the rollup? / Give a concrete case where every segment improves but the total declines.

Q11. A daily funnel mart must support both trend lines and segment decomposition. How do you structure the aggregation?

Ideal answer. I aggregate to `(day ~ segment-dimensions)` and store *additive components* – counts, not pre-divided rates – so any rate can be recomputed by summing num and den across whatever slice the question needs. A row is keyed by `day`, `device_category`, `channel_group`, `country` etc., carrying `sessions`, `reached_*` counts, `transactions`, and `revenue`. Rates are computed at read time with `SAFE_DIVIDE(SUM(num), SUM(den))`. Critically, this mart must aggregate the *session-grain fact* that already carries `session_revenue`, not the raw funnel-step table, or the dollar figures vanish.

```
SELECT day, device_category, channel_group,
COUNT(DISTINCT session_key) AS sessions,
COUNTIF(reached_purchase) AS purchasing_sessions,
SUM(session_revenue) AS revenue
FROM fct_funnel GROUP BY 1,2,3
```

Why Helios demonstrates it. `fct_daily_funnel` aggregates `fct_funnel` (which carries `session_revenue`) â€” explicitly not `int_ga4__funnel_steps` â€” because dropping revenue here would break the eval's dollar-at-risk labels; this is called out as a keystone. **Follow-ups.** Why must the mart store counts, not rates? / What grain do you choose and how does cardinality affect cost? / How does partitioning by `event_date` + clustering by `device_category`, `channel_group` help here?

Q12. How do you keep BigQuery scans cheap on a date-sharded `events_*` table?

Ideal answer. The wildcard table is sharded by date, so I prune with `_TABLE_SUFFIX BETWEEN '20201101' AND '20210131'` to scan only the needed shards â€” a `WHERE` on a derived `event_date` does not prune shards. In the dbt marts (which are partitioned by `event_date` and clustered by `device_category`, `channel_group`) I filter on the partition column to prune partitions and on cluster keys to prune blocks. I never `SELECT *` on a wide nested table; I project only the params I unnest. And I run a `dry_run` first to read the byte estimate before executing, narrowing the window or dimensions if it's over budget â€” never blindly retrying. **Why Helios demonstrates it.** `warehouse-mcp` is the sole BigQuery client and enforces a mandatory `dry_run` $\rightarrow$ `run_query` byte-budget gate (a fixed 5 GiB/run budget); over-budget queries get narrowed, not retried, and marts are partitioned/clustered exactly as above. **Follow-ups.** Why doesn't `WHERE event_date BETWEEN ...` prune wildcard shards? / Partitioning vs clustering â€” what does each prune? / How does `dry_run` change your query-authoring loop?

Q13. Helios's LLM never hand-writes SQL â€” `semantic-mcp` emits it. As an analyst, what does that change about how you work, and what's the tradeoff?

Ideal answer. It moves the craft from "write a query per question" to "encode each metric and dimension *once*, correctly, in a governed registry," then compose validated SQL from those primitives. The payoff is zero hallucinated columns, guaranteed consistent definitions across every brief, and free reconciliation â€” the same `session_conversion_rate` everywhere. The tradeoff is reduced ad-hoc flexibility: a genuinely novel question needs a registry change (a reviewed PR), not a quick one-off query, and the registry must be expressive enough (metrics Ã— dimensions Ã— filters Ã— windows) to cover real questions or analysts route around it. I accept that friction because correctness and trust beat speed-to-first-query for an autonomous system whose numbers ship to executives. **Why Helios demonstrates it.** `semantic-mcp.build_query` is the *only* path to SQL; it composes from the 47-metric/19-dimension registry in

`semantic_layer.yaml`, so an unknown name is a hard error, not a fallback to free SQL “
directly serving the 0-hallucination target. **Follow-ups.** How do you keep the registry expressive without it becoming a SQL-in-YAML escape hatch? / What's your process for adding a new governed metric? / Where does ad-hoc exploration legitimately still happen?

Metric Design

Q14. What is "ratio discipline" and why does every analytics team eventually get burned without it?

Ideal answer. Ratio discipline is the practice of treating every rate as an explicit numerator/denominator pair with a stated grain, and aggregating it as `SUM(num)/SUM(den)` “ never storing or averaging a pre-divided ratio. Teams get burned because pre-divided rates can't be re-aggregated (you can't sum percentages across days to get the period rate), and averaging ratios silently triggers Simpson's paradox. The discipline also forces clarity on the denominator: "conversion rate" is meaningless until you say sessions vs users vs add-to-carts. I always answer three questions before a rate ships: what's the numerator, what's the denominator, and at what grain were they counted. **Why Helios demonstrates it.** Every Helios rate metric in `semantic_layer.yaml` is defined as an explicit numerator/denominator with grain, and the repo rule mandates `SUM(num)/SUM(den)` after grouping “ the mix-vs-rate decomposition literally depends on this. **Follow-ups.** Why can't you sum daily conversion rates to get the monthly rate? / Give two metrics that share a numerator but differ only by denominator. / How does this rule interact with weighting in `decompose_change` ?

Q15. Walk through choosing the denominator for "conversion rate" in an ecommerce funnel.

Ideal answer. The denominator encodes the question. Session conversion rate uses `sessions` (the acquisition+UX question: of all visits, how many bought). User conversion rate uses distinct users (the demand question, but on GA4 it's cookie-grain since `user_id` is mostly NULL, so it's an approximation). Step rates use the *prior* step's count (`add_to_cart_sessions` / `view_item_sessions`) to isolate where in the funnel friction lives. Picking the wrong denominator makes a metric directionally misleading: a session-denominated rate falls when bot/direct traffic spikes even if buying behavior is unchanged. I default to session-grain for funnel diagnosis because the question is "what happened to the visits that converted." **Why Helios demonstrates it.** Helios's canonical funnel is session-scoped (`session_conversion_rate` = `purchasing_sessions` / `sessions`) with step rates denominated on the prior `reached_*` count, and user-level metrics are explicitly flagged as cookie-based approximations. **Follow-ups.** When is user-denominated conversion the right choice despite cookie-grain noise? / How do you denominate `checkout_to_purchase_rate` and why? / What does a rising session count do to a session-denominated rate?

Q16. What does it mean for a metric to be "governed," and what does a governed definition contain?

Ideal answer. A governed metric is defined exactly once in a versioned registry â€” name, numerator, denominator/aggregation, the dimensions it's sliceable by, allowed filters, and its grain â€” and every consumer reads that single definition rather than re-deriving it. Governance means a metric name resolves to one and only one SQL expression system-wide, an unknown name is a hard error (not a silent free-SQL fallback), and changing a definition is a reviewed code change with a clear blast radius. This is what kills "my conversion rate doesn't match yours" arguments: there is one `session_conversion_rate` and it's compiled, not retyped. **Why Helios demonstrates it.** `models/semantic/semantic_layer.yaml` holds 47 metrics + 19 dimensions, referential-integrity compiled, and is the single source of truth `semantic-mcp` reads; it maps 1:1 to dbt MetricFlow so the governance is mechanically enforced. **Follow-ups.** What's the difference between a metric registry and a BI tool's saved queries? / How does referential-integrity compilation catch a typo'd dimension? / How does this map onto dbt MetricFlow / semantic models?

Q17. What is metric reconciliation and how do you build it into a pipeline?

Ideal answer. Reconciliation asserts that a transformed/aggregated metric equals an independently-computed canonical total within a tight tolerance â€” proving the marts didn't silently drift from the raw source. I implement it as a test, not a hope: `SUM(revenue)` from the `orders` fact must equal `SUM(purchase_revenue_in_usd)` from the raw export, and the build/finding fails if drift exceeds tolerance. Revenue I reconcile to the cent (zero tolerance); rates and counts to a small percentage. The key is it runs automatically every build and gates the pipeline, so a regression can't ship unnoticed. **Why Helios demonstrates it.** `warehouse-mcp.reconcile` enforces aggregates against canonical totals with a >0.5% drift failing the finding, and `fct_orders` carries a custom singular test reconciling revenue to the source to the cent â€” the operational form of "verify-then-trust." **Follow-ups.** Why zero tolerance for revenue but a percentage for rates? / What's a realistic cause of drift that reconciliation catches? / Where in the run does reconcile execute and what does it block?

Q18. A stakeholder says conversion rate dropped 12%. Before diagnosing, what do you check about the metric itself?

Ideal answer. First, relative vs absolute: 12% relative (e.g. 2.0% -> 1.76%) is very different from 12 percentage points. Second, denominator stability â€” did the rate "drop" only because the denominator (sessions) grew from a traffic surge or a new low-intent channel, i.e. mix-shift, not behavior? Third, definition drift: did a tracking change, a new bot filter, or a redefinition move the metric rather than reality? Fourth, sample size and seasonality: is the window large enough to be significant, and is this just a weekday/holiday pattern? Only after ruling those out do I decompose

into mix vs rate and drill the rate effects. This sequencing stops me from "diagnosing" a measurement artifact. **Why Helios demonstrates it.** This is the Helios pipeline order: Monitor detects the anomaly, Decompose splits mix vs rate, then Diagnose drills *rate* effects first (real behavior) before mix (composition artifacts), and the Critic explicitly attacks seasonality and data-quality explanations before a finding ships. **Follow-ups.** How do you tell mix-shift from rate-change numerically? / What guardrail metric would confirm a data-quality artifact? / How big a sample do you need before 12% is significant?

Q19. Define guardrail metrics and give an ecommerce example set.

Ideal answer. Guardrail metrics are health metrics you watch alongside (and counter to) the metric you're trying to move, to ensure a "win" isn't bought by harming something else or by a measurement glitch. For a checkout-conversion experiment, guardrails include AOV and revenue-per-session (did conversion rise only by discounting away margin?), refund/cancellation rate, page-load latency, and add-to-cart rate upstream (did we just shift where the drop-off lives?). They're also data-quality tripwires: a sudden swing in sessions or null-rate flags instrumentation breakage, not behavior. The discipline is to declare guardrails *before* shipping a change so you're not retrofitting an excuse. **Why Helios demonstrates it.** Helios prescribes powered experiment cards with guardrails baked in via `experiment-mcp`, and the Critic's adversarial checks (insufficient sample, data-quality, seasonality) function as automated guardrails before any finding is trusted. **Follow-ups.** Which guardrail catches "conversion up because we discounted"? / How do guardrails differ from the primary metric statistically? / How would you alert on a guardrail breach?

Q20. How do you design a "revenue-at-risk in dollars" metric from a rate change?

Ideal answer. I translate a rate movement into money by multiplying the rate delta by the volume it acts on and the value per conversion: roughly $\text{revenue_at_risk} \approx \hat{\Delta}(\text{conversion_rate}) \times \text{sessions_in_window} \times \text{AOV}$, but only for the *rate* portion of the change, because mix-driven movement isn't a behavior problem to fix. I price the counterfactual "had the rate held, we'd have earned X more" annualize or window it explicitly, and attach the confidence interval from the significance test so the dollar figure carries uncertainty. Crucially I attribute it to the specific segment the decomposition isolates, not the blended top line, so the number points at an action. **Why Helios demonstrates it.** Quantifying revenue-at-risk in dollars is a Helios headline output; it uses `stats-mcp.decompose_change` to isolate the rate effect (not mix) before pricing it, so the dollar figure reflects fixable behavior, and every finding must carry a dollar impact to count as a finding. **Follow-ups.** Why price only the rate effect, not the full $\hat{\Delta}R$? / How do you put a confidence interval on the dollar figure? / How would you avoid double-counting risk across overlapping segments?

Q21. How do you avoid vanity metrics, and what makes a metric "actionable"?

Ideal answer. A vanity metric goes up and to the right but doesn't change a decision â€” total pageviews, raw session counts in isolation. An actionable metric is tied to a lever someone owns, is decomposable to a segment you can target, carries a baseline + significance so you know the move is real, and implies a next step. My test: "if this metric moves, what specifically would we do?" â€” if there's no answer, it's a reporting number, not a decision metric. I pair every headline rate with the absolute volume and dollar value so a 0.1pp move isn't dressed up as a crisis. **Why Helios demonstrates it.** Helios's fourth non-negotiable is "every finding is actionable" â€” a finding without a significance test, a dollar impact, and a recommended action is not a finding; the Prescribe agent converts each into a powered experiment card. **Follow-ups.** Give a metric that's vanity in one context and actionable in another. / How does significance testing separate signal from a vanity wiggle? / What's the minimum a metric needs to drive an experiment?

Q22. How do you handle small-denominator / sparse-segment metrics so they don't mislead?

Ideal answer. Tiny denominators produce wildly noisy rates that dominate naive averages and trigger false anomalies, so I (1) never average per-segment ratios â€” I aggregate `SUM(num)/SUM(den)` ; (2) apply a minimum-sample threshold before a segment's rate is allowed to drive a finding; (3) use a significance test, not just point estimates, so a 1/3 = 33% segment isn't treated like a 30000/90000 one; and (4) suppress or roll up segments below the threshold into an "Other" bucket. The point is to weight by volume and demand evidence proportional to the claim. **Why Helios demonstrates it.** Helios's Critic explicitly refutes "insufficient sample" hypotheses, `stats-mcp.significance_test` gates findings, and the volume-weighted `SUM(num)/SUM(den)` rule stops a sparse segment from hijacking the blended rate. **Follow-ups.** What minimum sample do you require and how do you choose it? / How does a confidence interval on a small segment look vs a large one? / When is rolling up to "Other" the wrong move?

Q23. Two dashboards report different numbers for the same metric. How do you diagnose and prevent it?

Ideal answer. I diff the definitions first: numerator, denominator, grain, filters (bot exclusion, date boundary, timezone), and the dedup rule. The usual culprits are different denominators (sessions vs users), one side averaging ratios while the other sums components, a timezone/date-cutoff mismatch, or one missing the `transaction_id` dedup. The fix isn't to reconcile by hand each time â€” it's to make both read one governed definition so the metric *can't* diverge, then add a reconciliation test that fails loudly if they do. Prevention beats forensics. **Why Helios demonstrates it.** This is the entire rationale for the semantic registry: one compiled definition per metric in `semantic_layer.yaml` consumed via `semantic-mcp` , plus `reconcile` as the automated tripwire â€” so "your number â‰‰ my number" is structurally impossible. **Follow-ups.** Which single mismatch causes the most "different numbers" incidents? / How does

timezone/date-boundary choice silently fork a metric? / How would you audit two existing dashboards quickly?

Q24. How do you design a metric to be robust to seasonality and known calendar effects?

Ideal answer. I separate the metric's *definition* from its *interpretation*: the metric stays a clean rate, and I handle seasonality at comparison time – year-over-year or same-day-of-week comparisons, a seasonal baseline/forecast the actual is judged against, and an explicit launch/holiday calendar so a Black Friday spike isn't flagged as an anomaly. I also prefer windows that span whole weeks to avoid weekday-mix artifacts. The metric shouldn't be "deseasonalized" by baking adjustments into its definition (that hides drift); instead the anomaly detector compares against a seasonal expectation. **Why Helios demonstrates it.** Helios keeps a seasonality/launch calendar in `helios_memory`, uses `stats-mcp.forecast` (prophet) to set seasonal baselines, and the Critic specifically attacks "this is just seasonality" before a finding ships – exactly this separation of definition from interpretation. **Follow-ups.** Why not bake a seasonal adjustment into the metric definition? / How does a forecast baseline beat a fixed prior-period comparison? / How do you encode and apply a launch calendar?

Q25. After a fix ships, how do you measure whether it worked when you can't run a clean A/B?

Ideal answer. With observational, historical data I can't randomize, so I use quasi-experimental readbacks: pre/post comparison against the forecasted counterfactual, and difference-in-differences comparing the treated segment to a comparable untreated control segment over the same window (which nets out shared seasonality/macro effects). I pre-register the metric, guardrails, and expected effect size from the original power analysis, then check whether the realized lift clears the significance bar and the confidence interval excludes zero – being explicit that this is causal *evidence*, not proof, and stating the assumptions (parallel trends, no concurrent confound). Honest framing of the design's limits is the senior move. **Why Helios demonstrates it.** Because the GA4 dataset is observational and historical, Helios designs and *sizes* experiments via `experiment-mcp` and runs pre/post and difference-in-differences readbacks rather than live A/Bs, and the action-tracking loop in `helios_memory` closes the "did-the-fix-work" question. **Follow-ups.** What's the parallel-trends assumption in diff-in-diff and how do you check it? / How does pre-registering the effect size guard against p-hacking the readback? / Where does the action-tracking memory store the outcome and how does it inform the next diagnosis?

Root Cause Analysis

Q26. Walk me through how you do structured root-cause analysis on a metric drop, versus just "looking at the data."

Ideal answer. Structured RCA replaces eyeballing with a decomposition-first discipline: I never start from "what looks suspicious," I start from an arithmetic identity that the total *must* obey, then attribute the change to its mathematically exhaustive parts. For a funnel-rate drop I first localize *which step* moved (each step rate is `numerator_sessions / upstream_sessions`), then decompose *that* step's change across segments into mix, rate, and interaction so I know whether traffic composition or in-segment behavior moved, and only then drill the largest *rate* contributors into a hypothesis tree. The discipline is: localize -> decompose -> attribute -> verify, where each stage narrows the search space before the next, so I never chase a segment that is merely a composition artifact. The failure mode I am avoiding is the "largest absolute delta" reflex - picking the segment that fell the most in raw count, which is dominated by your biggest-volume segment and is wrong roughly half the time.

Why Helios demonstrates it. This is the exact Monitor -> Decompose -> Diagnose pipeline: `detect_anomaly` localizes, `decompose_change` attributes, and the Diagnose agent runs a best-first hypothesis tree over the rate contributors - and the offline benchmark scores this at $\geq 85\%$ root-cause accuracy versus $\leq 45\%$ for the naive largest-delta baseline.

Follow-ups. How do you decide when to stop drilling the tree? / What stops the LLM from inventing a plausible-sounding cause that the data does not support? / How would this change if the metric were revenue rather than a rate?

Q27. Explain mix-shift versus rate-change to me as if I were a non-technical PM, then give me the math.

Ideal answer. Plain version: imagine overall conversion dropped. Two very different things could have happened - either *each kind of visitor* converts worse than before (rate-change, a real behavior problem you can fix), or *each kind still converts the same* but you simply got more low-converting visitors, say a cheap paid-social surge (mix-shift, a traffic-composition story, not a product problem). They demand opposite actions, so conflating them wastes analyst-weeks. The math: for an aggregate rate $R = \sum_i w_i \cdot r_i$ (segment volume share times segment rate), the change splits exactly as $\Delta R = \sum_i \Delta w_i \cdot r_i(t_0)$ (mix) + $\sum_i w_i(t_0) \cdot \Delta r_i$ (rate) + $\sum_i \Delta w_i \cdot \Delta r_i$ (interaction). The decomposition is exhaustive and exact - the three terms sum to the total change - which is what lets me say with confidence "78% of the drop is rate, not mix." That exactness is also how you dissolve Simpson's paradox: a metric can fall in aggregate while rising in every segment purely because the mix shifted toward low-rate segments.

Why Helios demonstrates it. `stats-mcp.decompose_change` computes these three terms deterministically in Python (never in token space), and the Diagnose agent is hard-wired to drill rate-effects before mix-effects - because rate is real behavior and mix is often a composition artifact.

Follow-ups. When is a mix-shift itself the actionable finding rather than a distraction? / How do you handle the interaction term when you summarize for an exec? / What if a segment appears in t1 but not t0 (zero baseline weight)?

Q28. What is a hypothesis tree in RCA and how do you keep it from exploding combinatorially?

Ideal answer. A hypothesis tree is a structured search where the root is the observed anomaly and each node is a candidate explanation that, if true, would account for some quantified share of the movement; children refine a parent into more specific sub-causes (e.g., "checkout-step drop" - > "mobile checkout drop" -> "mobile Safari checkout drop"). You keep it from exploding by making it *best-first and quantified*: you only expand the branch carrying the largest unexplained rate-effect, you prune any branch whose attributable share is below a materiality floor, and you cap depth once a node explains enough of the delta to be decision-grade. This is fundamentally different from a breadth-first crawl of every dimension combination, which is both expensive and noisy. The governing principle is that the decomposition gives every node a *number*, so expansion is driven by magnitude of unexplained variance, not by curiosity.

Why Helios demonstrates it. The Diagnose agent (Opus) runs exactly this best-first hypothesis-tree RCA, expanding the highest-rate-effect branch and SQL-verifying each node via semantic-mcp before it commits - so the tree is both bounded and grounded, never a free-form LLM ramble.

Follow-ups. How do you set the materiality floor without it being arbitrary? / What if two branches each explain 40% - how do you report a split cause? / How do you avoid double-counting when child segments overlap?

Q29. Why are "5 Whys" and similar qualitative frameworks insufficient for analytics RCA?

Ideal answer. 5 Whys is a useful *framing* heuristic but it has two fatal gaps for analytics: it has no notion of *magnitude* (each "why" is asserted, not measured, so you can walk a chain that explains 3% of the drop while feeling complete), and it is single-threaded (it assumes one causal chain, when real funnel moves are usually several causes summing to the total). Quantified attribution fixes both: every candidate cause carries an attributable share derived from the decomposition, the shares are mutually exclusive and sum to ~100% of the delta, and you rank by dollar or rate magnitude rather than by narrative plausibility. I still *use* the 5-Whys instinct to generate candidate causes, but I refuse to ship a cause without a number attached. The senior move is to treat qualitative frameworks as hypothesis generators and quantified decomposition as the adjudicator.

Why Helios demonstrates it. Principle 4 of the project - "a finding without a significance test, a dollar impact, and a recommended action is not a finding" - is enforced structurally: the Narrator

literally cannot emit a cause that lacks a `decompose_change` share and a `significance_test` verdict behind it.

Follow-ups. Give me a case where the qualitatively obvious cause was quantitatively trivial. / How do you communicate that a cause is "real but immaterial"? / Where does 5 Whys still earn its keep in your workflow?

Q30. How do you distinguish a real behavioral change from a confounder?

Ideal answer. A confounder is a variable correlated with both your segment split and the outcome, such that an apparent in-segment rate change is actually a composition effect inside that segment. My defense is layered: first the mix/rate/interaction split already separates composition from behavior at the chosen segmentation; second, I re-decompose along a *plausibly-confounding* dimension (if a "Paid Social rate drop" survives controlling for device and new-vs-returning, it is more credible); third, I demand the rate-effect be statistically significant given the segment's sample size, so I am not chasing noise. The conceptual point is that no single decomposition proves causation - I am stacking refutation attempts, and a finding earns trust by *surviving* the obvious confounders rather than by being asserted. Honest caveat: on observational data I can never fully close the causal loop, which is why the output is a *designed experiment*, not a causal claim.

Why Helios demonstrates it. This is precisely the Critic agent's job - it runs adversarial refutation against four named failure modes (mix-confound, insufficient sample, seasonality, data-quality) and returns a verdict of PASS / DOWNGRADE / DROP, so a finding only ships if it survives the confound attack.

Follow-ups. Which confounder bites GA4 funnel analysis most often? / How do you choose which dimension to re-decompose against? / When do you accept a finding as DOWNGRADE rather than DROP?

Q31. How do you separate seasonality from a genuine anomaly?

Ideal answer. Seasonality is expected, recurring variation; an anomaly is a deviation from the seasonally-adjusted expectation. So the right test is never "is today lower than yesterday" - it is "is today lower than what a model that already knows the weekly/holiday pattern predicted, by more than the prediction interval." I fit a forecast that captures weekly seasonality and holiday effects, take the residual, and only flag points whose residual exceeds a band scaled to historical noise. A naive week-over-week or day-over-day comparison will scream every Monday and every post-Black-Friday Tuesday. The maturity point is also maintaining a *known-events calendar* so that a promotion or a launch is treated as an explained deviation, not a mystery to diagnose.

Why Helios demonstrates it. `stats-mcp.forecast` (prophet/statsmodels) supplies the seasonally-aware expectation that `detect_anomaly` flags against, and the memory layer keeps seasonality and launch calendars so the Critic can DROP a finding that is just a known seasonal swing - directly relevant given the dataset's Nov-2020-to-Jan-2021 holiday window.

Follow-ups. How much history do you need before a seasonal model is trustworthy here? / How do you handle a one-off event with no prior precedent? / What is the cost of a false-positive anomaly to the analyst's trust in the system?

Q32. How do data-quality issues masquerade as anomalies, and how do you guard against them?

Ideal answer. Most "anomalies" in raw event data are tracking artifacts, not behavior: a tag deploy that drops `add_to_cart` events, a late-arriving partition that makes yesterday look low, a bot surge inflating sessions, or a consent change suppressing a channel. These produce textbook funnel drops that fool a naive system into prescribing a product fix for a measurement bug. My guard is a battery of cheap sanity checks before causal interpretation: reconcile totals against a canonical source, check for implausible step-rate values (>1 means broken monotonic flags), watch for a channel or device going abruptly to zero (a tracking signature, not a behavior signature), and confirm the partition is complete. The principle: rule out the instrument before you diagnose the patient.

Why Helios demonstrates it. Two structural guards - warehouse-mcp's mandatory `reconcile` ($>0.5\%$ drift fails the finding) and the `reached_*` max-downstream monotonic flags that make step rates impossible to exceed 1 - plus a data-quality bucket in the 50-scenario eval that includes pure data-quality artifacts the pipeline must label as such rather than diagnose.

Follow-ups. How would you detect a partial tag-drop that only affects mobile? / What is your reconciliation tolerance and why that number? / How do you keep the system from "crying wolf" on every late partition?

Q33. Give me your end-to-end RCA narrative for a single concrete anomaly.

Ideal answer. Say `session_conversion_rate` falls 12% week-over-week. (1) Localize: I decompose the overall rate into step rates and find the drop concentrates at `begin_checkout` → `add_payment_info`, not upstream. (2) Decompose that step across `device_category` x `channel_group`: the split returns ~80% rate-effect, ~15% mix, ~5% interaction - so this is behavior, not composition. (3) Drill the largest rate contributor: mobile + a specific channel, and the hypothesis tree narrows to mobile checkout. (4) Significance: confirm the mobile rate delta clears its sample-size threshold. (5) Refute: control for new-vs-returning to kill the confound, and check the calendar to rule out seasonality and a tag deploy. (6) Quantify: price the surviving drop

as dollars of revenue-at-risk via lost purchasing sessions x AOV. (7) Prescribe: a powered experiment on the mobile payment step. (8) Ship: a Decision Brief leading with the dollar number and the one recommended action. The whole point is that each step is a tool output, not a vibe.

Why Helios demonstrates it. This is the literal agent handoff chain - Monitor, Decompose, Diagnose, Critic, Prescribe, Narrator - passing a typed JSON "Finding" envelope, completing in under 5 minutes per run versus the 1-3 analyst-days the manual version costs.

Follow-ups. Where in that chain is the most likely place to be wrong? / How do you report it if the cause is genuinely split across two segments? / What would you do differently if you only had three days of history?

Business Analysis

Q34. How do you size the business impact of a funnel anomaly in dollars?

Ideal answer. Sizing converts a rate movement into money so it competes for attention on a common scale. The clean approach is counterfactual: estimate the conversions (or sessions) lost relative to the seasonally-adjusted expectation, then monetize them at the relevant value-per-conversion. For a checkout-rate drop: $\text{revenue_at_risk} = \text{lost_purchasing_sessions} \times \text{AOV}$, where $\text{lost_purchasing_sessions} = \text{affected_sessions} \times \hat{\Delta} \text{rate}$. I am careful to (a) size only the *rate* portion of the move, not the mix portion, since mix-driven changes are often not recoverable by a product fix; (b) bound the estimate with the significance interval so it reads as a range, not false precision; and (c) annualize cautiously, flagging that the dataset is a 3-month window. A concrete sketch:

```
-- revenue-at-risk from a rate drop at one funnel step (illustrative)
SELECT
  affected_segment,
  upstream_sessions,
  (rate_t0 - rate_t1) AS delta_rate,
  upstream_sessions * (rate_t0 - rate_t1) AS lost_purchasing_sessions,
  upstream_sessions * (rate_t0 - rate_t1) * aov AS revenue_at_risk_usd
FROM step_decomposition
WHERE effect_type = 'rate' -- price behavior, not composition
ORDER BY revenue_at_risk_usd DESC;
```

Why Helios demonstrates it. Every finding carries a dollar impact by design (Principle 4), priced off the rate-effect from `decompose_change` and AOV from the governed registry - and the eval even labels dollar-at-risk, which is why `fct_daily_funnel` must carry `session_revenue` rather than dropping it.

Follow-ups. Why size only the rate portion and not the whole delta? / How do you turn a point estimate into a defensible range? / How would you adjust the AOV if the affected segment has a different basket than the overall average?

Q35. How do you prioritize a backlog of findings when you can't act on all of them?

Ideal answer. Prioritization needs a single comparable score, and I build it from impact, confidence, and effort. Impact is the dollar revenue-at-risk (or upside) already computed; confidence is how well the finding survived refutation and how tight its significance interval is; effort/cost is the implementation and experiment runtime. A defensible ranking multiplies expected dollar impact by a confidence weight and divides by effort - essentially an ICE/RICE variant where "Reach" and "Impact" collapse into the quantified dollar number rather than a 1-5 guess. The discipline that separates this from typical RICE is that the inputs are *measured*, not voted: impact comes from decomposition, confidence from the Critic's verdict and the significance test, and runtime from a power analysis. The failure mode I avoid is the loudest-stakeholder problem, where bets are ranked by conviction rather than by defensible expected value.

Why Helios demonstrates it. The Prescribe agent emits powered experiment cards, and prioritization combines the dollar revenue-at-risk with the Critic's PASS/DOWNGRADE verdict and experiment-mcp's `runtime_estimate` - so the backlog is ranked by defensible expected impact per unit of effort, not by argument.

Follow-ups. How do you weight a high-impact / low-confidence finding against a low-impact / high-confidence one? / What happens to DOWNGRADE findings in the ranking? / How do you stop the ranking from always favoring the biggest-volume segment?

Q36. Translate a metric move into an actual business decision for me.

Ideal answer. A metric move only matters once it changes a decision, so I force every diagnosis to terminate in a recommended action with an owner and an expected payoff. The structure is: here is what moved (the step and segment), here is why (the dominant rate-effect, refuted against confounders), here is what it costs (dollars of revenue-at-risk), and therefore here is what we should do and what we expect from it (a specific experiment with a powered sample size and an upside estimate). For the mobile-checkout example: "We are losing $\sim X/\text{week}$ to a mobile payment - step drop concentrated in returning users; recommend an A/B on a simplified mobile payments sheet, powered to detect a 2pt lift, 14 - day runtime, expected recovery Y." That is a decision a PM can approve or reject, not a chart they have to interpret. The senior point: the deliverable is a *decision*, and the metric is just the evidence.

Why Helios demonstrates it. The Narrator's Decision Brief leads with the dollar number and one prioritized action backed by a Prescribe experiment card - the entire product is built to close the "insight-to-action gap" rather than report the insight and stop.

Follow-ups. What do you do when the right action is "do nothing - it's a mix-shift we expected"? / How do you set the recommendation if the experiment would take longer than the window of opportunity? / Who is the owner of a recommendation and how do you track whether it was acted on?

Q37. How do you handle a metric that is "statistically significant but practically irrelevant," or the reverse?

Ideal answer. Significance and materiality are orthogonal and you must report both. A huge sample can make a 0.1pt rate change "significant" while it is worth almost nothing in dollars; conversely a chunky, expensive movement in a small segment may fail significance yet still deserve a watch-flag. My rule is to gate on *both* - significance establishes the effect is real, the dollar size establishes it is worth a decision - and to never let one stand in for the other. When something is significant but immaterial I explicitly label it "real but not worth acting on" so it does not consume a slot in the backlog; when it is material but underpowered I prescribe *collecting more data* (or a longer experiment) rather than acting blind. The honesty here reads as senior: I am refusing to let a p-value masquerade as importance.

Why Helios demonstrates it. Helios separates the two channels structurally - `stats-mcp.significance_test` decides "real," `decompose_change` plus AOV decides "material" - and the Critic can DOWNGRADE a statistically-significant-but-immaterial finding so it does not crowd out a bigger fish.

Follow-ups. What materiality threshold do you set and how do you justify it? / How do you explain a non-significant but expensive movement to an exec without alarming them? / When does "collect more data" become an excuse for inaction?

Q38. A stakeholder insists "conversion is down, fix it." How do you scope the real problem before committing analyst time?

Ideal answer. I resist jumping to a fix and instead run a fast triage that scopes the problem in minutes: is the drop real (versus seasonality or a late partition), where is it (which step), what kind is it (mix versus rate), how big is it (dollars), and is it even ours to fix (a tracking artifact?). Most "conversion is down" panics dissolve at this stage - it is a known seasonal dip, or a tag deploy, or a paid-traffic surge diluting the mix rather than a product regression. Only the residual that survives triage earns a deep dive and a prescribed experiment. This protects the scarcest resource, analyst time, and it reframes the stakeholder conversation from "fix it" to "here is precisely what

moved and whether it warrants action." The mature framing is that *scoping is the deliverable* for half of these requests.

Why Helios demonstrates it. This triage is exactly the Monitor -> Decompose -> Critic front of the pipeline, which can rule out seasonality, data-quality, and mix-shift before the expensive Diagnose/Prescribe stages ever run - turning a 1-3-day investigation into a <5-minute autonomous run.

Follow-ups. How do you push back on a stakeholder without sounding dismissive? / What is the cheapest check that kills the most false alarms? / How do you log the triage so the same panic doesn't get re-investigated next week?

Q39. How do you quantify the upside of a fix, not just the downside of the problem?

Ideal answer. Downside is the revenue-at-risk you can recover; upside is the realistic share of that you expect a specific intervention to recapture, bounded by the experiment's minimum detectable effect. I never assume a fix recovers 100% of the loss - I size the recoverable portion as `expected_uplift = affected_sessions x detectable_rate_lift x value_per_conversion`, where the detectable lift comes from the power analysis, and I present it as a range. This is what makes the prescription honest: the experiment is powered to detect the *smallest lift worth the effort*, and the upside is tied to that floor rather than to wishful thinking. The senior nuance is that you also subtract the experiment's own cost and opportunity cost, so the recommendation is net-positive expected value, not just "directionally good."

Why Helios demonstrates it. experiment-mcp's `power_analysis` and `runtime_estimate` set the detectable effect and duration, so each Prescribe card's upside is grounded in what the experiment can actually prove - not a fabricated recovery number.

Follow-ups. How do you set the minimum detectable effect - business floor or statistical convenience? / What if the powered runtime exceeds the seasonal window the opportunity exists in? / How do you account for novelty effects inflating early uplift?

Dashboarding

Q40. When do dashboards genuinely help, and when do they fail?

Ideal answer. Dashboards excel at *monitoring known questions*: tracking a small set of agreed KPIs over time, confirming the business is on-plan, and giving everyone a shared, governed number. They fail at *diagnosis* - the moment a metric moves, a dashboard can show you the *what* but it cannot tell you *why*, because answering "why" requires an open-ended search through segment combinations that no fixed set of tiles can enumerate. The classic failure is the dashboard sprawl: dozens of charts, each answering a question nobody is currently asking, while

the actual root-cause investigation still happens in an analyst's ad-hoc notebook over the next two days. So dashboards are a *pull* surface for surveillance of known metrics; they are structurally incapable of the open-ended causal search that RCA demands, and pretending otherwise is why "self-serve BI" so often fails to reduce analyst load.

Why Helios demonstrates it. Helios is explicitly *not* a dashboard - it occupies the diagnosis gap a dashboard cannot fill, running the open-ended mix/rate hypothesis search a fixed tile set never could, and treating conversation as a secondary drill-down rather than the product.

Follow-ups. Where would you still put a dashboard alongside Helios? / Why doesn't adding more dashboard filters solve the why-problem? / What's the maintenance cost of a 40-tile dashboard nobody reads?

Q41. Explain push versus pull in analytics delivery and why it matters.

Ideal answer. Pull means a human must remember to go look (open the dashboard, run the query) - which means insights are found only when someone happens to be looking and already suspects something. Push means the system surfaces the insight to the person *when it occurs*, unprompted. The why-it-matters: most anomalies in a pull model are caught late or never, because monitoring 28 metrics across 16 dimensions by hand is not something humans do consistently. A push model inverts the burden - the default is "you will be told when something material moves and why," so attention is spent on *deciding*, not on *hunting*. The tradeoff to manage is alert fatigue: a push system is only trusted if its precision is high, which is why suppression, materiality gating, and seasonality-awareness are not optional features but the cost of being allowed to push at all.

Why Helios demonstrates it. Helios's heartbeat is the *autonomous scheduled run* (push), not a queried dashboard (pull); the memory layer's suppression list and the Critic's seasonality/data-quality DROPs exist specifically to keep push precision high enough to be trusted.

Follow-ups. How do you tune the materiality floor to balance recall and alert fatigue? / What's the first thing that erodes trust in a push system? / When would a user still want to pull rather than be pushed to?

Q42. Why is your "Decision Brief" an anti-dashboard, and what does it contain?

Ideal answer. A dashboard hands you evidence and makes *you* do the synthesis; a Decision Brief does the synthesis and hands you a *decision*. It inverts the cognitive load. The brief leads with the conclusion - what moved, the dollar revenue-at-risk, and the single recommended action - then provides the supporting chain underneath for anyone who wants to audit it: the decomposition shares, the significance verdict, the confounders ruled out, and the powered experiment design. It is the pyramid principle applied to analytics: answer first, evidence second. The reason this is the

anti-dashboard is that its unit of delivery is a recommendation a leader can approve or reject in one read, whereas a dashboard's unit of delivery is raw charts that still require an analyst-in-the-loop to interpret. The discipline is ruthless: if the brief cannot be acted on, it failed, no matter how rich the underlying analysis.

Why Helios demonstrates it. The Narrator agent renders the Decision Brief via report-mcp's `render_brief`, leading with the dollar number and one prioritized, experiment-backed action - the literal end-state that distinguishes Helios from a BI tool.

Follow-ups. How do you keep the brief honest when the finding is ambiguous? / What goes "above the fold" versus in the audit trail? / How long should an exec spend reading one?

Q43. How do you design a dashboard that supports diagnosis rather than just reporting, if you had to build one?

Ideal answer. If forced to build a diagnostic dashboard, I'd design it around the decomposition workflow rather than around metric tiles: a top-line anomaly indicator against a seasonally-adjusted expectation, then a drill path that always shows the mix/rate/interaction split for the selected metric and lets you pivot the segmentation dimension, with every number traceable to a governed definition. The key design choice is to encode the *analysis path* (localize -> decompose -> drill) into the navigation, not to scatter independent charts. But I'd be honest in the interview that this is a *crutch* - it speeds a human through the standard path but still relies on the human to drive the search and stop at the right node, which is exactly the labor an autonomous system should remove. So I'd build it as a drill-down surface attached to an automated engine, not as the primary product.

Why Helios demonstrates it. This mirrors Helios's deliberate ordering - localize, decompose, drill rate-before-mix - and its decision to make conversation/drill-down a *secondary* surface bolted onto the autonomous run, never the main deliverable.

Follow-ups. What governed definitions would you enforce on every tile? / How do you prevent the drill path from becoming a 30-click maze? / Where's the line between a helpful drill-down and re-inventing the analyst's manual grind?

Q44. What's the strongest argument *for* dashboards, and how do you reconcile it with building an anti-dashboard?

Ideal answer. The strongest case for dashboards is shared situational awareness with a governed number: an org needs one trusted place where everyone sees the same KPIs, defined the same way, updated reliably - that is real and valuable, and Helios does not replace it. I reconcile it by seeing them as different jobs: dashboards answer "are we on track" (monitoring known metrics, a solved problem), while Helios answers "why did we go off track and what do we do" (diagnosis

and prescription, the unsolved problem). The mistake is asking a dashboard to do the second job by piling on filters until it becomes an unusable ad-hoc query tool. So I'm not anti-dashboard dogmatically - I'm against the *category error* of expecting a reporting surface to do causal diagnosis. The two coexist: governed monitoring up front, autonomous diagnosis behind it.

Why Helios demonstrates it. Helios deliberately scopes itself to the diagnosis/prescription gap and leans on the same governed definitions (the 28-metric / 16-dimension semantic registry) a good dashboard would use - so it complements monitoring rather than competing with it.

Follow-ups. Where exactly do you draw the boundary between the two tools? / How do you keep both honoring the same metric definitions? / Have you ever seen a dashboard succeed at diagnosis? What made it work or fail?

Analytics Workflows

Q45. Why does a governed metrics layer matter, and what breaks without one?

Ideal answer. A governed metrics layer is a single, versioned registry where every metric and dimension is defined exactly once, so "conversion rate" means the same thing in every query, brief, and dashboard. Without it you get *metric drift*: three analysts compute "conversion" three ways (sessions vs users in the denominator, engaged vs all sessions), the numbers disagree in a leadership meeting, and trust in the entire data function erodes. The deeper failure for an AI system is hallucination - an LLM left to write SQL will invent plausible column names and silently wrong joins. Governing the metrics means the machine can only *compose* from validated definitions, never author free SQL, which is the difference between an analyst tool you can trust unattended and a demo. A subtle but critical rule the layer enforces: compute rates as `SUM(num)/SUM(den)` after grouping, never as an average of per-segment ratios - the latter quietly reintroduces Simpson's paradox.

Why Helios demonstrates it. semantic-mcp is the *only* path to SQL, composing from `semantic_layer.yaml` (47 metrics, 19 dimensions, referential-integrity compiled, 1:1 with dbt MetricFlow) - which is how Helios hits 0 hallucinated columns and why an unknown metric name is a hard error, not a fallback to free SQL.

Follow-ups. How do you add a new metric without letting it drift? / Why is "average of ratios" specifically dangerous? / How do you version a metric definition when its meaning legitimately changes?

Q46. How do you make an analysis reproducible?

Ideal answer. Reproducibility means anyone can re-run the analysis and get the same numbers, which requires controlling four things: the data (a frozen or version-pinned snapshot, not a live

shifting table), the definitions (the governed metric layer, so the SQL is regenerated identically), the computation (deterministic, seeded math rather than hand calculations or stochastic LLM arithmetic), and the parameters (the exact window, filters, and thresholds, captured as code/config). The enemy of reproducibility is hidden state - an analyst's local notebook with an un-pinned query and a mental note about which week they used. I push all four into governed, versioned artifacts so the run is a function of explicit inputs. The maturity point: an analysis you cannot reproduce is an anecdote, and an autonomous system that is not reproducible cannot be trusted to run unattended.

Why Helios demonstrates it. Determinism-where-it-matters is a core principle - all math runs seeded in real Python via stats-mcp (never token-space), SQL is regenerated from the versioned semantic registry, and the eval injects anomalies into a *frozen* GA4 copy so runs are exactly reproducible and CI-gated.

Follow-ups. Why route math through Python instead of letting the LLM compute it? / How does freezing the eval dataset help reproducibility? / What's the hardest source of hidden state to eliminate in practice?

Q47. The "analyst-time problem" - what is it and how does automation actually help?

Ideal answer. The analyst-time problem is that the scarcest resource in a data team is senior analyst attention, and it gets consumed by *repetitive, mechanical* RCA - the same localize/decompose/drill grind on every anomaly, costing 1-3 days each - leaving little time for the genuinely novel, strategic work only a human can do. Automation helps not by replacing the analyst but by *eliminating the mechanical 80%*: the system handles the deterministic decomposition, the significance testing, the dollar sizing, and the first-pass prescription, and escalates to the human only the ambiguous or strategic residual. The trap is automating the *easy* part (a chart) instead of the *expensive* part (the diagnosis), which is most BI tooling's mistake. Done right, automation shifts the analyst from *producing* the analysis to *adjudicating and acting on it* - higher leverage per hour.

Why Helios demonstrates it. Helios automates exactly the expensive part - the full Monitor-to-Narrator diagnosis chain in <5 minutes versus 1-3 analyst-days - and leaves the human the high-value act of approving the prescribed experiment, which is the right division of labor.

Follow-ups. Which part of RCA should *not* be automated and why? / How do you measure the analyst-time actually saved? / What's the risk of analysts deskilling if the machine does the diagnosis?

Q48. What does a healthy analytics workflow look like end to end, and where does automation slot in?

Ideal answer. A healthy workflow is a governed pipeline: raw events land, get modeled into clean, tested marts with a layered transformation DAG, surface through a single semantic layer, and feed both monitoring (push alerts on materiality) and on-demand diagnosis - with every output traceable back to a governed definition and a reproducible run. Automation slots in at two places: continuous monitoring (so anomalies are detected without a human watching) and first-pass diagnosis (so the why-analysis is drafted before an analyst opens it). The human stays in the loop for adjudication, strategic framing, and the act of deciding. The principle binding it together is *governance flows downstream* - if the metric layer is the single source of truth, then alerts, diagnoses, briefs, and dashboards are all guaranteed consistent, and you have eliminated the reconciliation meetings that eat data teams alive.

Why Helios demonstrates it. Helios is that pipeline made concrete: GA4 -> dbt staging/intermediate/marts -> the `semantic_layer.yaml` layer -> the autonomous scheduled run -> the Decision Brief, with warehouse-mcp's reconcile gate enforcing consistency to $\leq 0.5\%$ the whole way down.

Follow-ups. Where is the single most leveraged place to add governance? / How do you keep the dbt DAG and the semantic layer from diverging? / What part of this workflow is hardest to get an org to adopt?

Q49. How do you build trust in an automated analytics system so people actually act on its output?

Ideal answer. Trust is earned by being *correct, transparent, and accountable*, and you have to engineer all three. Correctness: ground every number in governed definitions and deterministic math, and prove accuracy on a labeled benchmark rather than asserting it. Transparency: show the work - the decomposition shares, the significance verdict, the confounders ruled out - so a skeptic can audit any claim rather than being asked to take it on faith. Accountability: close the loop by tracking whether the system's recommendations, once acted on, actually worked - a system that grades itself in production is one you can trust to run unattended. The failure mode is the black-box oracle that is occasionally spectacularly wrong and offers no audit trail; one such miss destroys adoption permanently. So the hard part was never generating insights - it was making them correct and *trusted enough to act on*.

Why Helios demonstrates it. All three pillars are built in: governed-SQL grounding (semantic-mcp) + deterministic stats (stats-mcp) for correctness, the auditable Finding envelope and Decision Brief for transparency, and the Critic plus the 85%-vs-45% offline benchmark and the did-the-fix-work action-tracking loop for accountability.

Follow-ups. What single piece of evidence convinces a skeptical analyst fastest? / How does the did-the-fix-work loop change behavior over time? / What would one high-profile wrong call do,

and how do you recover trust?

Q50. Where does automation *not* belong in analytics - what do you deliberately keep human?

Ideal answer. Automation belongs on the mechanical, well-specified, repeatable parts - decomposition, significance testing, sizing, seasonality adjustment - and explicitly *not* on three things: causal claims on observational data (the machine can suggest, but only a designed experiment can confirm), strategic prioritization across competing business objectives (which weighs goals the data cannot see), and novel-situation judgment with no precedent to learn from. I deliberately keep the human as the adjudicator who approves prescriptions and owns the decision, because accountability for a business action should sit with a person. The senior framing is that I automate to *concentrate* human judgment on the decisions that deserve it, not to remove the human - an autonomous diagnosis engine whose output a human rubber-stamps is fine; one that auto-executes a pricing change is reckless. Knowing where to stop is itself the senior skill.

Why Helios demonstrates it. Helios deliberately *designs and sizes* experiments rather than auto-running live A/Bs - honest about the dataset being observational and historical - and the Decision Brief presents a recommendation for a human to approve, never an action it executes autonomously.

Follow-ups. Give me a decision you'd never let Helios make alone. / How do you encode "this needs a human" into the system? / Where's the line between autonomous diagnosis and autonomous action, and why does it matter?

Section 6: Analytics Engineering Interview

This section covers the analytics-engineering layer that everything else in Helios stands on: the dbt project that turns raw GA4 events into governed marts, the semantic layer that is the single source of truth for every metric, and the dimensional modeling decisions (grain, layering, star schema, sessionization) that make the numbers correct and cheap to query. Throughout, I demonstrate each concept in general first and then ground it in the Helios dbt project, `models/semantic/semantic_layer.yaml` (47 metrics, 19 dimensions), and the `fact_*` / `dim_*` marts.

dbt

Q1. Walk me through how you structured a dbt project and chose materializations for each layer.

Ideal answer. I structure dbt as a strict one-directional DAG “sources to staging to intermediate to marts to a semantic layer” and pick the materialization per layer by trading off freshness, storage, and rebuild cost. Staging is `view` (thin rename/cast, no storage, always fresh); reusable business logic in intermediate is `ephemeral` so it inlines as CTEs and doesn't proliferate objects; the core day-grain facts are `incremental` so I only reprocess new partitions; small finance/growth facts are `table` because a full rebuild is cheap; and the semantic layer is `view` so it reflects marts instantly. The governing principle is: materialize where reuse or query cost justifies storage, otherwise stay a view. **Why Helios demonstrates it.** Helios' `dbt_project.yml` sets `staging: {+materialized: view}`, `intermediate: {+materialized: ephemeral}`, `marts.core: incremental`, `finance/growth: table`, `semantic: view` “exactly this layered policy, defaulted at the directory level so individual models stay clean. **Follow-ups.** When would you promote an ephemeral intermediate model to a table? / Why not materialize staging as tables to speed downstream reads? / What breaks if the semantic layer were a table instead of a view?

Q2. Explain incremental models in dbt and when you'd use one.

Ideal answer. An incremental model only transforms and appends/replaces new data on each run instead of rebuilding the whole table, gated by `is_incremental()` and a predicate that limits the source scan to recent rows. You use it when the table is large and append-mostly and a full rebuild is too slow or too expensive “typical of event/fact tables. The key design choices are the incremental *strategy* (how new rows are reconciled with existing ones), the unique key (for merge), and a lookback window wide enough to absorb late-arriving data. The cost saving is real but it introduces state, so I always keep `--full-refresh` working as the safe reset. **Why Helios demonstrates it.** Helios' core funnel facts (`fct_funnel`, `fct_daily_funnel`) are `incremental`, partitioned by `event_date`, with an `is_incremental()` predicate `where event_date ≥ date_sub(dbt_max_partition, interval 3 day)` so a freshly landed `events_YYYYMMDD` shard is processed without re-reading ~90 days of history. **Follow-ups.** How do you pick the lookback window? / What happens on the very first run of an incremental model? / How do you detect and recover from incremental drift?

Q3. Compare the `insert_overwrite` and `merge` incremental strategies on BigQuery. Which did you choose and why?

Ideal answer. `merge` matches new rows to existing rows on a unique key and updates/inserts row-by-row “flexible but requires a reliable surrogate key and pays a join cost. `insert_overwrite` atomically replaces *entire partitions* that appear in the new batch, which is cheaper on a date-partitioned table and inherently idempotent: reprocessing a corrected shard simply overwrites that day's partition with no dedup logic. I chose `insert_overwrite` partitioned by `event_date` because the GA4 export is date-sharded, so “the unit of correction is a day” maps

perfectly onto "the unit of overwrite is a partition." That guarantees byte-identical output on re-runs without ever needing surrogate-key dedup. **Why Helios demonstrates it.**

```
{{ config(materialized='incremental',
incremental_strategy='insert_overwrite',
partition_by={'field':'event_date','data_type':'date','granularity':'day'}) }}
```

Helios uses exactly this config on core marts; a late or corrected GA4 shard reprocesses cleanly because the whole `event_date` partition is replaced. **Follow-ups.** What's the risk if a late event lands outside your lookback window with `insert_overwrite` ? / When would `merge` be unavoidable here? / How does `insert_overwrite` interact with BigQuery's `require_partition_filter` ?

Q4. How do partitioning and clustering work on BigQuery, and how did you choose your partition and cluster keys?

Ideal answer. Partitioning physically segments a table (here by `DATE`) so a query with a date predicate prunes whole partitions and never reads them â€” coarse but huge savings. Clustering sorts data within each partition by chosen columns so block-level pruning further drops bytes on filters and joins on those columns â€” a fine-grained second filter. I partition on the column every query filters and cluster on the highest-cardinality, most-frequently-filtered dimensions. The two compose: partition pruning first, then cluster pruning. **Why Helios demonstrates it.** Every Helios fact partitions by `event_date` (the diagnosis window is always a date range, so a 14-day diagnosis scans ~14 of ~90 partitions) and clusters by `device_category`, `channel_group` â€” the two dimensions the Decompose agent slices on most when separating mix effect from rate effect. I also set `require_partition_filter = true` so a query missing a date predicate errors instead of full-scanning. **Follow-ups.** Why not partition by `event_date` and cluster by it too? / What's the cardinality limit on a cluster key before it stops helping? / How would you partition if queries filtered on `channel_group` more than date?

Q5. What are dbt macros, and how have you used them to enforce consistency?

Ideal answer. A macro is a reusable Jinja function that emits SQL, letting me define a transformation once and call it everywhere â€” the DRY principle applied to SQL. The high-value use is encoding business logic that must be identical across models: extracting a typed event parameter, building a key, or classifying a dimension. Centralizing it means a rule change is a one-file edit and there's no risk of two models drifting apart. The tradeoff is that over-macro'd SQL becomes hard to read, so I reserve macros for logic that is genuinely repeated or genuinely error-prone. **Why Helios demonstrates it.** Helios has four macros â€” `get_event_param(key, type)` (one canonical UNNEST extraction), `sessionize()` (builds the `(user_pseudo_id, ga_session_id)` key), `channel_group_case()` (the single source of truth for all 10-channel

classification), and a custom `revenue_reconciles` test. Because `channel_group_case()` lives in exactly one macro, an attribution-rule change can never silently diverge between `int_ga4__sessionized` and the marts. **Follow-ups.** How do you unit-test a macro? / When does a macro belong in a package instead of the project? / How do you document a macro so a reviewer trusts it?

Q6. How do you use dbt packages, and which have you relied on?

Ideal answer. Packages are versioned dependencies declared in `packages.yml` and pulled via `dbt deps`; they ship reusable macros and generic tests so I don't reinvent common utilities. The discipline is pinning a version range so a transitive update can't silently change my output, and preferring battle-tested packages over hand-rolled SQL for things like surrogate-key generation, date spines, and range/expression tests. I keep the dependency surface small "every package is code I'm implicitly trusting in my build." **Why Helios demonstrates it.** Helios pins `dbt-labs/dbt_utils` (`>=1.1.0, <2.0.0`) and uses it for generic tests like `dbt_utils.accepted_range` on `revenue` (min 0) "keeping schema tests declarative while the project's own logic stays in custom macros and a custom `revenue_reconciles` generic test.

Follow-ups. Why pin a version range rather than a single version? / Would you use `dbt_utils.generate_surrogate_key` or hand-roll `TO_HEX(MD5( ... ))`, and why? / What's the risk of relying on a package macro that changes behavior across versions?

Q7. What's the difference between `dbt run`, `dbt test`, and `dbt build`, and which do you run in CI?

Ideal answer. `dbt run` executes model SQL (materializes models); `dbt test` runs data/schema tests against already-built models; `dbt build` interleaves them in DAG order "it runs each model and then immediately runs that model's tests before proceeding, and also handles seeds and snapshots. `build` is the right default because it stops the pipeline at the first model whose tests fail instead of building a whole broken downstream graph on bad data. In CI I run `dbt build` so a contract violation blocks the merge; ad hoc I might `dbt run --select` a subgraph while iterating. **Why Helios demonstrates it.** Helios' production refresh and CI both use `dbt build` (`--select state:modified+` in CI), so a reconciliation or schema test failing on `fct_orders` halts the build and blocks the agent run rather than letting the diagnosis pipeline reason over silently-wrong marts. **Follow-ups.** Why does `build` catch bugs `run` then `test` would miss? / How does `build` order tests relative to downstream models? / When would you deliberately `run` without `test`?

Q8. Explain Slim CI in dbt and how you implemented it.

Ideal answer. Slim CI builds and tests only the models that changed plus their downstream children, instead of the whole project, by comparing the PR's code against a stored production

manifest (`state:modified+`) and deferring unbuilt upstream refs to prod (`--defer --state`). This cuts CI runtime and warehouse cost dramatically on large projects while still proving that a change and everything it touches is valid. The prerequisite is publishing a fresh prod `manifest.json` artifact that CI can diff against. **Why Helios demonstrates it.**

```
- run: dbt build --select state:modified+ --defer --state ./prod-manifest
- run: dbt test --select state:modified+
- run: python eval/run_benchmark.py # root-cause accuracy gate ≥85%
```

Helios' GitHub Actions CI uses `state:modified+ --defer` to keep PR builds under the fixed BigQuery byte budget, then layers the labeled eval gate on top so a change that drops root-cause accuracy below 85% (vs the $\leq 45\%$ naive baseline) fails the PR. **Follow-ups.** How do you keep the prod manifest fresh? / What does `--defer` actually do to unbuilt upstream refs? / What's a change Slim CI would *miss* that a full build would catch?

Q9. How do you guarantee a dbt model is idempotent, and why does it matter for an autonomous system?

Ideal answer. Idempotent means re-running over the same input window yields byte-identical output “no duplicates, no drift, no dependence on run count or wall-clock. I get it by making each run a function of the data window only: deterministic keys, partition-atomic overwrite (so a partition is fully replaced, never appended-to twice), and no `current_timestamp`-style nondeterminism baked into stored columns. It matters for an autonomous system because the scheduler will re-run on retries and late shards; without idempotency a transient failure quietly corrupts history and every downstream finding inherits the error. **Why Helios demonstrates it.** Helios pairs `partition_by event_date` with `insert_overwrite` so reprocessing a day overwrites exactly that partition, and the `revenue_reconciles` test asserts `sum(revenue)` from `fct_orders` equals the raw source total to the cent “any non-idempotent drift fails the build and blocks the agent run. **Follow-ups.** How would `merge` without a stable key break idempotency? / Where might a non-deterministic function sneak into a “deterministic” model? / How do you test idempotency in CI?

Semantic Layers

Q10. What is a semantic layer and why does an analytics org need one?

Ideal answer. A semantic layer is a governed, central definition of metrics and dimensions “names, formulas, grains, and allowed slices” that sits between physical tables and every consumer (BI tools, notebooks, APIs, agents). Its job is to make “revenue” or “conversion rate” mean exactly one thing everywhere, so two dashboards can't compute it two ways. Organizationally it ends the “whose number is right?” debate, lets analysts self-serve without re-

deriving SQL, and gives one place to fix a definition when the underlying schema changes. The cost is upfront modeling discipline and the governance to keep ad hoc SQL from bypassing it.

Why Helios demonstrates it. Helios' semantic layer is `models/semantic/semantic_layer.yaml` â€” 47 metrics and 19 dimensions defined once â€” and it's not advisory: `semantic-mcp` is the *only* path to SQL, so no consumer (including the LLM) can compute a metric off-definition.

Follow-ups. How is a semantic layer different from a well-modeled mart? / Who should own metric definitions â€” analytics engineering or the business? / What's the failure mode when a semantic layer is optional rather than enforced?

Q11. Walk me through the schema of your metric and dimension registry.

Ideal answer. Each metric is a typed YAML record: a canonical snake_case `name`, `label`, `description`, a `type` (count / sum / ratio / derived), the `entity` and physical `grain` it resolves against, the aggregation (`agg`) and additive `sql` for count/sum, `numerator` / `denominator` for ratios, an `expr` over other metrics for derived, governed `filters`, a `format`, a `dimensions` whitelist, an `owner`, and a semver `version`. Dimensions are simpler: `name`, `type` (categorical/temporal/boolean), `entity`, the resolving `sql`, and `format`. The whitelist and grain fields are the load-bearing ones â€” they constrain *what can be sliced by what*, which is the difference between a definition store and a true governance layer.

Why Helios demonstrates it. Helios' registry encodes all of this; for example `session_conversion_rate` is `type: ratio`, `grain: fct_funnel`, `numerator: purchasing_sessions`, `denominator: sessions`, with a `dimensions` whitelist of the 14 session dimensions and `version: 2.0.0`. **Follow-ups.** Why carry `entity` and `grain` separately? / What does the `dimensions` whitelist buy you over allowing any dimension? / Why semver each metric individually?

Q12. How does a registry of definitions actually prevent the LLM from hallucinating columns?

Ideal answer. Because the consumer never writes SQL â€” it passes *names*, and a deterministic resolver composes the SQL from the registry. If a caller references a metric or dimension that isn't registered, the lookup raises before any SQL exists; an invented `conversion_pct` can't become a query, it becomes a hard error. Physical column names live exclusively in the registry's `sql` fields owned by analytics engineers, so the model literally has no surface on which to emit a column. This converts "0 hallucinated columns" from a hope into a structural property of the system. **Why Helios demonstrates it.** Helios' `semantic-mcp.build_query(metric, dims, filters, window)` does `REGISTRY.metrics[metric]` (KeyError = hard fail) and rejects any dim not in that metric's whitelist before composing SQL â€” so the LLM passing a bad name is caught at the boundary, then the Critic agent flags it and the model retries with a real metric. **Why this is grounding rule G5.** An unknown name is a hard error, never a fallback to free SQL â€” that's the anti-hallucination guarantee in one rule. **Follow-ups.** What stops the LLM from calling

`warehouse-mcp.run_query` directly to bypass the registry? / How do you validate the registry itself is internally consistent? / Where does reconciliation fit on top of this?

Q13. What is "governed SQL," and how is governance enforced beyond just a YAML file?

Ideal answer. Governed SQL means every query a consumer runs is composed from versioned, owned, tested definitions rather than hand-authored â€” the YAML is the contract and a deterministic resolver is the only author. Enforcement is layered: the resolver rejects unknown names; CI compiles the registry and fails on dangling references or schema drift; and â€” critically â€” *credentials* are scoped so the path can't be bypassed. The deepest enforcement isn't software politeness, it's IAM: the service account that backs the semantic path can only read the governed views, not raw tables. **Why Helios demonstrates it.** Helios separates the `helios_semantic` dataset from `helios_marts`, and `sa-helios-semantic` (backing `semantic-mcp`) is granted read on `helios_semantic` *only* â€” so the LLM physically cannot reach raw or even mart tables to hand-author SQL. CI compiles the registry's referential integrity on every PR. **Follow-ups.** What's the threat model if all servers shared one service account? / How do governed `filters` (row-level governance) work in your registry? / How do you handle a legitimate one-off query the registry can't express?

Q14. What is dbt MetricFlow / the dbt Semantic Layer, and how does your registry map to it?

Ideal answer. MetricFlow is dbt's semantic-layer engine: you define *semantic models* over facts (with entities, dimensions, and measures), then define metrics on top (`simple`, `ratio`, `derived`, `cumulative`), and MetricFlow compiles and queries them so consumers ask for metrics-by-dimensions without writing joins or `GROUP BY`. My registry maps onto it almost 1:1: each `entity / grain` becomes a semantic model over a fact, additive `agg: sum|count_distinct` metrics become MetricFlow measures, `type: ratio` becomes a ratio metric, `type: derived` becomes a derived metric, and dimension `sql` becomes semantic-model dimensions. **Why Helios demonstrates it.** Helios' YAML maps 1:1 to MetricFlow â€” `entity: session / grain: fct_funnel` is a semantic model over `fct_funnel`, `aov` is a ratio metric (`gross_revenue / transactions`), `revenue_per_session` is a derived metric (`SAFE_DIVIDE({revenue},{sessions})`) â€” but I ship a thin custom resolver so it runs identically with or without a dbt Cloud Semantic Layer endpoint; the YAML is the single source of truth either way. **Follow-ups.** Why ship a custom resolver instead of depending on dbt Cloud? / How does MetricFlow prevent fan-out/fan-trap double counting across grains? / What does MetricFlow give you that a wide pre-aggregated table doesn't?

Q15. Why is "single source of truth" so important, and what concretely happens when a definition changes?

Ideal answer. A single source of truth means a metric's formula and the dimension classification logic each live in exactly one place, so consistency is structural rather than maintained by vigilance. When the definition changes, you edit one file, bump the version, and every consumer picks up the new behavior on the next build " and prior outputs remain interpretable because they're tagged with the version that produced them. The anti-pattern is the same CASE statement copy-pasted across five models: a change becomes a five-place edit where one place inevitably gets missed and silently diverges. **Why Helios demonstrates it.** Helios enforces two single sources of truth: every metric/dimension lives only in `semantic_layer.yaml`, and *all* channel-grouping logic lives only in the `channel_group_case()` macro. Definitions carry `owner` + `semver`; a breaking formula change bumps the major version and is recorded in the Memory store so old diagnoses stay interpretable, and the Critic checks a finding's cited metric version against the run's registry version. **Follow-ups.** How do you version a metric without breaking historical dashboards? / Where does channel logic belong " the macro or the registry " and why both? / How do you migrate consumers off a deprecated metric?

Q16. How do you test and validate a semantic layer itself, separate from the data?

Ideal answer. Two distinct kinds of validation. First, *structural* validation at compile time: every `numerator / denominator / expr` token must resolve to a registered metric, and every dimension in a whitelist must be a registered dimension on a compatible entity " dangling references fail the build. Second, *value* validation against reality: reconcile the composed metric's total against a canonical raw total, because a perfectly-consistent registry can still encode a wrong formula. The first proves the layer is coherent; the second proves it's correct. **Why Helios demonstrates it.** Helios compiles `semantic_layer.yaml` in CI with referential-integrity checks (dangling token = hard fail), and on top of that `warehouse-mcp.reconcile` asserts mart/semantic totals agree with raw canonical totals within $\leq 0.5\%$; the `revenue_reconciles` test pins `revenue` to the source to the cent. **Follow-ups.** How do you unit-test a `derived` metric's algebra? / What reconciliation tolerance is acceptable and why not always zero? / How do you catch a metric that's self-consistent but semantically wrong?

Q17. A stakeholder asks for a metric your semantic layer doesn't have. Walk me through adding it.

Ideal answer. I add it as a definition, not as ad hoc SQL: a new YAML record with name, type, grain, the additive `sql` or numerator/denominator/expr, a dimensions whitelist, an owner, and version `1.0.0`. CI then compiles the registry, runs referential-integrity checks, and " if it's a new fact measure " I add a reconciliation test. Code review confirms the grain is declared and tested and no physical column was hardcoded into a mart. Only after that does the metric become callable by name; this keeps the "0 hallucinated, 100% governed" guarantee intact instead of letting one urgent request open a hole. **Why Helios demonstrates it.** Helios' CLAUDE.md rule is

explicit: "add new metrics by editing the registry YAML, not by hand-writing SQL." A new metric flows through the same governance gate (compile, referential integrity, reconciliation, review) as every existing one of the 28. **Follow-ups.** How do you decide the new metric's grain? / What if the metric needs a dimension that doesn't exist yet? / How do you avoid metric sprawl â€" 200 near-duplicate definitions?

Modeling

Q18. Explain the staging / intermediate / marts layering and what belongs in each.

Ideal answer. Staging is a 1:1, lightly-cleaned mirror of each source â€" rename to snake_case, cast types, expose keys, no business logic â€" so downstream models never touch raw shapes. Intermediate holds reusable business logic that more than one mart needs (sessionization, flagging, channel classification) and exists to avoid duplicating that logic into every mart. Marts are the consumption layer: grain-explicit facts and conformed dimensions modeled for the questions analysts actually ask. The discipline is that logic flows in one direction and each layer has a single responsibility â€" staging cleans, intermediate computes shared logic, marts shape for consumption. **Why Helios demonstrates it.** Helios runs `src_ga4` -> `stg_ga4__events` / `stg_ga4__event_params` (typed views, one UNNEST of `event_params` done once) -> `int_ga4__sessionized` / `int_ga4__funnel_steps` (session key, channel group, `reached_*` flags) -> marts (`fct_funnel`, `fct_daily_funnel`, `fct_orders`, `dim_*`). The semantic layer is the fifth, governance layer on top. **Follow-ups.** Why is `event_params` unnested once in staging rather than per-mart? / What's the smell when business logic leaks into staging? / Why have intermediate models at all instead of long mart CTEs?

Q19. What is a star schema, and why prefer it over a fully normalized (snowflake) model for analytics?

Ideal answer. A star schema is central fact tables (the measurable events) surrounded by denormalized dimension tables (the descriptive context), joined on surrogate keys. It's preferred for analytics because the wide, denormalized dimensions minimize joins at query time and are intuitive to slice, which matters far more for read-heavy BI than the storage savings normalization gives you. A snowflake further normalizes dimensions into sub-dimensions, saving space but adding joins and cognitive load â€" rarely worth it on a warehouse where storage is cheap and scans are the cost. So I model a star: conform shared dimensions, keep facts at a clean grain, and denormalize dims. **Why Helios demonstrates it.** Helios' marts are a star: `fct_funnel` / `fct_sessions` (session grain) and `fct_orders` (transaction grain) at the center, with conformed `dim_users`, `dim_items`, `dim_channels`, `dim_date` around them, so the Decompose agent can pivot any metric across `device_category`, `channel_group`, `country` without runtime join gymnastics. **Follow-ups.** When would you accept a snowflaked dimension? /

What makes a dimension "conformed," and why does it matter for the decomposition? / How do slowly-changing dimensions fit the star?

Q20. What is grain, why must you declare it first, and how do you defend it?

Ideal answer. Grain is the precise meaning of one row in a table â€" "one row per X." You declare it before writing any SQL because every column, key, and aggregation depends on it; a model without an agreed grain is a model whose numbers no one can trust. You defend it with a uniqueness test on the grain key: if `transaction_id` is supposed to be unique and isn't, the fact is double-counting and the test fails the build. Mixed or ambiguous grain is the single most common cause of fan-out double counting, so grain is the first thing I write down and the first thing I test. **Why Helios demonstrates it.** Every Helios fact states its grain explicitly â€" `fct_funnel` is one row per session (PK `session_key`), `fct_orders` is one row per `transaction_id`, `fct_order_items` is one row per transaction-x-item line â€" and each grain key carries `unique` + `not_null` tests so a grain violation fails CI. **Follow-ups.** How does wrong grain cause a fan-out, concretely? / Why is `count(distinct)` your friend when grain is uncertain? / How do you reconcile two facts at different grains in one query?

Q21. What are surrogate keys, why use them, and how did you build yours?

Ideal answer. A surrogate key is a single synthetic key that stands in for a natural or composite business key, usually a hash of the columns that define the grain. You use them because they give every row one stable join key, hide composite-key complexity from consumers, and stay stable across runs if generated deterministically. The rule is determinism: hash the *exact* grain columns the same way everywhere so the key is reproducible and idempotent. I avoid keys that aren't stable across runs and I always hash the full natural key, not a subset. **Why Helios demonstrates it.** Helios' canonical session key is `session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))` â€" one deterministic expression used everywhere (never `FARM_FINGERPRINT`, never `COUNT(*)`), so `sessions = COUNT(DISTINCT session_key)` is reproducible run to run. `dim_channels` similarly uses `TO_HEX(MD5(channel_group))` as `channel_key`. **Follow-ups.** Why MD5-hex rather than `FARM_FINGERPRINT` or a row_number? / What happens to the key if one grain column is null? / How does a deterministic surrogate key support `insert_overwrite` idempotency?

Q22. Why build wide (denormalized) fact tables, and what's the tradeoff?

Ideal answer. A wide fact carries the full descriptive dimension set on the fact row itself (the Kimball wide-fact / one-big-table pattern) so the consumption layer can slice any metric by any dimension without runtime joins. The benefit is query simplicity and speed, and on a columnar warehouse the extra columns are nearly free because unselected columns aren't scanned. The

tradeoff is storage and the need to keep denormalized attributes consistent with their dimension source â€” which I manage by deriving those attributes once upstream rather than per-fact. For agent- or BI-driven slicing across many dimensions, wide facts are usually the right call. **Why Helios demonstrates it.** Helios' architectural ruling makes `fct_funnel` / `fct_sessions` wide â€” carrying the full session dimension set (`device_category` , `operating_system` , `browser` , `country` , `region` , `channel_group` , `source` , `medium` , `campaign` , `landing_page` , `is_new_user` , `ga_session_number`) alongside the `reached_*` flags and `session_revenue` â€” so the semantic layer slices any metric by any of those without a runtime join, which the diagnosis tree needs to drill every dimension. **Follow-ups.** How do you keep a wide fact's denormalized attributes from drifting from the dimension? / When is the join cost worth keeping the fact narrow? / How does columnar storage change the wide-vs-narrow calculus?

Q23. Sessionization isn't given in GA4 â€” how did you reconstruct sessions, and what are the pitfalls?

Ideal answer. GA4's BigQuery export is event-grained â€” there is no session row â€” so a session is reconstructed by grouping events that share a session identifier. The identifier lives inside the `event_params` array, so you UNNEST to extract `ga_session_id` , and critically you key on the *composite* (`user_pseudo_id` , `ga_session_id`) because `ga_session_id` is only unique within a user, never globally. The pitfalls are: keying on `ga_session_id` alone (cross-user collisions), forgetting that identity is cookie-grain since `user_id` is almost always null, and deriving session attributes from the wrong scope. I collapse events to one session row in an intermediate model and hash the composite key into a surrogate. **Why Helios demonstrates it.** Helios' `int_ga4_sessionized` collapses events to one row per (`user_pseudo_id` , `ga_session_id`) , hashes it into `session_key` , and derives session-scoped source/medium/channel and `reached_*` flags there â€” so downstream marts never re-implement UNNEST or re-derive the session. **Follow-ups.** Why is (`user_pseudo_id` , `ga_session_id`) the key rather than `ga_session_id` alone? / How do you handle a session that spans midnight / two date shards? / What's the identity limitation when `user_id` is null and how does it bias user-level metrics?

Q24. The GA4 `traffic_source` field is a trap â€” explain it and how your model handles it.

Ideal answer. The event-level `traffic_source` struct in the GA4 export is *user first-touch* attribution â€” stamped from the user's very first acquisition and copied onto every later event â€” not the source of the session that fired the event. Using it for session-level channel analysis systematically over-credits acquisition channels and under-credits re-engagement, and it can manufacture exactly the Simpson's-paradox confounds a diagnosis engine is trying to detect. The fix is to derive session-scoped source/medium from the session's own `event_params` (first non-null value ordered by timestamp) and use `traffic_source` only as a documented fallback when session params are entirely null. **Why Helios demonstrates it.** Helios derives

`session_source / session_medium` in `int_ga4__sessionized` from `event_params` (with `(direct) / (none)` normalization), feeds them into `channel_group_case()`, and only falls back to first-touch `traffic_source` when session scope is null – preventing first-touch leakage from corrupting the channel-level decomposition. **Follow-ups.** How does first-touch leakage create a fake mix-shift? / When is first-touch actually the right attribution (e.g., `dim_users.first_touch_channel`)? / How do you reconcile your derived channel split against GA4's own UI?

Q25. How do you compute a session-level conversion rate so it survives Simpson's paradox when aggregated?

Ideal answer. Materialize monotonic per-session boolean step flags, then compute every rate as `SUM(numerator) / SUM(denominator)` *after grouping* – never as the average of per-segment ratios. Averaging ratios weights segments equally regardless of size, which is precisely how Simpson's paradox produces a topline that moves opposite to every segment. Re-aggregating the raw numerator and denominator preserves the correct volume weighting at any grain. The flags must be max-downstream (reached step X = reached X or any later step) so they're monotonic and step rates can never exceed 1. **Why Helios demonstrates it.**

```
SAFE_DIVIDE(COUNTIF(reached_purchase), COUNT(DISTINCT session_key)) AS session_conversion_rate
```

Helios materializes `reached_*` max-downstream flags in `int_ga4__funnel_steps`, computes rates as SUM/SUM in `fct_daily_funnel`, and `stats-mcp.decompose_change` then splits any topline rate move into mix / rate / interaction – turning the re-aggregation discipline into an explicit, deterministic Simpson's-paradox defense. **Follow-ups.** Show me numerically how averaging ratios flips the topline. / Why must the flags be max-downstream rather than literal step-fired? / How does the mix/rate/interaction split formally dissolve the paradox?

Data Quality

Q26. What is the difference between a test and a monitor in an analytics pipeline, and when do you reach for each?

Ideal answer. A test is a deterministic, blocking assertion that runs at build/deploy time against a known invariant – it either passes or fails the build (e.g. a `unique` test on a primary key). A monitor is a continuous, often statistical observation over production data that *alerts* rather than blocks, tolerating gradual drift you cannot encode as a hard pass/fail (e.g. "row volume is 4 sigma below the trailing 28-day mean"). The rule I use: if I can state the invariant precisely and a violation should stop the pipeline, it is a test; if the signal is anomalous-but-not-illegal and a

human should triage it, it is a monitor. The failure of treating everything as a test is brittle pipelines that page on benign variance; the failure of treating everything as a monitor is silent corruption that ships because nothing blocked it. Mature stacks use both - tests guard contracts, monitors guard behavior. **Why Helios demonstrates it.** Helios encodes hard invariants as dbt tests (`revenue_reconciles` to the cent, funnel monotonicity, `accepted_values` on the 10 channel groups) and treats anomaly detection as a *monitor* - `stats-mcp.detect_anomaly` flags deviations for the Diagnose agent, it does not fail the build. **Follow-ups.** Where does dbt source `freshness` sit on the test-vs-monitor line, and why? / How would you avoid alert fatigue if every metric had a monitor? / Can a monitor's threshold itself be regression-tested?

Q27. Explain reconciliation as a data-quality control. Why is it stronger than schema tests alone?

Ideal answer. Reconciliation asserts that an aggregate computed at a downstream layer equals the same aggregate computed independently from the authoritative source, within a stated tolerance - it is an end-to-end conservation check across the whole transformation chain. Schema tests (`not_null` , `unique` , `accepted_values`) prove a column is well-typed and structurally valid, but a model can pass every schema test while silently dropping 12% of revenue because a join fanned out or a filter was too aggressive. Reconciliation catches exactly that class of *quantitative* drift that structural tests are blind to. The tradeoff is cost: reconciliation re-scans the source, so I scope it to the metrics that carry business and label weight (revenue, session counts) rather than everything. I treat the source total as ground truth and tolerate only a tiny, explicitly-justified delta. **Why Helios demonstrates it.** The `revenue_reconciles` generic test asserts `SUM(revenue)` in `fct_orders` equals `SUM(purchase_revenue_in_usd)` over raw `purchase` events with tolerance 0; at runtime `warehouse-mcp.reconcile` re-checks marts against canonical totals and any drift >0.5% fails the finding before it reaches the Decision Brief. **Follow-ups.** Why tolerance 0 for the build test but 0.5% at runtime? / How do you reconcile a *rate* rather than a sum? / What do you do when reconciliation fails - block, quarantine, or degrade?

Q28. Funnel monotonicity: what is the invariant and how do you test it?

Ideal answer. A session that reached `add_to_cart` necessarily reached `view_item` , so the funnel must satisfy `sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ ... ≥ reached_purchase` - counts are non-increasing down the funnel and every step rate is therefore ≤ 1 . I enforce this by computing the `reached_*` flags as *max-downstream* (a flag is true if the session reached that step or any later step), which makes monotonicity structural rather than hoped-for, then I add a singular test that fails if any adjacent pair violates the ordering:

```
-- tests/assert_funnel_monotonic.sql (fails ⇒ returns rows)
select event_date
from {{ ref('fct_daily_funnel') }}
```



```

where view_item_sessions > sessions
or add_to_cart_sessions > view_item_sessions
or begin_checkout_sessions > add_to_cart_sessions
or purchasing_sessions > add_payment_info_sessions

```

Without this, a sessionization bug or a non-monotonic flag produces step rates above 1, which silently poisons the mix/rate decomposition. **Why Helios demonstrates it.** Helios defines funnel flags as `reached_*` max-downstream in `int_ga4__funnel_steps` (the retired `did_*` naming was non-monotonic) precisely so the decomposition's segment rates `r_i` are always valid probabilities; the monotonicity test is a keystone because it fails silently otherwise. **Follow-ups.** Why is making the property structural better than only testing for it? / How would a late-arriving `purchase` event break monotonicity, and does your incremental strategy handle it? / Could you express this as a dbt relationship/expression test instead of a singular test?

Q29. How do you implement and reason about data freshness SLAs?

Ideal answer. Freshness is a contract on *latency*: the newest data must reach the consumption layer within a bounded window, with a warn threshold and a hard error threshold. I declare it on the source with a `loaded_at_field` and `warn_after` / `error_after` so dbt computes the gap between the latest loaded record and now, and I make a freshness *error* block the downstream consumer rather than letting it analyze stale data. The reasoning is that a diagnosis run on yesterday's incomplete shard is worse than no diagnosis - it produces a confident, wrong "why". The tradeoff is choosing thresholds tight enough to catch a broken pipeline but loose enough to absorb normal late-landing; for GA4 I use 36h warn / 48h error because GA4 events land late.

Why Helios demonstrates it. Helios declares source freshness on `src_ga4` (`warn_after: 36h`, `error_after: 48h`); a freshness error blocks the agent pipeline so the autonomous run never diagnoses a stale or partial day - protecting the "<5 min, correct" promise rather than just speed.

Follow-ups. The public GA4 dataset is static - so why bother with freshness at all? / How does freshness interact with your 3-day incremental lookback? / What's the difference between freshness and completeness, and which is harder to guarantee?

Q30. What is the "silent failure" problem in data pipelines, and how do you design against it?

Ideal answer. Silent failure is when a transformation produces output that is structurally valid and looks plausible but is quantitatively wrong - no error is raised, the build is green, and a human acts on a corrupted number for weeks. It is the most dangerous failure mode in analytics because the cost is unbounded and the detection is accidental. I design against it three ways: make invariants *structural* where possible (so the bad state cannot be represented), add *conservation* tests (reconciliation) that catch quantitative drift schema tests miss, and put the highest-leverage transforms under *golden tests* with hand-worked expected outputs. The mental model: the more downstream consumers trust a number blindly, the louder its upstream transform must fail when

wrong. **Why Helios demonstrates it.** Helios explicitly tags five keystones that "fail silently" - sessionization, the semantic registry, `decompose_change`, revenue reconciliation, and `fct_daily_funnel` carrying revenue - and gives each a golden or reconciliation test plus the CI eval gate, so a silent corruption surfaces as a red build, not a wrong brief. **Follow-ups.** Give an example of a bug that passes every schema test but is still wrong. / How does the offline eval benchmark act as a silent-failure detector for the agents? / Why are keystone transforms more likely to fail silently than leaf models?

Q31. When and how do you use `accepted_values` tests, and what are their limits?

Ideal answer. `accepted_values` asserts a column's domain is exactly a known, closed set - it is the right test for categorical dimensions whose cardinality is fixed by a business definition, because it catches both new unexpected values (a logic gap) and a typo'd mapping. I use it where the set is *canonical and closed*, not merely *currently observed*; the anti-pattern is pinning a high-cardinality or growing field, which turns the test into a maintenance treadmill that fails on every legitimate new value. Its limit is that it validates membership, not distribution - a column can be 100% one accepted value when it should be spread across ten, and `accepted_values` passes happily. So I pair it with a distribution monitor when the spread itself matters.

```
- name: channel_group
  tests:
    - accepted_values:
      values: ['Direct', 'Organic Search', 'Paid Search', 'Display', 'Paid Social',
              'Organic Social', 'Email', 'Affiliates', 'Referral', 'Other']
```

Why Helios demonstrates it. Helios pins `dim_channels.channel_group` to exactly the 10 GA4 default channel groups via `accepted_values` - this is the test that fails loudly if the `channel_group_case()` macro ever emits an 11th group or a "Paid Other" leak, keeping the single source of truth honest. **Follow-ups.** Why is `accepted_values` on `channel_group` better placed on the dim than on every fact? / How would you catch the "everything collapsed to one value" failure that `accepted_values` misses? / Would you ever generate the accepted set from a seed instead of hardcoding it?

Q32. How do you decide what tolerance to set on a reconciliation or range test?

Ideal answer. Tolerance should be derived from a known, explainable source of legitimate difference, never tuned to "make the test pass". I start at zero and only widen it for a documented reason: floating-point summation order, a deliberate rounding boundary, or a known late-arrival window. If I cannot name why a delta exists, the delta is a bug, not a tolerance. The danger of a loose tolerance is that it becomes a hiding place - a 5% band quietly absorbs a real 4% revenue leak. So I keep build-time exactness strict (the same data through two paths should match exactly)

and reserve looser runtime tolerances for genuinely noisy cross-system comparisons, with the tolerance value itself reviewed in code review. **Why Helios demonstrates it.** Helios uses tolerance 0 on the build-time `revenue_reconciles` test (same warehouse, two query paths must match to the cent) but a 0.5% drift ceiling at runtime via `warehouse-mcp.reconcile`, because runtime compares governed marts against canonical totals where rounding and edge filters can introduce a tiny, bounded difference. **Follow-ups.** Walk me through justifying a non-zero tolerance to a skeptical reviewer. / How do absolute vs relative tolerances behave differently at low volumes? / Could you make tolerance scale with sample size?

Q33. How do you test data quality on a column that is mostly null, like GA4's `user_id` ?

Ideal answer. First, I stop treating high-nullity as a defect to be tested away and instead model around it honestly - I document the expected null rate and pick a different grain. Testing `user_id is not null` would fail 99% of rows and tell me nothing useful. Instead I add a *bounded* data test: assert the null rate stays within an expected band (so a sudden drop to 0% non-null is caught as an instrumentation break), and route all identity logic through the field that is populated. The senior move is to make the limitation explicit in the model contract and metric definitions so consumers know the metric is an approximation, not to paper over it with a green test. **Why Helios demonstrates it.** In Helios `user_id` is almost always null, so identity is cookie-grain on `user_pseudo_id` and user-level metrics (`users` , ARPU) are documented as cookie-based approximations - the data model treats nullity as a stated assumption, not a test failure to suppress. **Follow-ups.** How would you alert on `user_id coverage` changing rather than its value? / What's the downstream metric distortion from cookie-grain identity, and how do you bound it? / Would you use a `not_null` test with a configured threshold (e.g. `dbt_utils`)?

Q34. How do you prevent data-quality issues in the source from being misdiagnosed as a business anomaly?

Ideal answer. A pipeline that cannot distinguish "the funnel moved" from "the data about the funnel changed" will confidently explain artifacts, which is worse than missing real signal. I build a data-quality gate *upstream* of diagnosis - freshness, completeness, and reconciliation must pass before any anomaly is interpreted - and I add a data-quality hypothesis to the root-cause search itself, so a tracking outage, a deployment-shaped step in a single channel, or a schema change is considered as a candidate explanation and can be ruled out. The principle is verify-then-trust: prove the data is sound before pricing a movement in dollars. **Why Helios demonstrates it.** Helios's Critic agent runs an explicit *data-quality* refutation (alongside mix-confound, insufficient-sample, and seasonality) and can DOWNGRADE or DROP a finding it judges an artifact; the eval benchmark even includes a dedicated data-quality-artifact bucket so the pipeline is graded on correctly *declining* to diagnose noise. **Follow-ups.** What does a tracking-outage anomaly look like in a decomposition, and how is it distinguishable from a real drop? / How do you keep the data-

quality check from suppressing real anomalies? / Where does this gate live relative to the dry-run cost gate?

Testing

Q35. Differentiate schema tests, data tests, unit tests, and golden tests. When is each the right tool?

Ideal answer. Schema tests assert structural contracts on columns (`unique` , `not_null` , `relationships` , `accepted_values`) - cheap, run on every build, catch type/key violations. Data tests assert *values are sane* over real data (ranges, freshness, row-count thresholds) - they catch plausible-but-wrong content schema tests pass through. Unit tests assert that a transform maps a *seeded synthetic input* to an *asserted output* in isolation, independent of production data - they pin business logic like sessionization. Golden tests are a unit test whose expected output is a hand-worked, authoritative reference value for a high-stakes computation. The rule: schema and data tests guard the data flowing through; unit and golden tests guard the *logic* itself, which is the only way to test a transform before any data exists. **Why Helios demonstrates it.** Helios's testing pyramid uses all four - `accepted_values` / `relationships` (schema), revenue-range and conversion-rate-bounds (data), seeded `given` / `expect` unit tests on the sessionizer and channel logic, and golden tests with hand-worked mix/rate/interaction values for `stats-mcp.decompose_change`. **Follow-ups.** Why can't a unit test replace a reconciliation test? / Which of these can run with zero warehouse cost, and why does that matter for CI? / Where do golden tests fit relative to property-based tests?

Q36. How do golden tests work for the decomposition, and why are they essential there specifically?

Ideal answer. A golden test fixes a small, fully-specified input and an expected output that I computed by hand (or analytically) and verified independently, then asserts the implementation reproduces it exactly within numerical tolerance. They are essential for the decomposition because mix/rate/interaction is the analytical centerpiece - a sign error or a wrong base period (`t0` vs `t1` weights) is *plausible* and would silently flip a diagnosis from "real behavior change" to "composition artifact". I construct goldens that exercise the adversarial cases, not just the happy path: a pure mix shift (segment rates held constant, only weights move), a pure rate change, and a Simpson's-paradox case where the aggregate falls while every segment rises. If the implementation passes all three with the additive identity `dR = mix + rate + interaction` holding, the math is trustworthy.

```
# pure mix shift: rates fixed, weights move ⇒ rate_effect must be ~0
# Simpson case: aggregate down, every segment up ⇒ mix_effect dominates negative
```

```
assert abs((mix + rate + interaction) - delta_R) < 1e-9
assert abs(rate_effect) < 1e-9           # pure-mix scenario
```

Why Helios demonstrates it. `stats-mcp.decompose_change` is a named keystone unit-tested against hand-worked golden values, and the eval injector exploits the same separation (rate-perturbation changes only `r_i`, volume-perturbation changes only `w_i`) so the golden tests and the benchmark validate the same identity from two directions. **Follow-ups.** Why use `t0` weights for the rate effect rather than an average of `t0` / `t1` ? / How do you pick the numerical tolerance for a float golden? / What does a failing Simpson's-paradox golden tell you about the bug?

Q37. What does TDD look like applied to SQL transformations?

Ideal answer. TDD on SQL means writing the assertion before the model: I declare the grain and the contract, write the failing test (a unit test with seeded rows and expected output, or a schema/relationship test), then write the SQL until the test goes green. It works especially well for transforms with a crisp definition - sessionization, funnel flags, channel grouping - because the test is the spec and forces me to pin edge cases (a session spanning midnight, a null medium) up front rather than discovering them in production. The tradeoff is that not all SQL is amenable to cheap unit testing - large aggregations are better guarded by reconciliation - so I apply TDD where the logic is subtle and the input can be small and synthetic. The discipline pays off most on the transforms that fail silently. **Why Helios demonstrates it.** Helios's operating rule is "TDD on SQL: write the dbt/golden test first, then make it pass," applied specifically to the keystone transforms (sessionization, `reached_*` flags) whose correctness every downstream number depends on. **Follow-ups.** What's a sessionization edge case you'd write a failing test for first? / How do dbt `unit_tests` (`given` / `expect`) let you TDD without touching the warehouse? / When is TDD on SQL not worth it?

Q38. Explain the "keystone test budget" - how do you decide where to invest scarce testing effort?

Ideal answer. Tests are not free - they cost authoring time, build time, and warehouse bytes - so I spend the budget where the product of *blast radius* and *silent-failure probability* is highest, not uniformly. A keystone is a transform that (a) feeds many downstream consumers and (b) can be wrong while looking right; those get the heavy artillery - golden tests, reconciliation, unit tests. Leaf models that are visibly broken when wrong, or that one report consumes, get light schema tests. This is risk-weighted testing: I'd rather have five bulletproof tests on the sessionizer and the decomposition than two hundred shallow `not_null` tests spread evenly. The anti-pattern is a high test *count* that gives false confidence while the load-bearing logic is untested. **Why Helios demonstrates it.** Helios names exactly five keystones (sessionization, the semantic registry, `decompose_change`, revenue reconciliation, `fct_daily_funnel` carrying revenue) and concentrates golden/reconciliation tests there - "get these right, test them hard, they fail silently."

Follow-ups. How do you identify a keystone from the DAG topology? / Doesn't concentrating tests leave leaf models under-covered - is that acceptable? / How would you measure whether your test budget is well-allocated?

Q39. How do you gate CI on analytics changes without making the pipeline slow or expensive?

Ideal answer. I make CI fast and cheap by building only what changed and its descendants, deferring everything else to a production manifest, and capping bytes. In dbt that's slim CI:

`state:modified+` selects modified models plus children, `--defer --state` resolves unchanged refs against the prod manifest so I don't rebuild the world, and tests run on the same selection. I add a byte guard so a runaway query is killed by the warehouse, not discovered on the bill. The principle is that CI must be cheap enough to run on *every* PR or it gets bypassed - a gate people route around is not a gate. The tradeoff is manifest staleness, which I manage by refreshing the prod manifest on merge.

```
- run: dbt build --select state:modified+ --defer --state ./prod-manifest
- run: dbt test --select state:modified+
- run: python eval/run_benchmark.py # accuracy gate
```

Why Helios demonstrates it. Helios runs slim CI (`state:modified+ --defer`) plus the eval harness on every PR, and bounds cost by dry-running every benchmark scenario first and aborting if total scanned bytes exceed the per-run budget - a 12-scenario smoke subset on every push, the full set on PRs to `main` . **Follow-ups.** What goes stale in the deferred prod manifest and how do you keep it fresh? / Why run a smoke subset on push but the full suite only on PR? / How do you keep CI deterministic when the agents involve an LLM?

Q40. In what sense is the eval harness an integration test, and how does it differ from your unit tests?

Ideal answer. Unit tests prove a component is correct in isolation; an integration test proves the assembled system produces the right end-to-end outcome through all its seams. The eval harness is exactly that - it exercises the full path (semantic SQL -> warehouse -> stats -> agents -> finding) against inputs where I know the right answer, and grades the *system-level* output, not any one function. It catches failures unit tests cannot: a correct `decompose_change` wired to the wrong dimension, a registry change that subtly shifts a metric, an agent that drills the mix effect instead of the rate effect. The difference is the assertion target - a unit test asserts `f(x)=y` ; the eval asserts "given a hidden anomaly, the system rediscovers the segment I injected". **Why Helios demonstrates it.** Helios's offline labeled benchmark injects known anomalies into a frozen GA4 copy and grades whether the pipeline rediscovers the root-cause segment (target $\geq 85\%$ vs $\leq 45\%$ naive); that is an integration test of the whole diagnosis chain, distinct from the golden unit test on `decompose_change` alone. **Follow-ups.** What integration bug would the eval catch

that all your unit tests would miss? / How do you keep an integration test that involves an LLM stable enough to gate on? / Why inject anomalies rather than test against historical labeled incidents?

Q41. How do you test non-deterministic, LLM-in-the-loop components so CI remains a reliable gate?

Ideal answer. I push determinism to the boundaries and grade on outcomes, not on exact token output. Concretely: all math and SQL are deterministic tools (seeded), so the only stochastic element is the LLM's *choice* of which tool calls to compose - and I grade whether the final finding is correct (right segment, right mix-vs-rate verdict), which is robust to wording variation. I run a labeled benchmark with a tolerance band rather than asserting a single byte-identical answer, set temperature low for the graded run, and gate on aggregate accuracy with a regression tolerance instead of demanding 100%. The mental model: make the *verifiable* parts deterministic and the *judgment* parts measurable, so non-determinism never reaches the assertion. **Why Helios demonstrates it.** In Helios the LLM never authors SQL or computes a statistic (those are deterministic via `semantic-mcp / stats-mcp`), so CI gates on benchmark accuracy with a 2-point regression tolerance against a committed `main` baseline rather than on exact agent text - non-determinism is contained, not gated on. **Follow-ups.** Why a 2-point regression band rather than a hard floor? / How do you debug a benchmark regression when the cause is the LLM's tool-call choice? / Could a flaky single scenario make CI unreliable, and how do you handle it?

Q42. What is the right regression-gating policy for an accuracy metric in CI?

Ideal answer. A regression gate should fail a change that makes the system meaningfully worse than the last known-good state, comparing against a *committed baseline* rather than an absolute number alone, because absolute targets drift and noise causes false fails. I gate on two things: an absolute floor (don't ship below the minimum bar) and a relative tolerance (don't regress more than N points versus the committed baseline), and I store the baseline in the repo so the comparison is auditable and updated deliberately on merge. The tolerance absorbs benign run-to-run noise; setting it too tight produces flaky red builds, too loose lets slow erosion through. Critically, I also gate the *non-negotiables* at zero tolerance - a hallucinated column is never an acceptable regression. **Why Helios demonstrates it.** Helios stores thresholds in `eval/gates.yaml` with `regression_tolerance: 0.02` against `eval/baselines/main.json`, failing the PR if top-1 drops >2 points or if any hallucination appears - a relative gate for accuracy, a zero-tolerance gate for governance. **Follow-ups.** When and how do you update the committed baseline? / Why is hallucination gated at zero but accuracy at a tolerance? / How would you prevent a series of within-tolerance regressions from silently eroding accuracy over many PRs?

Lineage

Q43. What is a data lineage DAG and what concrete problems does it let you solve?

Ideal answer. A lineage DAG is the directed acyclic graph of dependencies among data assets - sources, models, and consumers - where edges encode "is built from". It solves several concrete problems: build ordering (a topological sort tells you what to run before what), impact analysis (what breaks if I change X), root-causing a bad number (trace upstream to the first wrong model), and onboarding (the DAG is the map of the system). dbt derives this graph automatically from `ref()` and `source()` calls, so lineage is a byproduct of writing models correctly rather than a separate diagram that rots. The leverage is that a question like "what feeds this metric?" becomes a graph query, not an archaeology project. **Why Helios demonstrates it.** Helios's strict five-layer DAG (`src_ga4` -> staging -> intermediate -> marts -> semantic) gives model- and column-level lineage; the build order in the dependency map is a valid topological sort of exactly this graph, which is why no artifact is built before its dependencies. **Follow-ups.** Column-level vs model-level lineage - when do you need the finer grain? / How does `ref()` / `source()` make lineage a byproduct instead of separate documentation? / How would you detect an accidental cycle?

Q44. What are dbt exposures and why are they valuable?

Ideal answer. An exposure is a declared node representing a *downstream consumer* of the data - a dashboard, an application, an ML job, a report - that you register in the dbt project so it becomes part of the lineage graph even though dbt doesn't build it. Their value is that they close the loop on impact analysis: without exposures, lineage ends at the last model and you can't answer "if I change this mart, which business-facing surfaces are affected?". With them, you can select the full subgraph feeding a consumer, run only what a given deliverable needs, and communicate ownership and SLA. They turn the DAG from an internal build graph into a contract with the things people actually look at. **Why Helios demonstrates it.** Helios declares its seven agents and the Decision Brief as exposures in `exposures.yml`, so `dbt ls --select +exposure:helios_decision_brief` answers "what feeds the brief?" and a change can be impact-analyzed all the way to the consuming agent before it ships. **Follow-ups.** What metadata should an exposure carry to be useful (owner, maturity, URL)? / How do exposures change your CI selection logic? / Can an exposure depend on the semantic layer rather than a raw mart, and why prefer that?

Q45. How do you perform impact analysis before shipping a model change?

Ideal answer. Impact analysis answers "what does this change touch?" before it touches it. I use the lineage graph to select the changed model plus its full downstream closure, including exposures, and build/test exactly that subgraph - so I see every affected mart, metric, and consumer rather than guessing. The pattern is `+changed_model+` to get upstream context and all descendants, intersected with `exposure:*` to surface business-facing impact. This converts a

risky "I think this only affects X" into a verified blast radius, and it lets reviewers reason about scope. The discipline matters most for shared upstream models like sessionization, where a one-line change can ripple to every fact and every agent.

```
dbt ls      --select +int_ga4__sessionized+      # full blast radius
dbt build --select int_ga4__sessionized+ exposure:* # build descendants + consumers
```

Why Helios demonstrates it. Helios uses `dbt build --select +<changed_model>+exposure:*` so a change to a keystone like `int_ga4__sessionized` shows everything it touches - every mart and every agent exposure - before merge, backed by the CI eval gate as the final blast-radius check. **Follow-ups.** Why include exposures in the selection rather than just models? / How does impact analysis interact with slim CI's `state:modified+` ? / What change would have the largest blast radius in your DAG, and how would you de-risk it?

Q46. How does lineage support data governance and access control?

Ideal answer. Governance is about controlling *what* logic is authoritative and *who* can reach which data, and lineage is the backbone of both. Lineage lets you prove provenance (this number derives from these governed models, not an ad-hoc query), enforce that sensitive sources only flow through approved transforms, and scope credentials by layer so a consumer physically cannot bypass governance to hit raw data. The strongest control is structural: grant the consuming service read access only to the governed layer, so even a compromised or misbehaving consumer cannot reach raw facts. Lineage makes that boundary auditable - you can see, and test, that nothing reaches a consumer except through the governed path. **Why Helios demonstrates it.** Helios backs `semantic-mcp` with a read-only service account scoped to the `helios_semantic` dataset only, so the LLM *physically cannot* query raw or mart tables - the governance boundary is enforced by IAM along the lineage, not just by policy. **Follow-ups.** Why enforce the boundary with IAM rather than just instructing the agent not to query raw data? / How would you audit that no consumer bypasses the governed layer? / Where do column-level access controls (PII) fit into this?

Q47. How do you version and evolve data models and metric definitions without breaking consumers?

Ideal answer. I treat models and metric definitions as a versioned contract: changes go through PRs with impact analysis, breaking changes are flagged explicitly, and consumers reference *named, governed* metrics rather than raw SQL so a definition can evolve in one place. For genuinely breaking changes I use additive evolution where possible (introduce the new definition alongside the old, migrate consumers, then retire) and dbt's model contracts/versions to make a schema a promise CI enforces. The principle is that the cost of a change should be borne at

author time through review and tests, not at consume time through a silent break. Version control plus the DAG means every definition has history and every change has a reviewable blast radius.

Why Helios demonstrates it. Helios centralizes every metric in `semantic_layer.yaml` and channel logic in one macro, so a definition changes in exactly one governed place and CI's eval gate catches any consumer-visible regression; the retirement of the non-monotonic `did_*` flags in favor of `reached_*` is a documented, repo-tracked breaking-change migration. **Follow-ups.** How do dbt model contracts turn a schema into an enforced promise? / Walk through deprecating a metric with live consumers. / How does the eval baseline detect an unintended semantic drift from a "safe" definition change?

Q48. Explain "the registry as a contract." What does that buy you?

Ideal answer. A metric/dimension registry-as-contract means there is exactly one machine-readable definition of every metric and dimension, referentially compiled, that all SQL must be composed from - so the registry is not documentation, it is the enforced interface between intent and execution. It buys three things: a single source of truth (a metric is defined once, so two reports cannot disagree on what "conversion rate" means), structural elimination of invented columns (only registered names compile; an unknown name is a hard error, not free-text SQL), and safe evolution (change the contract, and every consumer follows). The tradeoff is upfront rigor - you must register a metric before you can query it - which is exactly the friction that prevents the sprawl of ad-hoc, subtly-divergent definitions. **Why Helios demonstrates it.** Helios's `semantic_layer.yaml` (47 metrics + 19 dimensions, referential-integrity compiled, 1:1 with dbt MetricFlow) is the sole SQL author via `semantic-mcp.build_query`; an unrecognized name is a hard error, which is how Helios achieves 0 hallucinated columns structurally rather than by hoping the LLM behaves. **Follow-ups.** How is registry-as-contract different from just having a metrics layer? / Why does "unknown name is a hard error" matter more than a warning? / How does the registry map to dbt MetricFlow, and why align them 1:1?

Q49. How is the CI eval gate a "regression firewall," and what would happen without it?

Ideal answer. A regression firewall is an automated barrier that prevents a known-good capability from silently degrading - it makes correctness a *merge condition*, not an aspiration. The eval gate runs the labeled benchmark on every PR and blocks the merge if accuracy regresses beyond tolerance or any governance invariant breaks, so a refactor that "looks fine" but quietly drops root-cause accuracy can never reach `main`. Without it, accuracy erodes the way untested behavior always does: each change is individually plausible, no one re-runs the full benchmark by hand, and three months later the system is meaningfully worse with no single commit to blame. The firewall converts that slow, invisible decay into a loud, attributable red build on the exact PR that caused it. **Why Helios demonstrates it.** Helios's CI eval gate (`.github/workflows/ci.yaml` + `eval/gates.yaml`) is explicitly the regression firewall: once green, no change may drop top-1

accuracy >2 points or introduce any hallucination - the 85%-vs-45% result is defended automatically on every commit, not just claimed once. **Follow-ups.** Why is "every PR" essential - what breaks if it runs only nightly? / How do you keep the firewall itself trustworthy (i.e. test the tests)? / What's the failure mode if the baseline is updated carelessly?

Q50. Tie it together: how do data quality, testing, and lineage combine into a single trust story for an autonomous system?

Ideal answer. For an autonomous system that acts without a human in the loop, trust cannot be asserted - it has to be engineered into the pipeline as enforceable structure. The three layers compose: lineage defines and constrains the *paths* data can take (governed layer only, auditable provenance, impact-analyzable change); testing proves the *logic* on those paths is correct (golden tests on the centerpiece math, reconciliation against source, monotonicity invariants); and data-quality controls guard the *inputs and outputs* at runtime (freshness, reconciliation drift, the data-quality refutation). Each layer catches a different failure class, and the CI eval gate ties them together by proving the assembled system still produces the right end-to-end answer. The thesis is that the hard part of an AI analyst is not generating insight - it is making insight *correct and trusted*, and that is an analytics-engineering problem before it is a modeling one. **Why Helios demonstrates it.** Helios's three defensibility pillars are exactly this stack - governed-SQL grounding (lineage + registry), deterministic statistics (golden-tested tools), and adversarial verification plus the offline eval (the Critic + the regression-firewall benchmark) - which is why Helios is positioned as an engineered trust system, not a clever prompt. **Follow-ups.** If you could keep only one of the three layers, which and why? / How would this trust stack change if Helios ran on live, multi-tenant data? / Where is the weakest link in this story today, and how would you harden it?

Section 7: AI Engineering Interview

This section covers the AI-engineering substrate of Helios: the Model Context Protocol that turns trust principles into wiring, the tool-calling contract that makes governed SQL and deterministic math enforceable rather than aspirational, and the multi-agent orchestration that runs the diagnosis loop on a deterministic state machine. The throughline is a single thesis "the hard part of an AI analyst is not generating insights, it is making them correct and trusted" and every answer ties an industry-general concept back to a specific Helios server, agent, or guardrail.

MCP

Q1. What is the Model Context Protocol, and what problem does it actually solve?

Ideal answer. MCP is an open client-server protocol that lets an LLM-driven agent invoke external capabilities ("tools"), read external state ("resources"), and reuse parameterized instruction templates ("prompts") over a single schema-typed JSON-RPC 2.0 interface. A server advertises a manifest of tools, each with a JSON Schema for its inputs and outputs; the client (the model runtime) marshals the model's structured tool calls to the server and returns typed results into context. The problem it solves is integration sprawl and trust: instead of every agent hand-rolling bespoke glue to each system, you get a uniform, introspectable, versionable capability surface "and because the surface is the *only* way the model touches the outside world, you can make that surface a hard boundary. The win is not "the model can call functions"; the win is that capabilities become declarative, discoverable, and governable. **Why Helios demonstrates it.** Helios exposes exactly five MCP servers (warehouse, semantic, stats, experiment, report) and the Claude Agent SDK acts as the MCP client, loading them from `mcp_servers.yaml`, fetching each manifest, and exposing the union of tools to the agents "so every database access and every number is a typed tool call, never free text. **Follow-ups.** How does a JSON-RPC tool manifest differ from an OpenAPI spec for the same backend? / What does MCP give you that a hand-written tool registry inside one process does not? / How do resources and prompts differ from tools, and did you use them in Helios?

Q2. Why adopt a tool *protocol* at all instead of just importing Python functions the agent can call?

Ideal answer. A protocol buys you process isolation, a stable typed contract, and independent deployment/versioning that in-process function imports cannot. If tools are just Python functions in the agent's address space, then the agent process holds every credential and every capability those functions need "a buggy or jailbroken agent can reach anything importable. A protocol lets each capability live behind a boundary with exactly its own credentials and blast radius, lets you swap a server's implementation without touching the agent, and gives you a single choke point to log, rate-limit, and validate every call. The cost is serialization overhead and an extra hop, which is real but small relative to LLM latency, and well worth it when the capability is "execute SQL against a billed warehouse." **Why Helios demonstrates it.** Only `warehouse-mcp` holds a BigQuery client and the service-account credential; the agent process and the four stdio servers inherit no warehouse credentials, so a compromised Narrator literally cannot reach BigQuery "a guarantee in-process imports cannot make. **Follow-ups.** Where is the protocol overhead actually paid, and is it ever the bottleneck in a <5-min run? / How would you version a breaking change to `build_query`'s schema without a flag-day? / When would in-process tools genuinely be the right call?

Q3. stdio versus HTTP transport "how did you decide per server, and what are the tradeoffs?

Ideal answer. stdio runs the server as a child subprocess with JSON-RPC framed over stdin/stdout: lowest latency, no network surface, trivially co-located, but single-host and lifecycle-coupled to the parent. Streamable HTTP runs the server as a network service: it can live inside a network/security boundary, hold shared credentials, be reached by multiple clients, and scale independently “ at the cost of a network surface that must be authenticated and a deployment to operate. The rule I apply: pure-compute, data-free, no-credential servers go on stdio; anything holding a sensitive credential or shared state that benefits from a security perimeter goes on HTTP. Transport is a deployment decision, not a capability one “ the tool contract is identical either way. **Why Helios demonstrates it.** `semantic`, `stats`, `experiment`, and `report` run on stdio (pure compute, no warehouse credential), while `warehouse-mcp` runs over streamable HTTP precisely so it can sit in a network boundary with the BigQuery service account, be guarded by a bearer token (`HELIOS_WH_TOKEN`), and be shared across runs. **Follow-ups.** What would force you to move `stats-mcp` off stdio to HTTP? / How do you authenticate the HTTP transport, and what's the threat model? / How does transport choice interact with horizontal scaling of runs?

Q4. Why split capability across *five* servers instead of one big tool server?

Ideal answer. The split follows the trust boundaries, not arbitrary modularity. The single most important separation is SQL *generation* (semantic) from SQL *execution* (warehouse), so a query is validated and reconciled before a byte is scanned and so the model cannot smuggle hand-written SQL into the executor “ they are different servers with different privileges. Math (stats) is separated from data fetch so statistical methods are version-pinned and unit-tested independently of warehouse state. Experiment design and narration are isolated because they have zero data-access needs “ they operate only on findings already produced. Each boundary is also a least-privilege and a blast-radius boundary: a bug or compromise is bounded by that server's tool catalog. The cost is more processes to operate; the payoff is that the trust story is *structural*. **Why Helios demonstrates it.** The five-server split lets the guardrail chain be physically real: `semantic-mcp` emits SQL strings but cannot execute them, `report-mcp` writes briefs but cannot query the warehouse, and only `warehouse-mcp` touches BigQuery “ so “the model can't hand-author SQL” is enforced by which server holds which capability, not by a prompt. **Follow-ups.** Where would you draw the line if you had to collapse to three servers? / Doesn't five servers multiply operational overhead “ how do you justify it for a portfolio-scale system? / Could one server's compromise still cascade, and how do you contain it?

Q5. Explain least privilege in this system “ concretely, what can each server *not* do?

Ideal answer. Least privilege means every component holds exactly the credentials and scope it needs and nothing more, so the worst-case damage of a bug or compromise is bounded. In practice it shows up at two layers: per-server credential scoping and per-agent tool allow-lists. At the server layer, only the warehouse server has a BigQuery client, scoped to

`bigquery.dataViewer` + `bigquery.jobUser`, read-only on the public dataset â€” no write, no delete, no other project. At the agent layer, each agent sees only the tools its role requires, so capability is sliced twice. The principle converts "please behave" into "you have no tool with which to misbehave." **Why Helios demonstrates it.** The per-agent allow-list is explicit: the Narrator is granted only `render_brief`, `save_diagnosis`, `export`, and `get_metric` â€” it has no `run_query` and no path to the warehouse â€” while the Decompose agent gets `build_query`, `run_query`, and `decompose_change` and nothing else, so even a misaligned agent's reach is bounded by its catalog. **Follow-ups.** Why scope at the agent layer when the server layer already isolates credentials? / What's the minimum privilege the Critic needs, and why does it differ from Diagnose? / How would you audit that the allow-lists match the actual call graph in production?

Q6. How is MCP different from "raw" function/tool calling, and when does that difference matter?

Ideal answer. Raw function calling is the model-side mechanic â€” the model emits a structured call against a schema you hand it inline, and your application code dispatches it. MCP is the system-side mechanic â€” a standardized way to *host, discover, isolate, and govern* the things being called across process and trust boundaries. They are complementary layers, not competitors: under the hood, an MCP client still surfaces tools to the model via the same structured tool-calling interface. The difference matters the moment tools need different credentials, independent deployment, cross-agent reuse, or a hard enforcement boundary â€” exactly when "a function in my process" stops being safe enough. For a single-agent script with no privilege separation, raw tool calling is fine and MCP is overhead. **Why Helios demonstrates it.** Helios needs the difference: the same tools must be reused by seven agents with *different* allow-lists, the warehouse credential must be isolated from the agent process, and the SQL-generation/execution boundary must be enforced â€” none of which falls out of raw in-process function calling, all of which MCP makes structural. **Follow-ups.** If the SDK ultimately surfaces MCP tools as ordinary tool calls to the model, where exactly is the enforcement happening? / When is MCP overkill? / How does tool discovery via manifest help when the toolset evolves?

Q7. How does MCP make the grounding boundary *enforceable* rather than a prompt-time suggestion?

Ideal answer. A prompt that says "never write raw SQL" is advisory â€” a model can ignore it, and you only catch the violation after the fact, if at all. MCP makes it physical: you simply do not give the model a tool that executes arbitrary SQL. The only SQL-producing tool, `build_query`, interpolates *only* registry-defined metric templates and dimension expressions, so a hallucinated column like `event_params.foo` cannot survive composition â€” there is no template that references it, and unknown names raise a hard error. The model's reachable action space is the union of its allowed tools, so "can't" is a property of the wiring, not the wording. That is the

difference between a policy and an invariant. **Why Helios demonstrates it.** `semantic-mcp` is the only path to SQL and `stats-mcp` is the only path to math; combined with `run_query` refusing any SQL not produced through that chain, Helios reaches its "0 hallucinated columns / 100% governed SQL" target by construction rather than by convention. **Follow-ups.** Could the model still hallucinate a *metric name* and what happens when it does? / Where would a determined model find a gap in this boundary? / How do you prove to an auditor that 100% of executed SQL was governed?

Q8. What are MCP resources and prompts, and why did Helios lean on tools instead?

Ideal answer. Beyond tools, MCP defines *resources* (read-only addressable state the client can pull into context, like a file or a row set) and *prompts* (reusable parameterized instruction templates a server offers). Tools are for actions with side effects or computation; resources are for context the model should *read*; prompts standardize recurring instruction patterns. Helios is overwhelmingly action-oriented – compose a query, run it under budget, decompose, test significance, render – so tools dominate. Read-only context that could be a resource (the metric/dimension catalog, prior diagnoses) is instead surfaced through explicit tools (`list_dimensions` , `recall_prior`) so the access is logged in the audit trail and the model's reads are as governed as its actions. **Why Helios demonstrates it.** Even pure reads go through tools: `recall_prior` and `list_dimensions` are tool calls, not ambient resources, so every context fetch lands in the `audit_log` and the run stays fully reconstructable – a deliberate choice to keep auditability uniform across reads and writes. **Follow-ups.** When would exposing the semantic registry as a resource beat exposing it via `get_metric` ? / Does routing reads through tools cost you anything in latency or tokens? / How would prompts help if you added a self-serve drill-down chat surface?

Q9. How do you keep the byte budget and reconciliation guarantees from being bypassable through MCP?

Ideal answer. A guarantee is only as strong as its narrowest enforcement point, so I put both checks inside the one server that can execute – they cannot be routed around because there is no other executor. Every `run_query` is hard-capped at the configured `max_bytes_billed` , and crucially `run_query` refuses any SQL whose normalized hash was not seen by a prior `dry_run` in the same run, so the cost estimate that gated the call is the cost that gets billed. Reconciliation lives there too: aggregates are checked against canonical totals via `reconcile` , and >0.5% drift fails the finding. Because the agents hold no other path to bytes, the budget is an invariant of the system, not a hope. **Why Helios demonstrates it.** `warehouse-mcp.run_query` raises `NotDryRunFirst` for any un-dry-run'd SQL and `ByteBudgetExceeded` past the fixed 5 GiB cap; with the agents' allow-lists granting no alternative executor, the per-run budget and the dry-run-before-run-query gate are unbypassable. **Follow-ups.** What stops an agent from issuing many small in-budget queries to blow the run total? / How does the same-run hash check survive

query-cache or whitespace differences? / Where is the audit evidence that the budget was respected?

Tool Calling

Q10. What makes a good tool schema for an LLM, and how did you design Helios's?

Ideal answer. A good tool schema is narrow, typed, and self-documenting: it accepts only canonical, validatable inputs, returns a structured typed result (never prose the model must re-parse), and names its error modes so failures are recoverable rather than silent. I prefer enums and registry-bound names over free strings, explicit windows over implicit "recent," and outputs shaped so the *next* tool can consume them directly. The schema should make the wrong call hard to express â€” if a parameter can only be a known dimension, the model cannot pass garbage. Concretely, `build_query` takes `(metric | [metric], dims, filters, window)` and returns `{sql, governed: true, metrics, dims}`, so its output drops straight into `dry_run`. **Why Helios demonstrates it.** Helios's schemas are registry-bound and chainable: `build_query`'s typed output feeds `dry_run`, whose `{total_bytes_processed}` gates `run_query`, whose row output is summarized into `decompose_change`'s `t0 / t1` segment arrays â€” the schemas are designed as a pipeline, not isolated calls. **Follow-ups.** Why pass a window object instead of letting the model write the date predicate? / How do you keep output schemas stable as you add metrics? / What error fields does a tool need for the model to self-correct?

Q11. Walk through the guardrail chain `build_query` $\rightarrow$ `dry_run` $\rightarrow$ `run_query`. Why this exact order?

Ideal answer. The order encodes "govern, then price, then spend." `build_query` first guarantees the SQL is *legal* â€” composed only from registry metrics and known dimensions, so it can reference only real columns. `dry_run` then guarantees it is *affordable* â€” BigQuery returns the exact bytes it would scan without executing, and the server records the SQL's normalized hash. `run_query` only then *spends*, and only on SQL that passed both gates: it rejects anything not previously dry-run'd in the same run and caps `max_bytes_billed`. Reversing any step breaks a guarantee â€” you cannot price ungoverned SQL safely, and you cannot trust a cost estimate you then let the model mutate before execution. **Why Helios demonstrates it.** This is grounding rule G3 made physical in `warehouse-mcp`: the same-run hash check ties the executed query to the one whose cost was approved, so the chain is `agent` $\rightarrow$ `semantic.build_query` $\rightarrow$ `warehouse.dry_run` $\rightarrow$ `warehouse.run_query`, and only the warehouse server can complete it. **Follow-ups.** What happens to the chain when `dry_run` reports over budget â€” retry, or something smarter? / Could the model dry-run a cheap query and run an expensive one? Why not? / Where does `reconcile` slot into this chain?

Q12. Why give the LLM *tools* instead of a database handle or a SQL string it can run?

Ideal answer. A database handle hands the model unbounded capability “arbitrary SQL, arbitrary cost, arbitrary tables, and an open door to hallucinated columns and runaway scans. Tools invert that: they expose a small set of safe, validated, budgeted operations, and the model composes *intent* (which metric, which dimensions, which window) rather than *implementation* (the SQL text). That preserves the three guarantees Helios sells “governed SQL, bounded cost, reconciled numbers” none of which can survive a raw handle. The model is good at deciding *what* to ask; it should never be trusted to author *how* against a billed, schema-fragile warehouse.

Why Helios demonstrates it. No Helios agent holds a database handle or a raw-SQL execution tool; data is reached only by composing governed metrics through `semantic-mcp` and executing through `warehouse-mcp`'s gated `run_query`, which is exactly why a hallucinated column is structurally impossible. **Follow-ups.** Doesn't forbidding raw SQL limit the questions the system can answer? How do you extend coverage? / What's your answer to "but text-to-SQL tools already do this"? / How would you handle a legitimately novel query the registry can't express yet?

Q13. Why do you insist that statistical tools be *deterministic*, and how do you guarantee it?

Ideal answer. Determinism means the same inputs always produce byte-identical outputs, which is non-negotiable for a system that must be reproducible, auditable, and regression-tested. If decomposition or significance were computed in token-space, the numbers would be non-reproducible, un-unit-testable, and silently wrong under paraphrase “you could never gate them in CI. So all math runs in real, version-pinned scipy/statsmodels/prophet code behind `stats-mcp`, with a fixed RNG seed for any stochastic method; the model passes arrays in and reads numbers out, never computing. This lets me unit-test the decomposition against hand-worked golden values and assert the same answer on every run. **Why Helios demonstrates it.** `stats-mcp.decompose_change` implements the exact identity $\text{mix} = \hat{\mathbb{E}} \hat{\mathbf{w}} \cdot \mathbf{r}(t_0)$, $\text{rate} = \hat{\mathbb{E}} \mathbf{w}(t_0) \cdot \hat{\mathbf{r}}$, $\text{interaction} = \hat{\mathbb{E}} \hat{\mathbf{w}} \cdot \hat{\mathbf{r}}$, seeded at `rng_seed: 1729`, so the mix/rate/interaction split is byte-reproducible and is one of the keystone transforms with golden-value tests. **Follow-ups.** Which Helios statistics are actually stochastic, and how does the seed make them reproducible? / How would non-determinism here break your CI eval gate? / What's the failure mode if the model “rounds” a tool's number in the brief?

Q14. Sketch a tool-error taxonomy and the recovery strategy for each class.

Ideal answer. I group tool errors by the right *response*, not by where they originate. There are four useful classes: (1) **transient infrastructure** (warehouse/BigQuery flakiness, timeouts) “retry with bounded exponential backoff. (2) **budget/resource** (over-budget dry-run) “do not retry blindly; narrow the window or drop dimensions, then re-issue. (3) **governance/semantic**

(unknown metric or dimension) â€” hard stop for that branch; re-plan against `list_dimensions`, never fall back to free SQL. (4) **data-quality/logic** (segment mismatch, zero sample, reconciliation drift) â€” fail the *finding*, not the run, and let the Critic record it. The discipline is that each class has one correct reaction, so the agent never papers over a governance error as if it were transient.

```
TRANSIENT (QueryTimeout, 5xx)      → retry x3, exponential backoff
BUDGET    (ByteBudgetExceeded)      → narrow window/dims, re-dry_run (no blind retry)
GOVERNANCE(UnknownMetric/Dimension) → hard stop branch; re-plan via list_dimensions
DATA/LOGIC(SegmentMismatch, ZeroSample, reconcile >0.5%) → fail finding, log, prune
```

Why Helios demonstrates it. This is encoded in the agent layer: a `dry_run` budget rejection (G3) forces scope-narrowing rather than a blind retry, an unknown-name error (G5) is a hard branch stop, and a single failed Diagnose branch is logged and pruned rather than aborting the run, while a >0.5% reconcile drift fails just that finding. **Follow-ups.** Why is retrying an over-budget query specifically dangerous? / How do you stop a retry loop from blowing the 5-min/run SLA? / Which errors should surface to the brief versus stay in the audit log?

Q15. How do you prevent silent failures â€” a tool that returns plausible-but-wrong numbers?

Ideal answer. Silent failures are the dangerous ones because the model will confidently narrate them, so I defend with redundancy and reconciliation rather than trust. Every aggregate is reconciled against a canonical total (`reconcile`, >0.5% drift fails the finding), every finding is re-attacked by an independent adversarial agent, and every tool call â€” its args hash, the SQL run, bytes scanned, latency, verdict â€” is appended to an audit trail so a wrong number is traceable to the exact call that produced it. Keystone transforms that fail silently (sessionization, the decomposition, revenue) get golden-value tests so a regression trips CI rather than shipping. The principle is verify-then-trust: a number earns its place in the brief by surviving an independent check. **Why Helios demonstrates it.** Revenue must reconcile to the cent (the `revenue_reconciles` test), `decompose_change` is golden-tested, and the Critic re-runs `significance_test` on holdout slices and probes for confounds before a finding can ship â€” three independent silent-failure tripwires. **Follow-ups.** Why is the decomposition specifically prone to silent failure? / How does an independent Critic catch what unit tests can't? / What's your "fail loud" path when reconciliation drifts?

Q16. How do you keep large tool outputs (row sets) from poisoning the context window?

Ideal answer. The model should see *aggregates and structure*, never raw row dumps â€” both to control cost and to prevent the model from re-deriving statistics it should be calling tools for. So tool outputs that could be large are summarized to the shape the next step needs before they enter the LLM's context: a query that returns thousands of rows is collapsed to per-segment (`weight`, `rate`) arrays for the decomposition, not streamed verbatim. Inter-stage hand-off is a

compact typed JSON envelope, not prose, so token usage stays roughly flat even as the dimensional search widens. This keeps each agent's context small, cheap, and focused on decisions rather than data. **Why Helios demonstrates it.** Raw query result sets are summarized to aggregates before entering context “the LLM never sees row-level dumps” and stages hand off the typed Finding envelope (`finding_id` , `metric` , `dims` , `t0` , `t1` , `decomposition` , `significance` , `dollar_impact` , `evidence_query_ids`), so token usage stays flat as the hypothesis tree expands. **Follow-ups.** Where does the summarization happen “in the tool, the agent, or the control plane? / How do you keep `evidence_query_ids` enough to reconstruct without re-embedding the rows? / What breaks if a row set is summarized too aggressively?

Q17. How do you test tools in isolation from the unreliable LLM and the live warehouse?

Ideal answer. Tools are ordinary typed functions, so they get ordinary deterministic tests “independent of the model and, where possible, of the warehouse. Pure-compute servers (stats, experiment) are unit-tested against hand-worked golden values with a fixed seed, with zero warehouse dependency. The semantic server is tested by asserting that `build_query` emits the expected governed SQL for known metric/dimension combinations and rejects unknown names, without executing anything. Warehouse interactions are validated via `dry_run` (no spend) and golden-row fixtures, and the full pipeline is graded end-to-end by the offline eval harness on a frozen dataset “so a tool regression is caught at the unit level *and* the system level before it can reach a real run. **Why Helios demonstrates it.** `stats-mcp` runs entirely on stdio with no credentials precisely so it is unit-testable in isolation, and the keystones “`decompose_change` , sessionization, revenue reconciliation “carry golden-value tests, while the CI eval gate grades the assembled pipeline on 50 frozen labeled scenarios. **Follow-ups.** How do you write a golden test for a stochastic forecast? / How do you test the *boundary* “that `run_query` truly refuses un-dry-run'd SQL? / What's the cheapest test that would have caught a hallucinated-column regression?

Agent Orchestration

Q18. Why a deterministic finite-state machine with model-driven *nodes*, rather than letting the LLM decide the flow?

Ideal answer. Control flow and reasoning are different jobs and should be owned by different things. Letting an LLM decide *what to do next* (an open agent loop) is non-reproducible, hard to budget, and hard to test “the same input can take different paths on different runs. So I make the *flow* deterministic code “a finite-state machine with named states and explicit transitions “while the LLM is a *composer within a node*: at each state it decides which tool calls to emit, but it cannot change the order of stages or skip the Critic. This gives the reasoning flexibility of agents (a best-first hypothesis search, an adversarial refutation) with the reproducibility, auditability, and

cost control of a fixed pipeline. The slogan: the model composes tool calls; it never controls flow.

Why Helios demonstrates it. The Orchestrator drives a fixed FSM “ PLAN → MONITOR → (CLEAN_RUN | DECOMPOSE → DIAGNOSE → CRITIC LOOP → PRESCRIBE → NARRATE) → END ” and worker output never ships directly; it must pass the Critic loop, so the diagnosis path is identical and reconstructable every run. **Follow-ups.** What concretely becomes untestable if the LLM controls transitions? / How do you handle a genuinely novel case the FSM has no state for? / Where in a node does the model still have real latitude?

Q19. Why seven agents instead of one well-prompted model?

Ideal answer. Specialization and adversarial review are the two things a single prompt cannot do well. A monolithic prompt asked to both confidently propose a root cause *and* rigorously refute it is in conflict with itself “ confirmation bias is structural. Splitting into role-specialized agents lets each have a tight system prompt, a minimal tool allow-list, and an isolated, testable responsibility, and lets one agent be purpose-built to *attack* another's output. It also lets me right-size the model per role and keep each agent's context small. The cost is orchestration complexity and inter-agent latency, which the deterministic FSM and typed hand-offs keep bounded. Multi-agent here is not fashion; it is how I get an adversarial check and least-privilege at the same time. **Why Helios demonstrates it.** The seven agents (Orchestrator, Monitor, Decompose, Diagnose, Prescribe, Narrator, Critic) each own one stage with its own allow-list, and the Critic exists specifically to refute the Diagnose agent “ a separation a single prompt cannot honestly reproduce. **Follow-ups.** Where is the latency cost of seven hops, and is it inside the 5-min budget? / Which two agents could you merge with least harm, and which must never merge? / How do you stop the agents from duplicating each other's queries?

Q20. How did you decide Opus versus Sonnet per agent?

Ideal answer. I matched model capability to the reasoning depth and cost-of-error of each role, not to seniority for its own sake. Opus goes to the three roles that search an open hypothesis space or carry a high cost of being wrong: the Orchestrator (global planning, budget allocation, branch routing), Diagnose (combinatorial hypothesis-tree search with self-directed SQL verification), and the Critic (adversarial reasoning that must out-think Diagnose to find a confound). Sonnet handles the high-volume, bounded-action-space stages “ Monitor, Decompose, Prescribe, Narrator “ where the decision is “which series to test,” “which decomposition to run,” “which power analysis to call,” “which template to fill.” This keeps cost per run inside the fixed budget while spending the expensive model only where judgment actually moves accuracy. **Why Helios demonstrates it.** The Opus/Sonnet split is exactly Orchestrator/Diagnose/Critic on Opus and Monitor/Decompose/Prescribe/Narrator on Sonnet “ reserving the costly model for open-ended reasoning and adversarial critique, the two places error is most expensive. **Follow-ups.** Could the Decompose agent be Sonnet *because* the math is

in a deterministic tool “ is that the reason? / When would you promote Monitor to Opus? / How do you measure whether an Opus role is worth its cost?

Q21. Explain the Critic loop “ verdicts, the retry bound, and why an adversarial agent matters.

Ideal answer. The Critic is a dedicated adversary that tries to *refute* every candidate finding along the four classic failure modes “ mix confound, insufficient sample, seasonality, and data-quality artifact “ and returns one of three verdicts. PASS: the finding survived refutation and proceeds to Prescribe. DOWNGRADE: plausible but under-evidenced (sample too small, decomposition not isolated to one dimension), so it returns to Diagnose for a *targeted* re-query, bounded by `MAX_REFUTE_ROUNDS = 2` ; after the cap it is demoted to a watchlist note rather than a confident finding. DROP: refuted (confound found, fails reconciliation, fully explained by known seasonality/launch from memory), discarded and locally suppressed so siblings don't re-raise it. The bound matters because an unbounded refute loop would blow the time and byte budgets; the adversary matters because the cheapest way to make a finding trustworthy is to try hard to break it first. **Why Helios demonstrates it.** The Critic (Opus) consults `recall_prior` for seasonality/launch and suppression, re-runs `significance_test` on holdout slices, and probes for a traffic-mix confound via `run_query` “ a structurally separate refutation pass that is the third defensibility pillar alongside governed SQL and deterministic stats. **Follow-ups.** Why cap at two refute rounds specifically “ what happens at the cap? / How does the Critic avoid simply agreeing with Diagnose? / What does a DROP add to suppression, and how is that scoped to the run?

Q22. What is the Finding envelope, why typed JSON, and how does it enable hand-off?

Ideal answer. The Finding envelope is the single typed contract that every stage produces and consumes, so agents communicate via structured data, not prose. It carries the identity and full evidentiary state of a candidate: the metric, dimension slice, time window, the mix/rate/interaction decomposition, the significance result, the dollar revenue-at-risk, and pointers to the evidence queries. Typed JSON matters for three reasons: it is validatable (a malformed finding fails fast), it is compact (token usage stays flat as the search widens), and it is the same shape the memory store persists “ so a finding flows from Diagnose through Critic to Narrator to `save_diagnosis` without re-parsing. Prose hand-off would be lossy, unvalidatable, and would tempt downstream agents to re-derive numbers they should inherit. **Why Helios demonstrates it.** The hand-off envelope is exactly `{finding_id, metric, dims, t0, t1, decomposition, significance, dollar_impact, evidence_query_ids}` , and it maps onto the `diagnosis_history` schema so the same typed object the Critic vets is what the Narrator renders and `save_diagnosis` persists. **Follow-ups.** Why include `evidence_query_ids` rather than the rows themselves? / What validation runs on the envelope between stages? / How does the envelope's shape line up with what memory needs to recall later?

Q23. How does context compaction work across the pipeline, and why is it load-bearing?

Ideal answer. Compaction means each agent receives the *minimum* context it needs to act, summarized and structured, rather than the accumulated transcript of the whole run. Concretely a worker gets: the run plan, the canonical metric/dimension catalog, the upstream stage's typed output (JSON, not prose), and any priors from `recall_prior` – and raw result sets are reduced to aggregates before they enter context. This is load-bearing for two reasons: cost and time (a flat, small context per agent keeps a multi-agent run inside the <5-min and token budgets), and *quality* (a focused context reduces distraction and keeps the model on its one job). Without compaction, a deep hypothesis-tree search would balloon context linearly with the number of branches and both cost and latency would blow. **Why Helios demonstrates it.** Helios compacts deliberately – summarized aggregates instead of row dumps, typed Finding envelopes instead of prose, and `recall_prior` priors decayed by $\exp(-\text{age_days}/\text{HALF_LIFE})$ – so token usage stays flat even as Diagnose widens its dimensional search to the bounded `MAX_BRANCHING = 4`, `MAX_DEPTH = 3`. **Follow-ups.** What information is safe to drop in compaction, and what must always survive? / How does memory decay interact with context size over many runs? / How would you detect that compaction silently dropped a needed fact?

Q24. How does the FSM handle failure isolation, retries, and the clean-run case?

Ideal answer. A robust pipeline degrades gracefully and never ships a partial result. Failure is isolated by scope: a single failed Diagnose branch is logged and pruned, not fatal, because one dead hypothesis shouldn't kill the run; but if Monitor or Decompose fail entirely, the Orchestrator aborts and writes an audit record rather than emitting a half-built brief. Retries are class-specific: transient warehouse errors back off and retry three times, a budget rejection forces scope-narrowing instead of a blind retry, and a governance error hard-stops that branch. The clean-run case is first-class: if Monitor finds no anomaly, the FSM short-circuits to a "no material change" brief and *still* writes a run-state audit row, so absence-of-finding is itself recorded and the run is reproducible. **Why Helios demonstrates it.** The FSM's `CLEAN_RUN → NARRATE(no-finding brief) → END` branch and the abort-with-audit-on-Monitor/Decompose-failure path are explicit, and every tool call lands in `audit_log` (args hash, SQL, bytes, verdict) so even a clean or aborted run is fully reconstructable. **Follow-ups.** Why record a clean run at all? / How do you prevent a flaky branch from silently degrading accuracy? / What's the recovery if the Orchestrator itself crashes mid-run?

Q25. How do you keep an autonomous multi-agent run cheap, bounded, and on-time?

Ideal answer. Autonomy is only safe if it is bounded, so cost and time are enforced at every layer rather than hoped for. The byte budget is hard-capped per `run_query` and the dry-run gate prices every query before it spends; the hypothesis search is breadth- and depth-bounded so it

cannot fan out indefinitely; the Critic loop is capped at two refute rounds; the model split spends Opus only on reasoning-critical roles; and context compaction keeps token cost flat. The Orchestrator also watches the running byte total and tightens scope mid-run if it nears the cap. Together these turn "an AI agent loose on a warehouse" into a run with a known worst-case cost and a sub-five-minute SLA – which is what makes scheduling it unattended defensible. **Why Helios demonstrates it.** The fixed 5 GiB byte budget (`byte_budget: 5368709120`), `MAX_BRANCHING = 4` / `MAX_DEPTH = 3` in Diagnose, `MAX_REFUTE_ROUNDS = 2` in the Critic, the Opus/Sonnet split, and mid-run scope-tightening off the `run_state.bytes_scanned` counter jointly hold each run under 5 minutes and inside budget. **Follow-ups.** Which bound binds first in practice – bytes, time, or tokens? / How do you choose the depth/branching caps without hurting the 85% accuracy target? / What's your alerting if a scheduled run breaches its budget?

Evaluation

Q26. How do you evaluate an autonomous diagnosis engine when there's no labeled "right answer" lying around in production?

Ideal answer. You manufacture ground truth. The general technique is to take a frozen real dataset, inject anomalies whose root cause you authored, and score whether the system rediscovers exactly what you planted – turning an unmeasurable open-ended task into a closed-form accuracy problem. I built an offline labeled benchmark: a frozen copy of the GA4 dataset into which I perturb specific segments, with a YAML label recording the true cause (which segment, mix vs rate, dollar impact), then run the full pipeline and grade its top-ranked finding against that label. This is strictly better than eyeballing a few outputs because it's repeatable, regression-gateable in CI, and reports a number you can defend. The alternative – LLM-as-judge over unlabeled prod runs – measures plausibility, not correctness, and an engine that confidently invents plausible-but-wrong causes is the exact failure I'm trying to kill. **Why Helios demonstrates it.** The `helios.eval` harness (`injector.py` -> `runner.py` -> `scorers/`) grades against authored labels in `scenarios/*.yaml` over a frozen `helios_eval` dataset, and the gate lives in CI. **Follow-ups.** Why is a frozen dataset copy essential – what breaks if you inject into live data? / How do you keep labels honest so you aren't grading the system against your own confirmation bias? / What's the cost of maintaining a labeled benchmark as the schema evolves?

Q27. Walk me through synthetic anomaly injection. How do you inject a *mix-shift* differently from a *rate-change*?

Ideal answer. The two perturbation modes map directly onto the decomposition identity $\hat{I}^R = \hat{I}^E \hat{I}^{W_i} \hat{I}^{R_i} + \hat{I}^E \hat{I}^{W_i} \hat{I}^{R_i} + \hat{I}^E \hat{I}^{W_i} \hat{I}^{R_i}$, so I inject each by moving

exactly one term. A **rate perturbation** holds segment volume shares `w_i` constant and depresses one segment's conversion `r_i` (e.g. drop mobile/Chrome `checkout_to_purchase_rate` by 20% by deleting a fraction of that segment's downstream `reached_*` flags) — true behavior change. A **volume/mix perturbation** holds every segment's `r_i` constant and reweights traffic `w_i` (e.g. shift session share from high-converting Direct to low-converting Display by duplicating/dropping whole sessions) — composition artifact with no behavior change. The discipline is that injection touches *one* lever so the label is unambiguous: a correct system must attribute a rate injection to `rate_effect` and a mix injection to `mix_effect`, and the naive "biggest absolute segment delta" baseline conflates them. The label stores the segment, the mode, the magnitude, and the derived dollar-at-risk so I can score cause, classification, and MAPE. **Why Helios demonstrates it.** `eval/injector.py` implements exactly two primitives — rate-perturbation (mutate `r_i` via `reached_*` flags) and volume-perturbation (mutate `w_i` via session resampling) — so each scenario's label is a clean mix-or-rate ground truth. **Follow-ups.** How do you prevent an injection from also accidentally moving a parent metric and contaminating a "single-cause" label? / Why inject by editing `reached_*` flags rather than the final rate column? / How do you inject a seasonality *decoy* that should NOT be flagged?

Q28. Describe the 50-scenario suite. Why those buckets, and what failure does each one police?

Ideal answer. Coverage matters more than count — a benchmark is only as good as the failure modes it can catch, so I designed seven buckets, not 50 random anomalies. They are: single-segment rate change (the base case), multi-segment rate change (additivity under several simultaneous drops), single-segment mix shift and multi-segment mix shift (Simpson's-paradox traps), seasonality decoys (a real-looking dip that is calendar-explained and must NOT be alarmed), no-anomaly controls (the system must stay silent — guards false-positive rate), and data-quality artifacts (a tracking gap or duplicate that looks like a drop but is an ingestion bug). The mix buckets specifically police the engine's core thesis: a baseline that chases the largest delta will misattribute composition artifacts to behavior. The controls and decoys police the opposite failure — an always-on agent that cries wolf gets muted, so silence-when-correct is a graded outcome, not an afterthought. **Why Helios demonstrates it.** The `scenarios/` set spans all 7 buckets; the no-anomaly and seasonality-decoy buckets are what let me report a real precision number and prove the Critic earns its keep. **Follow-ups.** Why include data-quality artifacts at all — isn't that the warehouse's job? / How do you weight buckets when computing a headline score? / What's the minimum scenarios-per-bucket for the accuracy number to be statistically meaningful?

Q29. What exactly do you measure per run? Give me the metric definitions, not just names.

Ideal answer. Five graded quantities, each tied to a trust pillar. (1) **Root-cause segment accuracy** = fraction of scenarios where the top-1 promoted finding names the injected segment *and* the correct mix/rate classification (the headline). (2) **Decomposition MAPE** = mean absolute percentage error of `stats-mcp` 's reported mix/rate/interaction magnitudes against the injected dollar/rate deltas "graded" $\approx 10\%$. (3) **Detection F1** over the anomaly-vs-control split, so missed anomalies and false alarms both cost. (4) **Hallucination rate** = count of columns/metrics in emitted SQL absent from the governed registry "a hard-zero gate. (5) **Faithfulness** = fraction of numeric claims in the Decision Brief that are verbatim tool outputs traceable to an `audit_log` row, not paraphrases. Accuracy is the product number; MAPE validates the math pillar; F1 validates detection; hallucination and faithfulness validate the grounding pillar. **Why Helios demonstrates it.** Each metric is a scorer under `eval/scorers/` (`accuracy.py` , `decomposition.py` , `detection.py` , `hallucination.py` , `faithfulness.py`); the headline targets are $\approx 85\%$ accuracy, $\approx 10\%$ MAPE, 0 hallucinations. **Follow-ups.** Why top-1 and not top-3 recall for the headline? / How do you score a finding that's right on segment but wrong on dollar magnitude? / How is faithfulness scored mechanically without another LLM in the loop?

Q30. Define the naive baseline precisely. Why 85% vs 45%, and why is that gap the whole point?

Ideal answer. The baseline must be the thing a competent analyst or a junior tool would actually do, or the comparison is a strawman. Mine is "**largest absolute segment delta**": cross-tab the metric by each dimension, rank segments by `|ÎR_segment|` , and report the biggest mover as the cause "no decomposition, no significance, no mix/rate split. It scores around 45% because it gets single-segment rate changes roughly right but is *systematically* fooled by mix shifts (it flags the segment whose volume moved, not the one whose behavior moved) and by small-but-significant changes it under-ranks. Helios hits $\approx 85\%$ because `decompose_change` separates the levers and Diagnose drills rate-effects first. The gap is the entire product thesis quantified: the hard part isn't surfacing a number, it's attributing it correctly, and 40 points of accuracy is the dollar value of doing the decomposition properly. **Why Helios demonstrates it.** The baseline runs in the same harness over the same scenarios, so `85 vs 45` is an apples-to-apples number, and the mix buckets are precisely where the gap opens. **Follow-ups.** Could you make a stronger baseline (e.g. delta normalized by segment volume) and does the gap survive? / On which bucket does the baseline come closest to Helios? / Why is "largest delta" the right strawman for *this* product rather than a random baseline?

Q31. How does eval run in CI, and what exactly does it gate?

Ideal answer. A benchmark that isn't a merge gate is a vanity metric, so I wired it into GitHub Actions with two tiers. Every push runs a **smoke subset** (a fast representative slice across buckets) for quick signal; every PR to `main` runs the **full suite**. The gate (`eval/gates.yaml`) blocks the

merge if top-1 accuracy regresses more than ~2 points from the baseline-on-main, if hallucination is anything but zero, if decomposition MAPE exceeds 10%, or if a run breaks the 5 GiB byte budget or the <5-min latency SLA. Crucially the `same_hallucination.py` scorer that gates CI also runs over the production `audit_log`, so the offline invariant and the online observation are enforced by one piece of code. This makes correctness a property the repo protects mechanically, not a claim that quietly rots. **Why Helios demonstrates it.** `.github/workflows/ci.yml` invokes `eval.runner --smoke` on push and `--full` on PR-to-main; `eval/gates.yaml` encodes the accuracy/hallucination/MAPE/budget thresholds as hard gates. **Follow-ups.** Why a 2-point regression tolerance and not zero? / How do you keep full-suite eval cost inside a CI budget given BigQuery scans? / How do you stop the benchmark from being silently overfit by repeated tuning against it?

Q32. What stops you from overfitting the agents to your own 50 scenarios?

Ideal answer. A benchmark you tune against eventually measures memorization, so I treat it like a held-out set with hygiene rules. I keep the injection *primitives* generic (perturb any `(segment, mode, magnitude)`) rather than hand-crafting bespoke cases the agents could pattern-match, and I parameterize scenarios so I can resample fresh segment/magnitude combinations the prompts have never seen. I separate a smoke subset used during development from the full suite that gates merges, and I watch per-bucket scores, not just the headline, so improving rate-cases by breaking mix-cases shows up immediately. The honest limitation I'd state in an interview: 50 scenarios over one observational dataset is a starting harness, not a guarantee of generalization, and the right next step is randomized scenario generation plus a second dataset. **Why Helios demonstrates it.** The `injector` is primitive-based and parameterized rather than a fixture of frozen answers, and the smoke/full split (`--smoke` vs `--full`) keeps a portion out of the daily tuning loop. **Follow-ups.** How would you build a true train/test split for scenarios? / What signal tells you you've started overfitting? / Would you trust the 85% on a second e-commerce dataset, and why or why not?

Q33. Why is an offline benchmark a stronger artifact than a slick live demo?

Ideal answer. A demo proves the system *can* be right once; a benchmark proves how often it *is* right, and the second is what earns trust for an unattended agent. Live demos are cherry-picked, non-reproducible, and silent about failure modes “they answer “does it look impressive” not “what's the error rate on mix-shift traps.” The benchmark gives me a defensible distribution: 85% top-1, zero hallucinations, MAPE under 10%, against a stated 45% baseline, reproducible on every commit. For autonomous tooling specifically this matters more than for a chatbot, because no human is in the loop to catch a wrong answer at runtime “the only place correctness can be established is offline, in advance. In an interview the benchmark also lets me discuss failure honestly, which reads as senior judgment rather than salesmanship. **Why Helios demonstrates it.**

The headline is reported as a graded distribution from `eval.runner`, not a screenshot – the trust story is "measured 85% vs 45%," which is the third defensibility pillar (adversarial verification + offline eval). **Follow-ups.** When IS a live demo the better artifact? / How do you communicate eval results to a non-technical stakeholder? / What would change your headline metric if a customer's funnel looked nothing like the Merch Store?

Prompt Engineering

Q34. How are the system prompts structured across the seven agents – one mega-prompt or per-agent?

Ideal answer. Per-agent, narrow, and role-scoped – a single mega-prompt invites the model to do everything badly and makes failures unattributable. Each agent gets a system prompt that states only its job, its allowed tools, its output contract, and the rules it must obey, so the prompt surface for any one node is small and testable. The Monitor's prompt knows only "call `detect_anomaly`, emit a typed Finding or 'clean'"; the Diagnose prompt knows the best-first search policy and the rate-before-mix ordering; the Critic's prompt is purely adversarial – "try to refute this finding via these four attack templates." The shared spine across all prompts is the grounding contract (G1-G5) and the typed "Finding" JSON envelope every agent reads and writes. Smaller role-scoped prompts also let me pick model tier per agent (Opus for Diagnose/Critic/Orchestrator reasoning, Sonnet for the mechanical nodes), which is both cheaper and more reliable. **Why Helios demonstrates it.** Seven distinct agents (`helios/agents/`) each carry a focused system prompt plus a per-agent MCP tool allow-list (Bible §18.9) – e.g. the Narrator's prompt and allow-list both forbid `run_query`. **Follow-ups.** How do you keep seven prompts consistent without copy-paste drift? / Why give Diagnose and Critic Opus but Narrator Sonnet? / How do you version and test a prompt change without a full eval run?

Q35. State the grounding rules G1-G5 and explain which are behavioral and which are structural.

Ideal answer. **G1** never emit raw SQL – get data only via `semantic-mcp.build_query` / `get_metric`. **G2** never compute a statistic in prose – anomaly, decomposition, significance, forecast, power come only from `stats-mcp` / `experiment-mcp`, and numbers in the brief are tool outputs verbatim. **G3** `warehouse-mcp.dry_run` before every `run_query`; over-budget means narrow the query, not retry blindly. **G4** reconcile aggregates via `warehouse-mcp.reconcile`; >0.5% drift fails the finding. **G5** use only canonical metric/dimension names – an unknown name is a hard error, never a fallback to free SQL. The distinction that matters: stating these in a prompt is *behavioral* (the model is asked to comply) but Helios also enforces them *structurally* – the agent literally has no SQL tool, only `semantic-mcp`; the per-agent allow-list makes `run_query` uncalleable from a writer agent; the byte gate and reconcile run

in `warehouse-mcp` regardless of what the model "intends." The prompt makes compliance the path of least resistance; the architecture makes violation impossible. **Why Helios demonstrates it.** G1-G5 are documented as the five non-negotiables in `CLAUDE.md`, and each has a structural counterpart: no raw-SQL tool exists, allow-lists gate tools, `dry_run` / `reconcile` are mandatory in `warehouse-mcp`. **Follow-ups.** Which of G1-G5 would still hold if the prompt were blank, and which wouldn't? / What's the failure mode if G3's `dry_run` is advisory rather than enforced? / How does G4's 0.5% threshold interact with floating-point rounding in the warehouse?

Q36. Why do you claim "structural beats prompt-only"? Convince me a prompt isn't enough.

Ideal answer. A prompt is a *request* to a probabilistic system; under distribution shift, long context, or an adversarial input it will eventually be violated, and for an unattended agent "eventually" means "in production, unobserved." The senior move is to make the desired property a structural invariant so violation isn't a low-probability event, it's impossible. Concretely: I can ask the model in a prompt to "only use governed metrics" (prompt-only) or I can give the agent no SQL-execution tool at all and route every data request through `semantic-mcp`, so the model *cannot* express a hallucinated column even if it wanted to (structural). Prompt-only gets you maybe 99% compliance; structural gets you 100% and, more importantly, gets you *provability* – I can scan `audit_log.sql_text` and assert every column came from the registry. The tradeoff is real: structural governance is more upfront engineering than a paragraph of instructions, and it constrains what the model can do, which is exactly why it's safe. **Why Helios demonstrates it.** Helios's hallucination rate is a hard-zero CI gate, achievable only because `semantic-mcp` is the *sole* SQL path and the registry is the single source of truth – the model never holds a SQL pen. **Follow-ups.** Give an example where prompt-only grounding silently fails at 99% compliance. / Does structural governance ever block a legitimate analysis the model should do? / How do you add a new metric without loosening the structural gate?

Q37. Walk me through writing the Critic's prompt. How do you prompt a model to attack rather than agree?

Ideal answer. LLMs are sycophantic by default – they want to ratify the finding handed to them – so the Critic's prompt inverts the objective: its job is to *refute*, and a finding survives only by surviving attack. I give it a fixed set of attack templates rather than open-ended "is this right?", because named adversarial moves are testable and reproducible: mix-confound (is this composition not behavior?), insufficient-sample (does `significance_test` actually clear?), seasonality (is this calendar-explained?), and data-quality (is this a tracking artifact?). The prompt forces a structured verdict – PASS, DOWNGRADE, or DROP – with a cited reason, and routes every check through tools (re-run `decompose_change`, consult the `seasonality_calendar`) so the refutation is grounded, not rhetorical. Framing it as "earn the finding's survival" rather than "grade the finding" is what flips the model out of agreement mode. **Why Helios demonstrates it.**

The Critic (Opus) runs four named refutation templates and emits a PASS/DOWNGRADE/DROP verdict per the Finding envelope; it consults memory's

`seasonality_calendar` / `launch_calendar` and re-runs `stats-mcp` rather than judging in prose. **Follow-ups.** How do you stop the Critic from being so aggressive it drops every real finding? / How do you eval the Critic in isolation? / Why named attack templates instead of a free-form adversarial prompt?

Q38. What does refusal of free SQL look like in practice “ show me the path when the model wants a column that doesn't exist.

Ideal answer. The model never gets to "want a column" in SQL terms because the only data verb it can call is `semantic-mcp.build_query(metric, dims, filters, window)` over canonical names. If it composes a call referencing a name not in the registry, `semantic-mcp` returns a hard error listing the valid names “ there is no escape hatch that drops to hand-written SQL (G5). For example:

```
build_query(metric="add_to_cart_conversion", dims=["device"])
→ ERROR: unknown metric "add_to_cart_conversion" (did you mean "view_to_cart_rate?");
unknown dimension "device" (did you mean "device_category?"). No SQL emitted.
```

The agent's correct response is to re-compose with valid names, not to route around the gate, because no around exists “ there's no `run_sql` tool in its allow-list. This is the structural enforcement of G1/G5: a hallucinated column is caught at compose time as a typed error, never reaching the warehouse, which is why the eval can claim a hard zero on hallucinations. **Why Helios demonstrates it.** `semantic-mcp.build_query` validates every metric/dimension against `semantic_layer.yaml` (47 metrics + 19 dimensions, referential-integrity compiled) and errors on unknowns; no agent has a raw-SQL tool. **Follow-ups.** Should the error suggest near-matches, and does that risk the model rationalizing a wrong metric? / What if the analysis genuinely needs a metric not yet in the registry? / How does this differ from a text-to-SQL system's "validate after generate" approach?

Q39. How do you prompt for the typed "Finding" envelope, and why a typed handoff at all?

Ideal answer. Free-text handoffs between agents lose structure and let errors compound silently, so every agent reads and writes one typed JSON "Finding" “ segment, metric, mix/rate classification, magnitudes, significance, dollar impact, Critic verdict, and provenance. The prompt specifies the schema and instructs the agent to emit *only* valid JSON conforming to it, and I validate the output against the schema before the FSM advances “ a malformed envelope is a node failure, not a downstream surprise. This makes the pipeline composable (Diagnose's output is exactly Critic's input), debuggable (every field is inspectable in `audit_log`), and lets the deterministic FSM, not prose, control flow. The envelope is also what decouples the agents from

the data: they pass typed findings, never rows, which is why `stats-mcp` and the workers are stateless and replicable. **Why Helios demonstrates it.** The typed "Finding" envelope is the contract handed between the seven agents on the FSM; validating it at each node is what lets the Orchestrator drive flow deterministically rather than parsing model prose. **Follow-ups.** What do you do when the model emits prose around the JSON? / How do you evolve the Finding schema without breaking older diagnoses in memory? / Why JSON over a tool-call/function-calling structured output?

Memory Systems

Q40. Describe Helios's memory. What's persisted, and in what stores?

Ideal answer. Memory turns a one-shot analyst into one that learns across runs, so I persist four things in `helios_memory` (BigQuery) plus a vector store. (1) **Diagnosis history** "every shipped finding with its embedding, so a new finding can be matched against priors semantically, not just by exact segment match. (2) A **suppression list** "acknowledged or auto-suppressed findings with a reason and TTL, so the engine stops re-alarms on known issues. (3) **Seasonality and launch calendars** "known periodic effects and product/marketing events the Critic uses to refute calendar-explained dips. (4) An **action-tracking** table "the prescribed experiment, its status, and the measured readback, closing the did-the-fix-work loop. BigQuery holds the structured rows and durable state of record; the vector store holds embeddings for semantic recall. Crucially, all cross-run learning lives in these stores, never in an agent's heap, which is what keeps the agents stateless and the system restartable. **Why Helios demonstrates it.**

`helios_memory` + vector store hold diagnosis history (embeddings), suppression list, seasonality/launch calendars, and action-tracking; `report-mcp` exposes `save_diagnosis` / `recall_prior` as the memory interface. **Follow-ups.** Why split structured rows (BigQuery) from embeddings (vector store)? / What's the schema of the diagnosis-history row? / How does keeping state out of the agent heap help horizontal scale?

Q41. How does `recall_prior` work, and how does it change a diagnosis?

Ideal answer. `recall_prior` is semantic retrieval over diagnosis history: given the current finding (segment, metric, movement), it embeds the query and returns the nearest prior diagnoses plus their outcomes and any suppression/calendar matches. This changes the diagnosis three ways "it lets Diagnose say "we've seen this mobile-Safari checkout drop before, here's what we found and whether the fix worked," it lets the Critic check whether this exact issue is suppressed or calendar-explained before promoting it, and it lets the Narrator add continuity ("recurring, third time this quarter") instead of treating every run as amnesiac. The retrieval is semantic so it matches *related* priors (a Safari issue surfacing the prior Chrome-mobile finding), not just byte-identical ones. The guardrail: recall *informs* but never *overrides* the live

decomposition “priors are evidence the current tools must still confirm, so a stale prior can't manufacture a finding the data doesn't support. **Why Helios demonstrates it.** `report-mcp.recall_prior` performs embedding-based retrieval over the diagnosis history; Diagnose and the Critic both consult it before promoting a leaf. **Follow-ups.** How do you stop a confident prior from biasing the model toward a wrong-but-familiar cause? / What's your embedding model and similarity threshold? / How do you handle a prior whose schema/metrics have since changed?

Q42. Explain the suppression list and its TTL. Why is "don't re-alarm" a memory problem?

Ideal answer. An always-on engine that re-reports the same known issue every run gets muted by its operators “so suppression is the difference between a tool people keep running and one they turn off. When a stakeholder acknowledges a finding (or the Critic auto-suppresses a known artifact), I write a suppression row with the segment/metric signature, a human reason, and a TTL (default 30 days). Diagnose and Critic consult it before promoting a leaf; a match is auto-DROPPed with reason "suppressed." The TTL is the subtle part: suppression must *expire* so a genuinely re-emerging problem eventually re-surfaces rather than being silenced forever “a permanent mute is how you miss the second, real occurrence. This is a memory problem because it's cross-run state about operator intent, which can't live in a stateless agent or be re-derived from the data alone. **Why Helios demonstrates it.** The TTL'd suppression list in `helios_memory` is consulted by Diagnose and the Critic; a match yields an auto-DROP verdict with reason "suppressed." **Follow-ups.** Why 30 days and not indefinite or 7 days? / How do you distinguish "suppress this instance" from "suppress this pattern forever"? / What happens if a suppressed issue's dollar impact suddenly triples within the TTL?

Q43. How does the action-tracking loop close, and why does it make Helios more than a reporter?

Ideal answer. A diagnosis without a feedback loop is a reporter; closing the loop is what makes it an analyst that's accountable for its prescriptions. When Prescribe ships an experiment card, I write an action-tracking row (the recommended action, the targeted segment/metric, the expected effect, status). Because the dataset is observational, Helios can't run a live A/B, so it closes the loop with a *quasi-experimental readback* “pre/post or difference-in-differences on the segment after the action's stated date “and writes the measured outcome back to the row. That outcome then feeds future `recall_prior` calls, so the next time a similar issue appears the engine knows whether its prior fix actually worked. This is the did-the-fix-work loop, and it's what lets Helios improve its prescriptions over time instead of repeating advice that didn't land. **Why Helios demonstrates it.** Action-tracking in `helios_memory` plus the honest constraint that the data is observational (Nov 2020-Jan 2021) “so Helios designs/sizes experiments and runs pre/post or diff-in-diff readbacks rather than live A/Bs. **Follow-ups.** Why is diff-in-diff a defensible readback on observational data, and where does it break? / How do you attribute a

metric move to your action vs confounders? / What do you do when the readback says the fix didn't work?

Q44. Memory grows unbounded. How do you handle decay, staleness, and relevance over time?

Ideal answer. Unbounded memory degrades both cost and quality â€” stale priors mislead recall and the byte budget balloons â€” so memory needs decay just like a cache. Suppression entries decay via their TTL. Diagnosis history I'd weight by recency and by whether the action-tracking readback confirmed the diagnosis, so a confirmed recent finding outranks an old unconfirmed one in `recall_prior`; outright pruning or archiving very old, low-relevance diagnoses keeps the vector index lean. Seasonality entries are inherently periodic so they persist, but launch-calendar entries age out once their effect window passes. The principle is that recall relevance should reflect *current* truth, and that means actively forgetting: a year-old finding about a since-redesigned checkout is noise, not signal. Honestly, at 3 months of data this is more design-for-scale than a present pain, but it's the right thing to build before memory is large. **Why Helios demonstrates it.** TTL'd suppression plus recency/confirmation-weighted recall over the embedded diagnosis history â€” decay is a first-class property of the memory schema, not an afterthought. **Follow-ups.** How would you measure whether decay is helping or hurting recall quality? / Should a *contradicted* prior be deleted or kept as a negative example? / How do you bound the vector store's growth in a multi-tenant deployment?

Architecture Decisions

Q45. The biggest decision: grounding over generation. Defend it against "just let a strong model write the SQL."

Ideal answer. Letting a strong model free-text SQL optimizes for *capability demo*; grounding optimizes for *correctness at scale*, and for an unattended engine the second is the only one that matters. A model writing SQL will, at some rate, hallucinate a column, mis-define a rate as an average-of-ratios (re-introducing Simpson's paradox), or silently change a metric's definition between runs â€” and no one is watching at 6am. Grounding routes every query through `semantic-mcp` over a governed registry, so columns are provably real, definitions are stable, and the same `revenue` means the same thing every run. The tradeoff is honest: I gave up the model's flexibility to express arbitrary novel queries, and I pay upfront cost building the semantic layer. But that's exactly the trade that buys a hard-zero hallucination gate, multi-tenant scale (add a registry, not a fork), and reproducibility. The thesis: the hard part of an AI analyst isn't generating insights, it's making them correct and trusted â€” and you can't trust SQL you can't prove. **Why Helios demonstrates it.** `semantic-mcp` as the sole SQL path over `semantic_layer.yaml` is grounding pillar #1; it's what makes "0 hallucinated columns" a CI gate rather than a hope. **Follow-ups.** What's the largest class of legitimate query your grounded approach *can't* serve? / How do you

keep the registry from becoming a bottleneck on analyst velocity? / Where would you allow generated SQL, if anywhere?

Q46. "Determinism where it matters" – where do you allow non-determinism, and where do you forbid it?

Ideal answer. The rule is: non-determinism is fine for *search*, forbidden for *truth*. I let the model be non-deterministic in which hypotheses it explores – that's creative search over a combinatorial space and variance there is acceptable because the Critic and significance gates filter it. I forbid non-determinism in three places: the math (all decomposition/significance/forecast/power run in seeded Python in `stats-mcp`, reproducible byte-for-byte), the SQL (emitted from a fixed registry, so two runs reference the same columns), and the control flow (a deterministic FSM, not the model, decides node order). So the model is a *composer inside a deterministic shell*: its variance is confined to hypothesis selection, and every number in the brief is a verbatim tool output reconstructable from `audit_log`. The failure I'm avoiding is "the AI said so" – a system where you can't tell whether a different answer means new data or just a different sample from the model. **Why Helios demonstrates it.** Deterministic FSM + seeded `stats-mcp` + governed `semantic-mcp` confine non-determinism to which hypotheses Diagnose explores, which the Critic then filters. **Follow-ups.** How do you make scipy/statsmodels/prophet runs actually reproducible? / If hypothesis selection varies, can two runs disagree on the top finding – and is that acceptable? / Where would more determinism hurt rather than help?

Q47. Build vs buy: you used Claude Agent SDK and MCP but built your own semantic layer, stats, and eval. Why that line?

Ideal answer. I bought the commodity and built the moat. The agent runtime, tool-calling protocol (MCP), and CI are commodities – well-solved, not differentiating – so I used the Claude Agent SDK and MCP rather than reinventing an orchestration framework. But the things that make Helios *trustworthy* are the differentiators, and those I built: the governed semantic layer (mapping 1:1 to dbt MetricFlow, so I'm not even reinventing metric definitions – I'm aligning with the standard), the deterministic stats in real Python, and the offline labeled eval. I also deliberately *didn't* buy a generic text-to-SQL or BI tool because their "validate after generate" model can't give me a hard-zero hallucination guarantee. The principle: build where correctness and trust live, buy where the problem is already solved, and don't build a framework when a protocol exists. **Why Helios demonstrates it.** Stack = bought MCP + Claude Agent SDK + GitHub Actions; built `semantic_layer.yaml` (1:1 to MetricFlow), `stats-mcp`, and the `helios.eval` harness – the three trust pillars. **Follow-ups.** Why MCP over a hand-rolled tool interface? / Would you have used a managed semantic layer (LookML, MetricFlow) if one fit? / What would you buy next as the project grows?

Q48. Cost control: how do you keep an autonomous, multi-agent, LLM-plus-BigQuery system from getting expensive?

Ideal answer. Cost in this system has two axes “warehouse bytes and model tokens” and I bound both with hard gates, not vigilance. On bytes: a mandatory `dry_run` precedes every `run_query` (G3), the run must fit a fixed 5 GiB budget, marts are incremental and partitioned/clustered so queries scan partitions not full history, and over-budget means *narrow the query*, not retry. On tokens: I tier models (Opus only for the three reasoning-heavy agents “Orchestrator, Diagnose, Critic” Sonnet for the mechanical nodes), the best-first search is depth/breadth-capped (`MAX_DEPTH=3` , `MAX_BRANCHING=4`) so it doesn't explore a combinatorial blowup, and the typed-envelope handoffs keep context small rather than passing rows. The byte budget is also a CI gate, so a change that makes runs scan more fails the build. The principle: make cost a *budget the system enforces on itself*, not a bill you discover later. **Why Helios demonstrates it.** The `dry_run` -> budget gate in `warehouse-mcp` , incremental partitioned marts, per-agent model tiering (Opus/Sonnet), and capped best-first search; the 5 GiB budget is gated in CI. **Follow-ups.** What's your single biggest token cost and how would you cut it? / How does the `dry_run` gate behave when a legitimate query is genuinely large? / How would cost scale with tenants, and what's the per-run unit economics?

Q49. What would you change if you rebuilt Helios from scratch today?

Ideal answer. Three things, in priority order. First, the eval: 50 scenarios over one observational dataset is a harness, not a guarantee, so I'd build randomized scenario generation and add a second, structurally different dataset to test generalization “the 85% is honest but single-dataset. Second, identity: the dataset's `user_id` is almost always null so everything is cookie-grain (`user_pseudo_id`), and I'd design the identity layer to gracefully upgrade to true user-level metrics where a stitching key exists, rather than baking cookie-grain assumptions throughout. Third, the experiment loop: because the data is historical I can only do quasi-experimental readbacks (pre/post, diff-in-diff), so on a live deployment I'd wire real A/B integration so Prescribe's cards close the loop with actual experiments, not just designs. None of these are bugs “they're the honest edges of an observational-data portfolio project, and naming them is the point. **Why Helios demonstrates it.** These map to stated limitations “single-dataset eval, cookie-grain identity (null `user_id`), and observational-only readbacks “which Helios surfaces rather than hides. **Follow-ups.** Of those three, which most threatens the 85% claim? / What would you NOT change because it's already right? / How would real A/B integration change the Critic's job?

Q50. Tie it together: how do the three defensibility pillars and the eval/Critic/memory components make this *trustworthy*, not just impressive?

Ideal answer. Trust is the product, and it's the sum of three pillars each backed by a concrete component. Pillar one “governed-SQL grounding: `semantic-mcp` is the only SQL path, so every column is provably real and the hallucination gate is a hard zero. Pillar two “deterministic statistics: `stats-mcp` is the only math path, seeded, so the decomposition that separates mix from rate is reproducible and the brief's numbers are verbatim tool outputs. Pillar three “adversarial verification plus offline eval: the Critic tries to refute every finding via named attacks, and the labeled benchmark grades the whole pipeline at 85% vs a 45% baseline, in CI, on every commit. Memory makes it accountable over time “suppression stops crying wolf, the action-tracking loop checks whether fixes worked. The synthesis: dashboards tell you *what* and ask you to trust them; Helios tells you *why*, prices it in dollars, prescribes a powered experiment “and then *proves* each step is correct. The hard part was never generating insight; it was making insight you can put your name on. **Why Helios demonstrates it.** The three pillars (`semantic-mcp` grounding, `stats-mcp` determinism, Critic + 85%-vs-45% eval) plus `helios_memory` (suppression + action-tracking) are the entire trust architecture “every impressive output is gated by a verification. **Follow-ups.** If you could keep only one pillar, which earns the most trust per unit of effort? / How would you explain the trust story to a skeptical VP in two minutes? / Where is the weakest link in the trust chain today?

Section 8: System Design Interview

This section treats Helios as a system to be defended on a whiteboard. A system-design interviewer does not want product trivia; they want to know whether you reason about scale, failure, observability, latency, and correctness as first-class concerns “and whether your design choices have *reasons*. For each question I state the general principle first, then show how a specific Helios component embodies it. The recurring theme: Helios is a *governed, deterministic control plane around a non-deterministic model*, and almost every system-design property falls out of that one decision.

Scalability

Q1. How does Helios scale from one funnel on one dataset to many tenants and many warehouses?

Ideal response. The general principle is to push variability behind a stable interface so the expensive parts (compute, governance, statistics) stay shared and only the *configuration* multiplies. A naive analytics tool hard-codes its SQL to a schema; scaling it to a new tenant means a rewrite. I designed Helios so the only thing that knows SQL dialect or schema is `warehouse-mcp`, and the only thing that knows a tenant's metrics is its `semantic_layer.yaml` registry. To

add a tenant I add a registry + a byte budget + a credential; the seven agents, the FSM, and `stats-mcp` are unchanged because they operate on the typed "Finding" envelope, not on rows. To add a warehouse (Snowflake, Databricks, DuckDB) I add an adapter behind `warehouse-mcp`; because `semantic-mcp` is the sole SQL author, dialect differences are localized to one server and validated by the same `reconcile -to-<=0.5%` test on both engines. The failure mode this avoids is the classic "every customer is a fork." The tradeoff I accepted: a governed semantic layer is more upfront work than letting the model free-text SQL, but it is the thing that makes horizontal scale *and* trust possible at once.

Why Helios demonstrates it. Multi-tenancy = per-tenant `semantic_layer.yaml` (47 metrics + 19 dimensions today) + per-tenant byte budget; warehouse-agnosticism = adapters behind `warehouse-mcp` with `semantic-mcp` as the single SQL author.

Q2. The dimensional search space is combinatorial â€” 16 dimensions, many values each. How do you keep that from exploding?

Ideal response. Combinatorial search must be bounded by *value-of-information*, not enumerated. The general move is best-first search with hard depth/breadth caps and a pruning predicate. Diagnose treats RCA as best-first search over the dimensional space, always expanding the slice with the largest *rate* effect (genuine behavior change) before chasing *mix* effects (composition artifacts), and prunes any branch below `MIN_RATE_EFFECT`, with `significance_test p > 0.05`, or below the minimum sample. Search is breadth-bounded (`MAX_BRANCHING = 4`) and depth-bounded (`MAX_DEPTH = 3` dimensions). This is what keeps a 16-dimension space tractable in <5 min without exhaustively cross-tabulating millions of cells. The alternative â€” full cube materialization â€” would blow both the byte budget and the latency SLA and is exactly what a junior implementation does.

Why Helios demonstrates it. The Diagnose hypothesis-tree with `MAX_BRANCHING / MAX_DEPTH` caps and rate-effect-ordered expansion.

Q3. How do the MCP servers themselves scale?

Ideal response. Scale-out is easy when components are *stateless and idempotent*; it is a nightmare when they carry session state. I made `semantic-mcp`, `stats-mcp`, and `experiment-mcp` pure functions: same inputs -> same outputs, no internal state, so they can be replicated horizontally behind a pool and a crashed worker loses nothing. `warehouse-mcp` is the only stateful boundary (it holds the BigQuery client and the byte-budget counter), and even there the state of record is the `run_state / audit_log` tables in `helios_memory`, not process memory â€” so the server is effectively restartable. All cross-run learning lives in BigQuery + the vector store, never in an agent's heap. The failure mode avoided: "the analyst's laptop had the context."

Determinism in `stats-mcp` (seeded scipy/statsmodels) also means horizontal replicas never disagree.

Why Helios demonstrates it. Stateless idempotent stats/semantic/experiment servers; durable state confined to `helios_memory` (BigQuery + vector store).

Reliability

Q4. What happens when a single agent or tool call fails mid-run? Do you ship a half-finished brief?

Ideal response. Reliability for an autonomous system means *fail closed, isolate the blast radius, and leave an audit trail* – never ship a partial result silently. The deterministic FSM is the backbone: the Orchestrator drives a finite-state machine (PLAN -> MONITOR -> DECOMPOSE -> DIAGNOSE -> CRITIC -> PRESCRIBE -> NARRATE -> END) and worker output never ships directly. Transient `warehouse-mcp` / BigQuery errors retry with exponential backoff (3 attempts). A single failed Diagnose branch is logged and pruned, not fatal. But if Monitor or Decompose fail entirely, the Orchestrator *aborts* and writes an audit record – no partial brief ships. A clean (no-anomaly) run is not silence either: Narrator emits a "no material change" brief and still writes a `run_state` row, so absence-of-finding is itself recorded. The principle is that an autonomous system that runs unattended must make its non-actions auditable, or operators stop trusting it.

Why Helios demonstrates it. Orchestrator FSM with abort-on-upstream-failure, bounded retries, branch-level failure isolation, and a `run_state` / `audit_log` row for every run including clean ones.

Q5. The LLM is non-deterministic. How is this system reliable at all?

Ideal response. The trick is to make the model a *composer* inside a deterministic shell, never the control plane. The FSM decides flow; the LLM only chooses which governed tool calls to emit at a node. All math is deterministic Python in `stats-mcp` (seeded), so decomposition, significance, power, and forecast are reproducible byte-for-byte. All SQL is emitted by `semantic-mcp` from a governed registry, so two runs over the same data reference the same columns. The model's variance is therefore confined to *which hypotheses it explores*, and the Critic + significance gates + reconcile-to- $\leq 0.5\%$ absorb that variance before anything ships. This is the difference between "the AI said so" and a system whose every number is a tool output verbatim and whose every run is reconstructable from `audit_log`.

Why Helios demonstrates it. Deterministic FSM + deterministic `stats-mcp` + governed `semantic-mcp` confine non-determinism to hypothesis selection, which the Critic then filters.

Q6. How do you stop the system from re-alarms on the same known issue every single run?

Ideal response. An always-on diagnostic that cries wolf weekly gets muted, so reliability includes *not re-nagging*. Helios persists a suppression list in `helios_memory`: when a stakeholder acknowledges a finding (or the Critic auto-suppresses), a row is written with a `reason` and a TTL (default 30d, so a re-emerging issue eventually re-surfaces). Diagnose and Critic both consult it before promoting a leaf; a match is auto-DROPPed with reason "suppressed." Known seasonality and launches live in `seasonality_calendar / launch_calendar`, which the Critic uses to refute "this is just the post-holiday dip." This is the operational maturity that separates a demo from something a team would actually leave running.

Why Helios demonstrates it. The memory suppression list (TTL'd) + seasonality/launch calendars consulted by Diagnose and Critic.

Observability

Q7. A run produces a wrong diagnosis. How do you debug it after the fact?

Ideal response. Observability for an autonomous reasoning system means *full reconstructability*: you must be able to replay exactly what the system saw and did. Every MCP call from any agent appends an `audit_log` row â€” `run_id`, `step_seq`, `agent`, `mcp_tool`, `args_hash` (sha256 of normalized args), the actual governed `sql_text`, `bytes_scanned`, `latency_ms`, and Critic `verdict`. The control plane wraps all tool invocations, so this is not best-effort logging â€” it is structurally complete. Combined with the pinned dbt model SHA and the semantic-registry hash, a run is fully reproducible: I can see every hypothesis explored, every query run, why each finding PASSed/DOWNGRADEd/DROPPed, and replay it. The failure mode avoided is the unobservable LLM agent that you can only "re-prompt and hope."

Why Helios demonstrates it. The `audit_log` table (per-tool-call, with `sql_text` + `args_hash` + `verdict`) and pinned model/registry hashes.

Q8. How do you verify the "0 hallucinated columns / 100% governed SQL" claim in production, not just in the eval?

Ideal response. A claim you cannot continuously verify is marketing. Because `audit_log.sql_text` records the actual SQL run and every query must have come through `semantic-mcp`, I can run an AST check over `sql_text` against the registry + GA4 schema and prove provenance â€” any column not in the registry is a hard alarm. The eval harness runs this same `hallucination.py` scorer as a *hard zero gate* in CI; in production the same check runs over the audit log. The principle: the same invariant should be enforced offline (CI gate) and observed

online (audit-log scan) with the same code, so there is no gap between what you test and what you ship.

Why Helios demonstrates it. Shared hallucination AST-check run as a CI hard-zero gate and as a production audit-log scan, both sourced from governed `sql_text`.

Q9. What operational signals would you put on a dashboard for Helios itself?

Ideal response. Observe the things that protect the SLAs and the trust pillars: per-run `bytes_scanned` vs the 5 GiB byte budget, per-run wall-clock vs the <5 min target, per-agent `latency_ms` and token cost (the Opus stages are the cost drivers), Critic verdict distribution (a spike in DROPs signals a Monitor/Decompose regression upstream), DOWNGRADE-to-PASS conversion after re-query, and the longitudinal eval scoreboard (top-1 accuracy, decomposition MAPE, hallucination rate). The `run_state` table already aggregates cost, timing, and finding counts per run; the rest are rollups of `audit_log`. Meta-point: I instrument the *trust* metrics, not just the infra metrics, because for this product correctness is the SLA.

Why Helios demonstrates it. `run_state` cost/timing/finding rollups + `audit_log` latency/verdict aggregations + the archived eval history table.

Latency

Q10. How do you hit <5 minutes per run when BigQuery scans are slow and the model is in the loop?

Ideal response. Latency and cost are the same lever here — bytes scanned. The general principle is to estimate before you spend and to bound the search. Every query is `dry_run` -cost-checked *before* `run_query`; if the estimated `bytes_scanned` would exceed the per-run byte budget the call is rejected and the agent must narrow the window or dimensions rather than blindly retry (rule G3). dbt marts are partitioned by `event_date` and clustered by `device_category`, `channel_group`, so partition/cluster pruning keeps scans small, and `fct_daily_funnel` is pre-aggregated so the hot path reads a day-grain table, not raw events. The hypothesis tree's `MAX_DEPTH` / `MAX_BRANCHING` caps bound how many queries a run can issue. Net effect: a representative full benchmark run scans ~2.1 GB and completes in ~4m12s, inside both the 5 GiB budget and the 5-minute SLA.

Why Helios demonstrates it. Mandatory `dry_run` byte gate (G3), partitioned/clustered marts, pre-aggregated `fct_daily_funnel`, and bounded hypothesis search.

Q11. The model itself adds latency. How do you keep token-time down without hurting accuracy?

Ideal response. Spend the expensive model only where the cost-of-error is high. I reserve Opus for the three reasoning-critical roles “ Orchestrator (planning/budget/routing), Diagnose (combinatorial hypothesis-tree search), and Critic (adversarial refutation that must out-think Diagnose) ” and use Sonnet for the bounded, mechanical stages: Monitor, Decompose, Prescribe, Narrator, where the action space is small (which series to test, which decomposition to run, which template to fill). I also compact context aggressively: each worker gets the run plan, the canonical catalog, and the upstream stage's *typed JSON* output “ never row-level dumps. Result sets are summarized to aggregates before entering the model context, so token usage stays flat even as the dimensional search widens. The tradeoff: an all-Opus design is marginally more capable but breaks both the latency and the cost budget.

Why Helios demonstrates it. The Opus/Sonnet split by cost-of-error + compacted typed-JSON hand-offs that keep row data out of context.

Evaluation

Q12. How do you know any of this is correct, and how do you stop a regression from shipping?

Ideal response. Correctness for a system that emits causal claims must be *measured against ground truth*, continuously. The live GA4 export has real anomalies but they are unlabeled, so I built an offline labeled benchmark: synthetic anomalies are injected into a frozen copy of the data “ a rate perturbation changes only `r_i`, a volume perturbation changes only `w_i` ” and the pipeline must rediscover what was hidden, graded against the ground truth recorded at injection time. 50 labeled scenarios span 7 buckets (single/multi rate, single/multi mix, seasonality decoys, no-anomaly controls, data-quality artifacts). Headline contract: root-cause segment top-1 accuracy $\geq 85\%$ (vs $\leq 45\%$ for a naive "largest absolute segment delta" baseline), decomposition MAPE $\leq 10\%$, and 0 hallucinated columns. This runs in GitHub Actions as a *required* check; CI fails the PR if any gate is breached or top-1 regresses more than the 2pt tolerance against the committed `main` baseline.

Why Helios demonstrates it. The offline injection benchmark (50 scenarios, perturbation primitives) + the CI gate in `eval/gates.yaml` with a regression tolerance.

Q13. The benchmark is synthetic. Doesn't that make the 85% number meaningless on real data?

Ideal response. Synthetic-injection is the *only* way to get labeled ground truth here, and I am explicit about its limits “ that honesty is the point. The mechanism is principled: perturbations are applied to aggregates with conserved totals, so the analytic mix/rate/interaction split is *exact* and serves as the gold target for MAPE. The seasonality-decoy and data-quality buckets

specifically test the failure modes that synthetic-only tests usually miss – the system must NOT flag a real seasonal swing and must catch a NULL spike or duplicated transaction_id as data quality, not behavior. So the benchmark stresses both detection and *refutation*. The honest caveat I state in interviews: this validates the decomposition-and-RCA machinery, not external causal validity, which is why Prescribe only *designs and sizes* experiments (the dataset is observational) and runs quasi-experimental readbacks rather than claiming live A/B causality.

Why Helios demonstrates it. Conserved-total perturbations (exact gold targets) + decoy/data-quality buckets + the stated observational-data limitation feeding quasi-experimental readbacks.

Q14. How do you keep the eval itself within budget – won't running 50 scenarios blow the cost ceiling?

Ideal response. The eval is a first-class consumer of the same cost controls as production. The harness runs every scenario through `warehouse-mcp.dry_run` first and aborts if total scanned bytes would exceed the per-run budget. To keep PR feedback fast and cheap, a smoke subset runs on every push and the full 50 run only on PRs to `main`. The harness pins random seeds, freezes the dbt model SHA, and records the registry hash, so a run is reproducible and the cost is stable across reruns. The principle: your test infrastructure must respect the same SLOs as production, or it becomes the thing that's too expensive to run – and an eval you stop running is no eval at all.

Why Helios demonstrates it. `dry_run` -gated eval, push-time smoke subset vs PR-time full run, and seed/SHA/registry pinning for reproducible, bounded-cost runs.

Driving the Helios system-design whiteboard

A short live-sketch playbook. Draw it in this order – each layer answers a different interviewer concern, and the order *is* the argument.

- 1. Start from the data flow (left to right).** GA4 events -> dbt staging (`stg_ga4__*`) -> intermediate (`int_ga4_sessionized` , `int_ga4_funnel_steps` with monotonic `reached_*` flags) -> marts (`fct_funnel` , `fct_daily_funnel` , `fct_orders` , `dim_*`). Say out loud: "session = (user_pseudo_id, ga_session_id); the funnel is `session_start` → `view_item` → `add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase` ." This shows you can model data before you orchestrate agents.
- 2. Draw the grounding boundary (the trust box).** Box the five MCP servers and label the two non-negotiable edges: `semantic-mcp` is the *only* path to SQL, `stats-mcp` is the *only* path to math. Put `warehouse-mcp` as the sole BigQuery client with the `dry_run` →

`run_query` byte gate and `reconcile` -to-<=0.5%. This is where you earn "0 hallucinated columns" â€” say it explicitly: the model physically never holds a raw-SQL tool.

3. **Draw the FSM on top of the boundary.** The seven agents around a deterministic finite-state machine: Orchestrator (Opus) -> Monitor (Sonnet) -> Decompose (Sonnet) -> Diagnose (Opus) -> **Critic loop** (Opus, PASS/DOWNGRADE/DROP) -> Prescribe (Sonnet) -> Narrator (Sonnet). Draw the Critic as a *gate*, not a step. Emphasize: the LLM composes tool calls, the FSM controls flow. Drop in the decomposition identity `deltaR = mix + rate + interaction` as the centerpiece of the Decompose/Diagnose stage.

4. **Close with the eval loop and memory.** Off to the side, draw the offline benchmark feeding the CI gate (85% vs 45%, hard-zero hallucination, 2pt regression tolerance) and `helios_memory` (diagnosis history, suppression, calendars, action-tracking, `audit_log`) as the state plane the FSM reads/writes. End on the one-liner: "the hard part isn't generating insights â€” it's making them correct and trusted, and every box on this board exists to do exactly that."

Section 9: Tough Critic Questions

A skeptical interviewer is testing temperament as much as content: do you get defensive, or do you concede the valid part and then win on a specific design decision and a number? Every answer below does both. The meta-rule: never argue that the skepticism is wrong; argue that you *already designed for it*.

Q1. Why not just use dashboards?

The challenge. "Every company already has Looker or Tableau. Why build this?"

Ideal answer. Fair â€” dashboards are essential and Helios doesn't replace them. But a dashboard answers *what* happened; it cannot tell you *why*, *how many dollars* it costs, or *what to do*. When `session_conversion_rate` falls from 2.1% to 1.7%, the dashboard shows the line going down and then a human spends 1-3 analyst-days doing root-cause analysis by hand â€” and routinely confuses mix-shift with rate-change (Simpson's paradox). Helios is the *autonomous diagnostic layer that sits on top of the dashboard*: it runs `decompose_change` to separate composition change from behavior change, prices the movement in dollars, and ships a Decision Brief before anyone asks. The category isn't "BI"; it's autonomous growth diagnosis. A dashboard is a question generator; Helios is an answer generator that closes the insight-to-action gap from weeks to one review cycle.

Q2. Why not use ChatGPT (or a single LLM) directly on the data?

The challenge. "I could paste my schema into a chatbot and ask why conversion dropped. Why all this machinery?"

Ideal answer. You could, and you'd get a fluent, confident, frequently *wrong* answer " because a raw LLM hallucinates column names and computes statistics in token-space, which is not arithmetic. Helios structurally forbids both: the model never authors SQL (it composes governed metrics through `semantic-mcp`, yielding 0 hallucinated columns) and never does math in prose (decomposition, significance, power all run as deterministic Python in `stats-mcp`). A chatbot is also reactive " it answers one question at a time when prompted " whereas Helios runs on a schedule and proactively diagnoses before a human notices. The proof is the benchmark: the disciplined system hits 85% root-cause accuracy vs the 45% a naive aggregate read scores. The difference between those two numbers *is* the value of the machinery.

Q3. Why MCP? Isn't that just function-calling with extra steps?

The challenge. "MCP is a buzzword. You could do all this with plain tool/function calls."

Ideal answer. Mechanically, yes " MCP is a tool protocol. The reason it matters here is that it lets me make the trust boundary *structural* rather than *behavioral*. Each agent is granted only the MCP tools in its allow-list, so the Narrator, for instance, physically cannot call `run_query`; the model can't break a rule it has no tool to break. That's stronger than "the system prompt says don't write SQL," which an LLM will eventually ignore. MCP also gives me clean capability isolation " `semantic-mcp` as sole SQL author, `stats-mcp` as sole math path " as stateless, independently testable, independently scalable servers. The win isn't the protocol; it's that capability-as-tool-boundary turns a policy into an invariant I can enforce and audit.

Q4. Why multiple agents instead of one?

The challenge. "Seven agents sounds like resume-driven complexity. One good agent could do this."

Ideal answer. The decisive reason is adversarial separation: the Critic must be a *different* agent than Diagnose, because an agent grading its own work is not a check. The Critic (Opus) attempts to refute every finding " mix-shift confound, insufficient sample, seasonality, data quality " and only PASSing findings ship; that's worth ~40 points of accuracy on the benchmark. Beyond that, the split lets me match model to cost-of-error (Opus on Orchestrator/Diagnose/Critic, Sonnet on the bounded stages) so I hit the cost and latency budgets, and it gives clean failure isolation in the FSM. I'd push back on "complexity": each agent has one narrow responsibility and a typed JSON hand-off " that's *less* coupled than one monolithic prompt trying to plan, query, decompose, diagnose, refute, and write all at once.

Q5. Why not just write one big SQL query?

The challenge. "A senior analyst could answer this with one well-written query. Why an AI system?"

Ideal answer. A senior analyst absolutely could "for one anomaly they already suspect, after they've context-switched in. Two problems. First, the hard part of root-cause is *search*: you don't know which of 16 dimensions and their crossings holds the cause, so a single static query can't express the best-first hypothesis-tree walk that drills rate-effects before mix-effects. Second, it doesn't scale or persist: the query lives in someone's head, runs when they're free, and forgets what it found last week. Helios runs unprompted on a schedule, bounds the combinatorial search (`MAX_DEPTH=3` , `MAX_BRANCHING=4`), keeps every scan inside a 5 GiB budget via `dry_run` gating, and remembers via `helios_memory` . The SQL still gets written "by `semantic-mcp` , governed and reconciled " just not by hand and not once.

Q6. Isn't this over-engineered for a portfolio project?

The challenge. "This is a 3-month static public dataset. Why the semantic layer, the FSM, the eval harness, the memory store?"

Ideal answer. I'd concede that for a one-off chart, yes, it would be. But the project's thesis is precisely that *the hard part of AI analytics is not generating insights "it's making them correct and trusted "* and you cannot demonstrate that with a thin demo. Each piece earns its place against a specific failure: the semantic layer kills hallucinated columns (0%), the deterministic stats layer kills token-space math, the Critic + eval kill confident-but-wrong diagnoses (85% vs 45%), and memory kills weekly re-nagging. I also scoped honestly: it's single-tenant batch today, with multi-tenant/streaming explicitly deferred to a later phase. Over-engineering is complexity without a corresponding failure it prevents; every component here maps to a named failure mode and a number. That mapping is the senior signal.

Q7. How do you KNOW the diagnoses are right?

The challenge. "You assert 85%. On what? The model could be confidently wrong and you'd never know."

Ideal answer. That's exactly why I don't trust it on assertion "I measure it against ground truth I control. I inject synthetic anomalies into a frozen copy of the data where I *know* the true root cause (a rate perturbation moves only `r_i` , a volume perturbation only `w_i`), then grade whether the pipeline rediscovers the hidden segment and effect. Across 50 labeled scenarios, top-1 root-cause segment accuracy is $\geq 85\%$ and decomposition MAPE $\leq 10\%$, both gated in CI. On live data I can't have a label, so there I lean on the in-run checks: `reconcile` -to- $\leq 0.5\%$, `significance_test` gating, and the adversarial Critic. "Right" is decomposed into measurable-offline plus verified-in-run; I never ask anyone to trust the model's word.

Q8. What happens when the LLM is wrong?

The challenge. "Models fail. When yours produces a bad hypothesis, what stops it from shipping?"

Ideal answer. I designed assuming the model *will* be wrong, and confine the damage to hypothesis *selection* – the one place non-determinism lives. A bad hypothesis hits three filters before it can ship: `significance_test` ($p > 0.05$ prunes the branch), `reconcile` ($> 0.5\%$ drift fails the finding), and the Critic (refutes confound/sample/seasonality/data-quality with verdict DROP). A plausible-but-thin finding is DOWNGRADED and sent back to Diagnose for a targeted re-query, bounded by `MAX_REFUTE_ROUNDS=2`, after which it's demoted to a watchlist note rather than a confident finding. If an upstream stage fails outright, the FSM aborts and writes an audit row – it fails closed, never shipping a partial brief. And because every step is in `audit_log`, a wrong run is fully reconstructable for debugging.

Q9. Is 85% too low to trust?

The challenge. "85% means one in seven diagnoses is wrong. Why would I act on that?"

Ideal answer. Two honest framings. First, the comparison: the realistic human/naive baseline scores ~45% on the same benchmark – 85% is nearly a doubling, and the relevant bar is "better and faster than the status quo," which it clears decisively. Second, the *shape* of the error matters: top-3 accuracy is ~95%, so when the #1 segment is wrong the true cause is almost always in the shortlist the analyst reviews – and the Critic plus the dollar-at-risk ranking mean you're triaging the highest-value findings first, not acting blind on a single guess. Helios also doesn't claim final authority: it ships a *Decision Brief* a human reviews in <5 minutes with full evidence and an audit trail. So it's not "act on 85% blind"; it's "start every investigation 40 points ahead with the evidence already assembled." I'd take that trade every time.

Q10. Why should anyone trust the dollar figures?

The challenge. "Revenue-at-risk sounds like a number you made up to look impressive."

Ideal answer. It would be, if it were a model guess – so it isn't one. Revenue-at-risk is a deterministic computation from governed metrics: at a leaf finding it's (counterfactual rate at t_0 - observed rate at t_1) x affected `sessions` x `aov`, all canonical metrics pulled via `semantic-mcp` and reconciled to source within 0.5%. The LLM supplies no digits; the number is a `stats-mcp` /governed-SQL output verbatim, traceable through `audit_log` to the exact query. I also validate the *method* in the eval: the injector computes a true `dollar_at_risk_usd` from the conserved-total perturbation, and Helios's estimate is graded against it with a $\leq 15\%$ error target. So the dollar figure is auditable, reconciled, and benchmarked – three things a made-up

number is not. The honest caveat: it's a correlational revenue-at-risk on observational data, not a causal lift estimate, which is why Prescribe pairs it with a powered experiment to confirm.

Q11. What does a run cost, and does it actually scale?

The challenge. "BigQuery and Opus tokens aren't free. What's the unit economics, and what breaks at scale?"

Ideal answer. Per run, BigQuery cost is capped *structurally*: every query is `dry_run`-estimated before execution and rejected if it would exceed the 5 GiB byte budget, and a representative full run scans ~2.1 GB in ~4m12s — well inside budget and the <5 min SLA. Token cost is controlled by the Opus/Sonnet split (Opus only on the three reasoning-critical roles) and by compacting context to typed JSON aggregates so tokens stay flat as the search widens. On scale: the stats/semantic/experiment servers are stateless and idempotent, so they replicate horizontally; durable state is in BigQuery, not process memory; and adding a tenant is adding a registry + budget + credential, not a fork. What I'd watch as the real scaling cost is the Opus reasoning stages and per-tenant BigQuery scan — both observable in `run_state` / `audit_log`, both bounded by the byte budget and the search caps. The honest limit: it's single-tenant batch today; multi-tenant and streaming are a deliberately deferred phase, not hand-waved.

Q12. Why not just buy Amplitude or Mixpanel?

The challenge. "Product-analytics tools already do funnels and retention. Why not use one?"

Ideal answer. They're excellent and I'd happily sit Helios alongside one. But Amplitude and Mixpanel are still *descriptive*: they show the funnel and let a human pull a report — they answer *what* and some *who*, on human trigger. None of them autonomously decomposes a movement into mix-shift vs rate-change to dissolve Simpson's paradox, prices it in dollars, refutes it adversarially, and prescribes a powered experiment — unprompted, on a schedule, grounded in governed SQL with an offline accuracy benchmark. That diagnostic-and-prescriptive loop is the category Helios creates, and it's complementary: you could feed its briefs from a warehouse that also serves Mixpanel. The buy-vs-build framing also misses the portfolio point — the engineering thesis is the *trust architecture* (governed SQL + deterministic stats + adversarial eval), which is exactly the part a SaaS tool hides and an interviewer wants to see me reason about.

Section 10: Resume Bullets

Every bullet below describes the same project — Helios, my Autonomous Growth Diagnosis Engine on the Google Merchandise Store GA4 public dataset. The numbers are consistent across surfaces (85% vs 45% root-cause accuracy, <5 min/run, 0 hallucinated columns, 50 labeled

scenarios, 5 MCP servers, 7 agents, a 28-metric / 16-dimension semantic layer) so an interviewer who reads my resume, my LinkedIn, and my one-pager hears one coherent story.

5 resume bullets

These are tuned for ATS parsing: action-verb-led, one quantified outcome each, canonical keywords spelled out.

- **Architected** an autonomous **7-agent** AI Growth Analyst (Claude Agent SDK, Model Context Protocol) that diagnoses *why* an e-commerce funnel moved, achieving **85% root-cause accuracy vs 45%** for a naive largest-delta baseline on a **50-scenario** offline labeled benchmark.
- **Engineered** a mix-shift-vs-rate-change decomposition (mix / rate / interaction) over GA4 funnel metrics that resolves Simpson's paradox, computed deterministically and validated to **<=10% MAPE** against analytically-derived ground truth.
- **Built** a governed-SQL grounding layer across **5 MCP servers** with a **28-metric, 16-dimension** semantic registry as the sole SQL author, eliminating LLM hallucination to **0 hallucinated columns** while holding query cost under a fixed **5 GiB** BigQuery byte budget.
- **Designed** the dbt analytics layer (staging -> intermediate -> marts) on the GA4 BigQuery export with session-grain reconciliation tests (**<=0.5%** drift) and a **GitHub Actions CI** eval gate blocking any accuracy regression.
- **Delivered** time-to-diagnosis **<5 min/run**, with **100%** of findings carrying a significance test, a dollar revenue-at-risk figure, and a prioritized, power-analyzed experiment "each adversarially vetted by a Critic agent before release."

10 LinkedIn bullets

Slightly more narrative and first-person "these read like someone talking about work they're proud of, not a keyword block."

- I built Helios, an always-on "AI Growth Analyst" that runs on a schedule and proactively tells you *why* an e-commerce funnel moved "not another dashboard, and not a SQL chatbot."
- The technical heart is a mix-shift-vs-rate-change decomposition: I split every aggregate rate change into a traffic-composition effect and an in-segment-behavior effect, which is exactly what dissolves Simpson's paradox in real funnel data.
- I made the system *trustworthy*, not just clever: the LLM never writes raw SQL and never computes a statistic "it composes governed metrics through a semantic layer and calls deterministic Python (scipy / statsmodels / prophet) over 5 MCP servers."
- To prove it works I designed an offline labeled benchmark "50 scenarios across 7 buckets" by injecting *known* anomalies into a frozen GA4 copy and grading whether the pipeline rediscovers the segment I hid.

- The headline result: 85% root-cause-segment accuracy versus 45% for the obvious "largest absolute delta" baseline, with that gap regression-gated in CI on every pull request.
- I orchestrated 7 agents on a deterministic finite-state machine â€" Orchestrator, Monitor, Decompose, Diagnose, Prescribe, Narrator, and an adversarial Critic that tries to refute every finding before it ships.
- I used the right model for each job: Opus for the hard reasoning roles (orchestration, diagnosis, critique) and Sonnet for the high-volume worker steps, to balance quality against token cost.
- Every finding Helios emits is *actionable* by construction â€" it carries a significance test, dollars of revenue-at-risk, and a powered experiment card, never a naked assertion.
- I modeled the GA4 export end to end in dbt and handled the classic gotchas honestly: sessions keyed on `(user_pseudo_id, ga_session_id)`, and the `traffic_source` struct being user first-touch rather than session source.
- I was deliberate about limits: the dataset is ~3 months and observational, so Helios *designs and sizes* experiments and runs quasi-experimental readbacks (pre/post, difference-in-differences) rather than pretending to run live A/Bs.

Internship-discussion bullets

These connect my GA4 / GTM / funnel-tracking internship work directly to Helios â€" useful when an interviewer asks "so what did you actually do at your internship, and how does this project build on it?"

- In my internship I instrumented GA4 / GTM event tracking and built funnel reports; the recurring pain was that the dashboards showed *what* changed but the "why" still took an analyst a day or more â€" Helios is my answer to that exact gap.
- I'd lived the Simpson's-paradox trap manually â€" a funnel rate dropping while every segment held flat because mobile/paid traffic mix shifted â€" so in Helios I encoded that decomposition as a first-class, deterministic tool instead of a thing analysts re-discover by hand.
- I knew the GA4 BigQuery export's real edges from instrumenting it (event-scoped params, the `traffic_source` first-touch gotcha, no session row to query), so I modeled sessionization and channel grouping correctly from day one rather than learning it the hard way.
- The ecommerce funnel I tracked in GTM (session_start -> view_item -> add_to_cart -> begin_checkout -> purchase) is the same funnel Helios diagnoses â€" I turned a reporting workflow I knew intimately into an autonomous diagnosis engine.
- Measurement-discipline habits from the internship â€" reconciling event counts to source totals, distrusting attribution defaults â€" became Helios's verify-then-trust principles: dry-run cost gating, $\leq 0.5\%$ reconciliation, and an adversarial Critic.

Placement-season bullets

One-liners for a campus CV / one-pager, mapped to the specific role a given company is hiring for. Pick the variant that matches the JD.

- **Product Analyst:** Built an autonomous funnel-diagnosis engine on GA4 that separates traffic-mix shifts from behavior changes and quantifies revenue-at-risk in dollars (85% vs 45% root-cause accuracy on 50 labeled scenarios).
 - **Growth Analyst:** Turned conversion-rate drops into segment-attributed root causes and a prioritized, power-analyzed experiment backlog, shipped as an executive Decision Brief in under 5 minutes per run.
 - **Data Analyst:** Modeled the GA4 BigQuery export in dbt (staging -> marts) with a governed 28-metric / 16-dimension semantic layer and reconciliation tests holding drift $\leq 0.5\%$.
 - **Analytics Engineer:** Designed the dbt DAG and a MetricFlow-aligned semantic registry as the single source of truth for all SQL, with a CI gate enforcing 0 hallucinated columns and a fixed 5 GiB byte budget.
 - **AI / Data Engineer:** Orchestrated 7 agents over 5 MCP servers (Claude Agent SDK) with governed SQL and deterministic stats as enforceable trust boundaries, benchmarked at 85% accuracy with an adversarial Critic.
-

Section 11: STAR Stories

Seven behavioral stories, each in explicit Situation / Task / Action / Result form with a one-line signal of what it demonstrates. I keep them concrete – real decisions, named tradeoffs, honest limits.

Story 1 – Scoping a flagship from a vague brief (project ambiguity)

Situation. I wanted one flagship portfolio project for placement season, and the only fixed inputs were a public dataset (the GA4 obfuscated ecommerce sample) and my own GA4/funnel background. The brief was essentially "build something impressive with analytics + AI" – wide open, and easy to drift into a generic dashboard or yet another text-to-SQL chatbot.

Task. Convert that ambiguity into a sharp, defensible product thesis I could actually ship and explain in an interview.

Action. I started from the pain, not the tech: dashboards report *what* happened; the expensive, judgment-heavy work is *why*, and it takes an analyst 1-3 days per anomaly. I wrote an explicit anti-product stance – "not a dashboard, not a SQL chatbot, autonomous-first" – to keep myself honest, then scoped Helios down to a single load-bearing centerpiece (mix-shift vs rate-

change decomposition) and a five-phase maturity ladder (L1 intern MVP through L3 top-1% and two production-frontier phases) so each phase was independently demoable with green CI exit criteria.

Result. A crisp 60-second pitch and a buildable plan: an autonomous engine that diagnoses funnel movement, prices it in dollars, and prescribes experiments. The anti-product stance kept scope from sprawling, and the phase ladder meant I always had a shippable, demoable increment.

Signal: Turns an open-ended brief into a sharp thesis and a phased plan; defines what *not* to build.

Story 2 – The mix-vs-rate decomposition (a hard technical challenge)

Situation. Overall `session_conversion_rate` fell week-over-week. The obvious read – and the one a naive "largest absolute segment delta" heuristic produces – was "checkout is broken, go fix it."

Task. Find the *real* cause before prescribing a fix, and do it in a way I could defend mathematically rather than by intuition.

Action. I implemented `stats-mcp.decompose_change`, which treats an aggregate rate as $R = \sum_i (w_i * r_i)$ and splits `deltaR` into a mix effect $\sum_i (dw_i * r_i(t_0))$, a rate effect $\sum_i (w_i(t_0) * dr_i)$, and an interaction term $\sum_i (dw_i * dr_i)$. I enforced the Simpson's-paradox-safe discipline of computing rates as $\text{SUM}(\text{numerator})/\text{SUM}(\text{denominator})$ after grouping – never averaging per-segment ratios – and unit-tested the function against hand-worked golden values so the algebra was provably correct.

Result. The per-segment rates were essentially flat; the drop was almost entirely *mix* – a surge of low-converting mobile / Paid Social traffic had shifted the weights. The prescription flipped from "fix checkout" to "fix acquisition mix," and the answer was fully auditable. That decomposition became the technical heart of the whole project.

Signal: Goes past the obvious answer to the mathematically correct one; recognizes and resolves Simpson's paradox.

Story 3 – Governed SQL via MCP, deterministic stats (an architecture decision)

Situation. I wanted an LLM in the loop for reasoning, but LLMs do two things that are fatal in an analytics tool: they hallucinate column and metric names, and they "compute" statistics in token-space that look right and aren't.

Task. Eliminate both failure modes *structurally* â€” not by prompting the model to be careful, but by making the bad outcomes impossible.

Action. I split the system's capabilities across 5 MCP servers and drew hard boundaries:

`semantic-mcp` (a governed 28-metric / 16-dimension registry) is the *only* path that emits SQL, so the model composes metric names like `view_to_cart_rate` and never authors raw SQL; `stats-mcp` is the *only* path to math, so every decomposition, significance test, and forecast is real seeded Python. I considered the simpler alternative â€” let the model write SQL and validate it afterward â€” and rejected it: validation catches errors after the fact, whereas a registry makes a hallucinated column a hard error at composition time. `warehouse-mcp` holds the sole BigQuery client behind a mandatory dry-run byte gate.

Result. 0 hallucinated columns by construction, every number reproducible, and the architecture itself is the trust argument â€” capability isolation became enforceable trust boundaries rather than hopeful prompting.

Signal: Designs for correctness structurally; names the rejected alternative and why it's weaker.

Story 4 â€” The labeled benchmark (evaluation design)

Situation. "The AI says so" is not shippable, and the live GA4 data's real anomalies are *unlabeled* â€” I don't know their true root causes, so I can't grade against them.

Task. Make the system's trustworthiness *measurable* with a number I could put on a resume and defend.

Action. I built an offline labeled benchmark by injecting *known* anomalies into a frozen copy of the data. A seeded Python injector applies one of two surgical primitives â€” a rate perturbation (change `r_i` only, holding segment volume `w_i` fixed) or a volume/mix perturbation (change `w_i` only, holding rates fixed) â€” and writes a ground-truth label, including the analytically-exact mix/rate/interaction split and true dollar-at-risk. I built 50 scenarios across 7 buckets (single/multi rate, single/multi mix, seasonality decoys, no-anomaly controls, data-quality artifacts) and â€” crucially â€” I implemented the *naive baseline* too (largest absolute segment delta) so my headline number had an honest comparison point. The whole harness runs in GitHub Actions and gates every PR.

Result. 85% root-cause-segment accuracy vs 45% for the naive baseline, decomposition MAPE $\leq 10\%$, 0 hallucinations â€” with regressions caught automatically in CI. The no-anomaly controls and seasonality decoys also let me measure false positives, not just hits.

Signal: Knows that a metric without a baseline is meaningless; builds the adversary and the controls, not just the happy path.

Story 5 “ Learning MCP, the Agent SDK, and dbt fast (learning new tech)

Situation. Helios required three technologies I hadn't shipped before: the Model Context Protocol, the Claude Agent SDK for multi-agent orchestration, and dbt for the warehouse layer. My prior experience was GA4/GTM analytics, not agent systems or analytics engineering at this depth.

Task. Get productive in all three quickly enough to build a coherent system, without producing a fragile pile of half-understood glue.

Action. I learned each by anchoring it to a concept I already understood. MCP I treated as a typed RPC boundary “ which let me reframe it as the *trust boundary* of the whole design rather than just plumbing. The Agent SDK I constrained deliberately: I ran the 7 agents on a deterministic finite-state machine where the LLM only ever *composes tool calls* and never controls flow, which made the system debuggable instead of an emergent mess. dbt I learned through its conventions “ staging/intermediate/marts layering, tests-as-first-class “ and front-loaded the two keystones (sessionization and the semantic registry) because they fail *silently* if wrong.

Result. A working end-to-end system across all three, and “ more importantly “ opinions about each: why a constrained FSM beats free-form agent autonomy here, and why the MCP boundary is the feature, not the framework. I learned them well enough to defend the design choices, not just operate the tools.

Signal: Learns unfamiliar tech by mapping it to known concepts; ends up with defensible opinions, not just working code.

Story 6 “ Catching my own attribution mistake (handling feedback / self-correction)

Situation. Early on, my channel attribution looked subtly wrong, and separately my interview-prep docs had drifted “ different sections cited different scenario counts and slightly different metric framings as the spec evolved.

Task. Find and fix the root cause of the attribution error, and reconcile the documentation so the project told one consistent story.

Action. On attribution, I discovered I had trusted the event-level `traffic_source` struct as the session's source “ but in GA4 it's the user's *first-touch* attribution, not session source. I rebuilt `channel_group` on session-scoped `event_params` source/medium with first-touch only as a documented fallback, and I centralized the logic in a single `channel_group` macro so it could never diverge again. On the docs, I treated a Canonical Reference Card as the single source of

truth and reconciled every cross-section inconsistency back to it (scenario counts, metric names, the 10 channel groups) rather than patching sections independently.

Result. Correct GA4-style channel grouping, and a documentation set that no longer contradicts itself – one canonical vocabulary, enforced. The fix also became one of my strongest "honest gotcha" talking points in interviews.

Signal: Owns and roots-out their own error; fixes the *class* of bug (single source of truth) rather than the instance.

Story 7 – Deferring LTV under data and budget constraints (prioritization under constraints)

Situation. Two hard constraints bounded what Helios could honestly claim: the dataset is only ~3 months (Nov 2020 - Jan 2021) and observational, and I held myself to a fixed 5 GiB BigQuery byte budget per run with a <5 min/run latency target.

Task. Decide what to build now, what to defer, and – most importantly – how to be honest about it rather than overclaiming.

Action. I explicitly deferred true customer LTV and long-horizon retention: a 3-month window cannot support lifetime curves, so I report short-horizon proxy retention with stated assumptions instead of inventing a number. Because there's no experiment assignment in the data, I scoped Helios to *design and size* experiments and run quasi-experimental readbacks (pre/post, difference-in-differences) rather than claiming live A/B testing. For cost, I made `dry_run` byte-estimation mandatory before every `run_query`, pruned date shards, and pre-aggregated the daily funnel fact so runs stayed under budget. I wrote these deferrals into the roadmap with explicit rationale.

Result. A system that's both performant (<5 min/run, under the 5 GiB budget) and *credible* – every limitation is stated with its reason. In interviews, naming what I deliberately didn't do, and why, reads as senior judgment rather than a gap.

Signal: Prioritizes ruthlessly under real constraints and states limitations as deliberate, reasoned tradeoffs – maturity, not weakness.

Section 12: Founder Round

This section is the one where the resume and the architecture stop mattering and a single question takes over: *would I bet a company on this?* A founder is not grading my dbt DAG; they are pressure-testing whether I understand who pays, why, what kills the thing, and whether I can

think in dollars and not just in p-values. I answer in the same voice I built Helios in â€” quantify, name the failure mode, state the alternative, be honest about the risk.

Business

Who actually pays for this, and why?

Ideal answer. The buyer is a growth or analytics leader at a company doing enough e-commerce or product volume that a one-point conversion swing is a real number â€” say a Head of Growth, VP Analytics, or a founder still wearing the data hat at a Series Aâ€”C company. They pay because root-cause analysis is the single most expensive recurring task on an analyst's plate: in my experience and in the literature it runs roughly **one to three analyst-days per anomaly**, and most teams see several genuine anomalies a month. The thing they are buying is not "insights" â€” dashboards already flood them with insights. They are buying *collapsed time-to-answer* and *defensible answers*: Helios turns a multi-day manual dig into a sub-five-minute autonomous run that arrives with a decomposition, a significance test, and a dollar number. **Why Helios demonstrates it:** the product's heartbeat is the scheduled autonomous run, not a chat box â€” the value proposition is literally "the analysis is already done when you open it."

What would you charge, and on what model?

Ideal answer. I'd anchor to the value, not the compute. One avoided misdiagnosis â€” telling a team to "fix checkout" when the real issue was acquisition mix â€” burns weeks of engineering and a quarter of misallocated spend, easily five figures. So I'd price as a per-tenant SaaS seat tied to data volume and run frequency, something like **2kâ€”8k/month** for a mid-market growth team, with the framing "less than a fifth of one analyst's loaded cost, and it never sleeps." The honest constraint is my marginal cost: every run consumes BigQuery bytes and Opus/Sonnet tokens, so I'd gate runs under the fixed **5 GiB byte budget** per run and meter token spend, which is exactly why the architecture splits Opus (Orchestrator, Diagnose, Critic) from Sonnet (the high-volume worker agents). **Why Helios demonstrates it:** the `dry_run`-before-`run_query` byte gate isn't just a correctness control, it's the unit-economics control that makes a per-run price sane.

How big is the market â€” TAM?

Ideal answer. I'd size it bottom-up, not with a top-down "digital analytics is a \$X0B market" hand-wave that founders rightly distrust. The serviceable wedge is companies that already run GA4 (or a warehouse-native equivalent) on BigQuery and employ at least one analyst â€” that's hundreds of thousands of mid-market and enterprise e-commerce and product orgs globally. If even a fraction carry an analytics tooling budget of a few thousand a month, the reachable market is comfortably in the **low billions of ARR**. But I'd be candid that TAM is a fundraising slide; what

matters at the seed stage is whether *ten* design-partner teams will pay, and I'd rather over-index on that proof than on a big number.

Growth

What's the wedge, and what's the moat?

Ideal answer. The **wedge** is the single most painful, most repeated, least-automated workflow: "conversion dropped â€" why?" It's narrow, urgent, and lands inside a team that already has the data. I don't try to replace the BI stack; I sit on top of it and answer the one question dashboards can't. The **moat** is *governed trust*, and it has three reinforcing layers that compound over time: (1) the semantic layer means SQL is composed from a governed registry, so there are **zero hallucinated columns** â€" a generic LLM-on-your-warehouse competitor cannot promise that; (2) the **offline labeled benchmark** (85% root-cause accuracy vs 45% naive baseline, gated in CI) is a defensible, auditable trust claim that takes real effort to build; and (3) the **memory store** â€" diagnosis history, suppression list, action-tracking â€" gets more valuable per tenant the longer Helios runs, because it learns that tenant's seasonality and which fixes actually worked. **Why Helios demonstrates it:** the three defensibility pillars (governed-SQL grounding, deterministic stats, adversarial Critic + eval) are not just engineering hygiene â€" they are the competitive wall.

Build vs. buy vs. just use the incumbent â€" why doesn't GA4/Looker/an in-house script win?

Ideal answer. The incumbents solve a *different* problem and that's the opening. GA4 and Looker are reporting surfaces â€" they tell you *what* happened with great fidelity and say nothing about *why*; the analyst still does the root-cause dig by hand. An in-house "ask-your-data" LLM script is the dangerous middle option: it's cheap to start and silently wrong, because it hallucinates columns and computes statistics in token-space. My whole architecture is a structural answer to that failure: the LLM **never authors SQL** (semantic-mcp is the only path) and **never computes a statistic** (stats-mcp is the only path). So the build-vs-buy pitch is "you could wire GPT to your warehouse in a weekend, and it would confidently misdiagnose your funnel â€" the hard part isn't generating an answer, it's making it *correct and trusted*, and that's a quarter of engineering you don't want to own." **Why Helios demonstrates it:** the five-MCP trust boundary is precisely the part a customer cannot cheaply rebuild.

ROI

What is the ROI of a single Decision Brief?

Ideal answer. I'd make it concrete with the canonical story. Aggregate `session_conversion_rate` falls week-over-week; the obvious read is "checkout is broken," and a

team acts on it â€” eng sprints on the checkout flow, PM reprioritizes. Helios runs `decompose_change` and shows the per-segment rates are *flat*: the drop is almost entirely **mix** â€” a surge of low-converting Paid Social mobile traffic shifted the weights. The prescription flips from "fix checkout" (weeks of wasted eng, zero lift) to "fix acquisition mix." The ROI of that one brief is the **misdiagnosis cost avoided**: a wasted eng sprint plus a quarter of mis-aimed ad spend, against a brief that costs a few dollars of BigQuery and tokens. Even one such save a quarter pays for the product many times over. And the brief carries a **dollar revenue-at-risk** number natively, so the buyer doesn't have to construct the ROI â€” Helios hands it to them. **Why Helios demonstrates it**: revenue-at-risk ($\hat{\text{rate}} \times \text{affected sessions} \times \text{downstream value}$) is a first-class field on every finding, not an afterthought.

How does the value change as the company scales?

Ideal answer. It compounds in three directions. First, **data volume**: more segments and more traffic mean more places for Simpson's paradox to hide, so the mix-vs-rate decomposition gets *more* valuable, not less â€” exactly when manual RCA gets harder. Second, **organizational distance**: at scale the person who can diagnose a funnel and the person who owns the budget are different people in different time zones; the autonomous scheduled brief bridges that gap without a meeting. Third, **memory**: the action-tracking loop ("did the fix work?") and seasonality calendars mean the system's accuracy and relevance climb with tenure on the account, which lowers churn. The honest counter-pressure is cost â€” more data is more bytes â€” which is why warehouse-agnostic adapters and per-tenant byte budgets are on the roadmap rather than pretending scale is free. **Why Helios demonstrates it**: the memory store and the deterministic decomposition are the two components whose value is explicitly super-linear in scale.

Product Strategy

What would you build next?

Ideal answer. I sequence by trust-then-reach. The immediate next build is **closing the action-tracking loop** â€” automatically reading back whether a prescribed experiment shipped and whether the fix moved the metric, via quasi-experimental pre/post and difference-in-differences. That turns Helios from "diagnoser" into "accountable diagnoser," which is the retention story. After that, **warehouse-agnostic adapters** behind warehouse-mcp (Snowflake, Databricks, DuckDB) so the semantic layer stays the only SQL author regardless of dialect â€” that's the reach unlock beyond GA4-on-BigQuery. I would *not* prematurely build a chat UI or a dashboard; that's drifting into the anti-product. **Why Helios demonstrates it**: the roadmap is already staged P0â€™P4 with the principle "never start a phase until the prior phase's exit criteria are green in CI," which is the same trust-first discipline.

How do you know customers actually want this?

Ideal answer. I'd be honest that I can't fully prove demand from a portfolio project on a public dataset – that's a limitation, not a strength to oversell. What I *can* point to is the strength of the underlying pain signal: every analytics team I've worked with or studied spends real days on manual RCA and routinely confuses mix-shift with rate-change. The validation plan is design partners: get five-to-ten growth teams to run Helios on their own GA4 export for a month and measure whether the briefs change a real decision. The kill-criterion is explicit – if leaders read the brief and still send it to an analyst to "double-check," I haven't earned trust and the value prop fails. **Why Helios demonstrates it:** the offline benchmark is my honest proxy for "is it right" before I have real users; demand validation is the next experiment, designed the same disciplined way.

What kills this product?

Ideal answer. Three things, in order of seriousness. (1) **Trust collapse** – if Helios confidently ships one wrong root cause to an executive, the relationship is over; that single risk is why the entire architecture is built around verify-then-trust, the adversarial Critic, and the 85%/45% benchmark gate. (2) **The "good-enough" incumbent** – if GA4 or Looker bolts on a credible native explanation feature, my wedge narrows; my defense is depth (real decomposition, governed stats, the accountability loop) and warehouse-agnosticism. (3) **Unit economics** – if per-run BigQuery and token cost outpaces what a tenant will pay, the model breaks; the byte budget and the Opus/Sonnet split are the live mitigations, but it's a real constraint I watch. The mature read is that the existential risk is almost entirely #1 – and it's the one I've invested the most architecture in defending against. **Why Helios demonstrates it:** "the hard part is not generating insights – it is making them correct and trusted" is the thesis, so the thing that kills it is the thing the thesis is built to prevent.

Section 13: Top 1% Interview Prep

This is the section that separates a candidate who *built* the project from one who can *defend* it under adversarial questioning. The structure is deliberate: master a small set of concepts cold, know the boundary of what I can hand-wave, pre-arm the weak spots, disarm the traps, and walk in with a tight rehearsal plan.

Concepts you MUST understand deeply

These are non-negotiable. If I fumble any of them, the project's credibility falls apart.

- **Mix-vs-rate decomposition and Simpson's paradox.** I must be able to write $R = \sum_i w_i \cdot r_i$ and derive $\hat{R} = \text{mix} + \text{rate} + \text{interaction}$ ($\hat{R} = \sum_i \hat{w}_i \cdot \hat{r}_i(t_0) + \hat{\epsilon}$

$w_i(t_0) \hat{=} \hat{r}_i + \hat{E} \hat{=} w_i \hat{=} \hat{r}_i$) on a whiteboard, and explain *why* drilling rate effects (real behavior) before mix effects (composition artifacts) dissolves Simpson's paradox. The clinching detail: rates must be computed as `SUM(num)/SUM(den)` after grouping, never an average of per-segment ratios â€” that re-aggregation discipline is the defense.

- **Sessionization.** GA4 ships no session row; I reconstruct it from events sharing `(user_pseudo_id, ga_session_id)`, with the canonical hashed `session_key`. I must explain why this matters: get sessionization wrong and every downstream number is silently wrong, which is why it's a tested keystone.
- **The grounding boundary.** Why the LLM never authors SQL (semantic-mcp) and never computes a statistic (stats-mcp), and why that boundary is *structurally* enforced via MCP tool allow-lists rather than just asked-for-nicely in a prompt.
- **Experiment power and MDE.** I must define MDE, explain the power/ $\hat{I} \pm$ /baseline-rate/sample-size relationship, and connect it to `experiment-mcp.power_analysis` and `runtime_estimate` â€” every prescribed card is powered, not guessed.
- **The eval methodology.** Offline labeled benchmark: inject known rate-only or mix-only perturbations into a frozen GA4 copy, make the pipeline rediscover them, grade root-cause accuracy and decomposition MAPE. I must explain *why offline* (the real data's anomalies are unlabeled) and what the 85%-vs-45% claim actually measures.
- **Why determinism matters.** All math runs as seeded Python in stats-mcp, not in token-space, because reproducibility and auditability are the trust contract â€” an executive decision cannot rest on a number an LLM might generate differently next run.

Concepts you can explain at a high level

Know enough to be credible and to say "I'd reach for X" â€” but don't oversell mastery I don't have.

- **Prophet internals.** Additive decomposition (trend + seasonality + holidays) with a piecewise-linear trend and Fourier-series seasonality; I use it via `stats-mcp.forecast` for expected-vs-actual residuals. I can name the alternative (pmdarima/auto-ARIMA) and the tradeoff (Prophet is robust and low-tuning; ARIMA can be tighter on stationary series).
- **MCP wire protocol.** A JSON-RPC client-server protocol; stdio transport for the in-process tool servers, streamable-HTTP for warehouse-mcp; the value is schema-typed tool calls that make the grounding boundary physically enforceable. I don't need to recite the spec.
- **dbt incremental internals.** `insert_overwrite` partitioned by `event_date`, clustered by `device_category`, `channel_group`, which gives idempotent re-runs over a partition. I can explain *why* (cost + no duplicate diagnoses) without reciting every dbt macro.
- **Vector-store internals.** Embeddings of prior diagnoses for similarity recall in the memory store; I understand ANN/cosine-similarity retrieval conceptually. I won't pretend to have

benchmarked HNSW parameters.

Likely weak spots

I name these *before* the interviewer does â€” it reads as maturity.

- **Live A/B vs. observational.** The dataset is observational and historical (Nov 2020â€”Jan 2021). Helios **designs and sizes** experiments and runs quasi-experimental readbacks (pre/post, difference-in-differences); it does **not** run live A/Bs. I say this proactively.
- **Causal inference.** The decomposition is *correlational* attribution, not causal identification. The honest framing: Helios separates composition from behavior and the Critic flags confounds, but true causal claims (DiD with parallel-trends checks, synthetic control, double-ML) are the P4 frontier, gated on data that supports them.
- **Cost at scale.** Per-run bytes and tokens are a real constraint; the byte budget and Opus/Sonnet split are mitigations, and warehouse-agnostic per-tenant budgets are roadmap, not done.
- **Cross-device identity.** `user_id` is almost always null, so identity is cookie-grain (`user_pseudo_id`); user-level metrics are explicitly cookie-based approximations and stitching is unsound on this data.
- **Statistical depth under probing.** If pushed past the surface (e.g., multiple-comparisons correction across many segments, choice of significance test for proportions, confidence intervals on decomposition terms), I show I know the *direction* of the right answer and where it lives in stats-mcp, rather than bluffing a derivation.

Interviewer traps

Pre-scripted disarms for the predictable provocations.

- **"So it's just a chatbot."** No â€” conversation is a *secondary* drill-down surface. The product is the autonomous scheduled run that produces a Decision Brief proactively. A chatbot is reactive and ungoverned; Helios is proactive, governed, and graded.
- **"85% isn't production-grade."** Correct, and the comparison is the point: it's 85% vs a **45%** naive baseline on a labeled benchmark, gated in CI so accuracy can't silently regress. The number is a floor that ratchets up, and the Critic + reconciliation catch the failures that slip past raw accuracy.
- **"Why not just pandas?"** Pandas in an agent's hands means the LLM writing transformation code â€” back to ungoverned generation and silent errors. stats-mcp wraps `scipy/statsmodels/prophet` behind a *typed, deterministic, seeded* tool surface so the math is auditable and the LLM can only *call* it, never author it.
- **"Show me the SQL the LLM wrote."** Trick question â€” the LLM never wrote SQL. `semantic-mcp.build_query` composes it from the governed registry; I can show the

governed SQL and trace every column to the GA4 schema, which is exactly the zero-hallucination guarantee.

- **"You've mixed up mix and rate."** I slow down and re-derive: $\text{mix} = \hat{\mathbb{E}} \hat{\mathbb{P}}_{w_i} \cdot r_i(t_0)$ (composition moved, behavior fixed), $\text{rate} = \hat{\mathbb{E}} w_i(t_0) \cdot \hat{\mathbb{P}}_{r_i}$ (behavior moved, composition fixed). If I can re-derive it live, the trap becomes a strength.
- **"You said you run experiments."** I correct it immediately: I *design and size* experiments and run *quasi-experimental* readbacks; the data is observational, so claiming live A/Bs would be the dishonest answer.

Final preparation checklist

A tickable list tuned for placement season across Product/Growth/Data Analytics, Analytics Engineering, and AI/Data roles.

- ☐ **Rehearse the 60-second pitch and the 2-minute deep-dive** until they're muscle memory; lead with the autonomous-run + mix-vs-rate hook, not the tech stack.
 - ☐ **Rehearse the four STAR stories** (the mix-shift insight; trust via the eval harness; the MCP tool-boundary; the `traffic_source` first-touch gotcha) – each lands a different competency.
 - ☐ **Memorize from the Fact Sheet:** the seven funnel stages in order; the seven agents and their models (Opus: Orchestrator/Diagnose/Critic; Sonnet: Monitor/Decompose/Prescribe/Narrator); the five MCP servers and which is the *only* path to SQL vs math; the ten channel groups.
 - ☐ **The one diagram to draw cold:** the seven-agent plan-execute-critique loop with the Critic as a gate, the five MCP servers as the trust boundary, the funnel, and the identity $\hat{\mathbb{P}}_R = \text{mix} + \text{rate} + \text{interaction}$.
 - ☐ **The three numbers to never forget:** 85% vs 45% (root-cause accuracy vs naive baseline); 0 hallucinated columns; <5 min per run under a 5 GiB byte budget.
 - ☐ **Role-tune the opener:** for Product/Growth lead with revenue-at-risk and the prescribed experiment backlog; for Analytics Engineering lead with the dbt DAG, the semantic registry, and reconciliation; for AI/Data lead with the MCP grounding boundary, the agent FSM, and the eval gate.
 - ☐ **Pre-arm every weak spot and trap above** so each becomes a maturity signal instead of a stumble.
 - ☐ **Mock-interview targets:** at least three full mocks – one technical-deep (decomposition + sessionization + eval), one founder/commercial (Section 12 questions), one behavioral (the STAR stories) – with someone allowed to interrupt and push back.
-